

rapiddweller-benerator-ce

manual

rapiddweller GmbH & Volker Bergmann

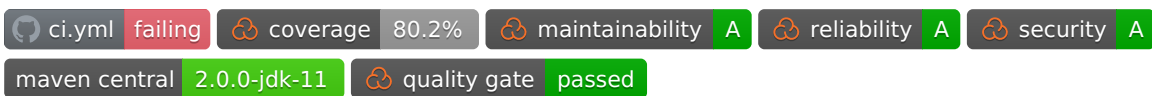
Copyright © 2006 - 2021 rapiddweller GmbH & Volker Bergmann

Table of contents

1. rapiddweller BENERATOR	3
2. Introduction to Benerator	5
3. Installation	11
4. The Benerator Project Wizard	14
5. Quick tour through the descriptor file format	17
6. Data Generation Concepts	24
7. Data Anonymization Concepts	47
8. Regular Expression Support	52
9. Data File Processing	54
10. Generating XML Files	58
11. Distribution Concepts	63
12. Using Relational Databases	66
13. Message Queue Access (Enterprise Edition)	80
14. Kafka Access (Enterprise Edition)	82
15. Environment Files	85
16. Advanced Topics	87
17. Generating Unique Data	98
18. Scripting	101
19. rapiddwellerScript	103
20. Command Line Tools	107
21. Domains	113
22. Component Reference	121
23. Using DB Sanity	140
24. Maven Benerator Plugin	141
25. Extending Benerator	145
26. Using Benerator as Load Generator	151
27. Troubleshooting	152
28. Monitoring Benerator	157
29. Benerator Performance Tuning	159
30. Migrating Benerator Projects	164



Benerator, the smart way to handle data



1. rapiddweller BENERATOR

rapiddweller Benerator is a software solution to **generate**, **obfuscate** (**anonymize** / **pseudonymize**) and **migrate** data for development, testing and training purposes.

1.1 Summary

rapiddweller Benerator allows creating realistic and valid high-volume test data, used for testing (unit/integration/load), training and showcase setup.

1.1.1 Generate data synthetically

Describe your data model on the most abstract level. Involve your business people or tester as no developer skills are necessary
Write your own benerator extensions in Javascript, Python or Java Integrate your data generation processes into any CI Pipeline

1.1.2 Mask and obfuscate sensitive production data

Define processes to anonymize or pseudonymize data on abstract level. Stay GDPR compliant with your data and protect the privacy of your customers. anonymize sensitive data for BI, test, development or training purposes Combine data from various sources (subsetting) and keep the data integrity

1.1.3 Migrate data

Migrate and transform your data in multisystem landscapes. Reuse your testing data models to migrate production environments. Keep your data consistent and reliable in a microsystem architecture

1.1.4 Requirements

rapiddweller Benerator is built for Java 11.



Note

If you need support for Java 8 or earlier, please consider using the versions `<= 1.0.1`.

1.2 Preface of this manual

This document is supposed to become a complete summary of everything you need of benerator usage, use it efficiently and extend it as you need. This reference is under construction and will update from time to time. Feel free to contribute your ideas in our repo at:

<https://github.com/rapiddweller/rapiddweller-benerator-ce/>

If problems remain unsolved after reading this doc, do not hesitate to contact us for help. rapiddweller-benerator-ce is and remains open-source and is provided for free.

If you are interested in additional support, and our premium features, we encourage you to check the website www.benerator.de. We offer additional services to make your data generation project a success and provide detailed use cases to ease your start into more complex scenarios.

Since you can do quite a lot of different things with Benerator but surely are interested in just a part of it, here's some guidance:

Goals and Features, introduces you to the goals and features of Benerator.

Find advice on how to install a binary distribution and how to get the sources and set up an IntelliJ/Eclipse project for using, debugging, and customizing Benerator in **Installation**.

Data Generation Concepts, **Descriptor File Format** and **Advanced Topics** then provide you with a structured and complete introduction into the Benerator descriptor file setup.

Benerator supports a multitude of service provider interfaces (SPIs). It comes along with some implementations for specific business domains (**Domains**) and general-purpose classes in **Component Reference**.

Finally, you are instructed how to write custom SPI implementations in **Extending Benerator**.

2. Introduction to Benerator

2.1 Goals

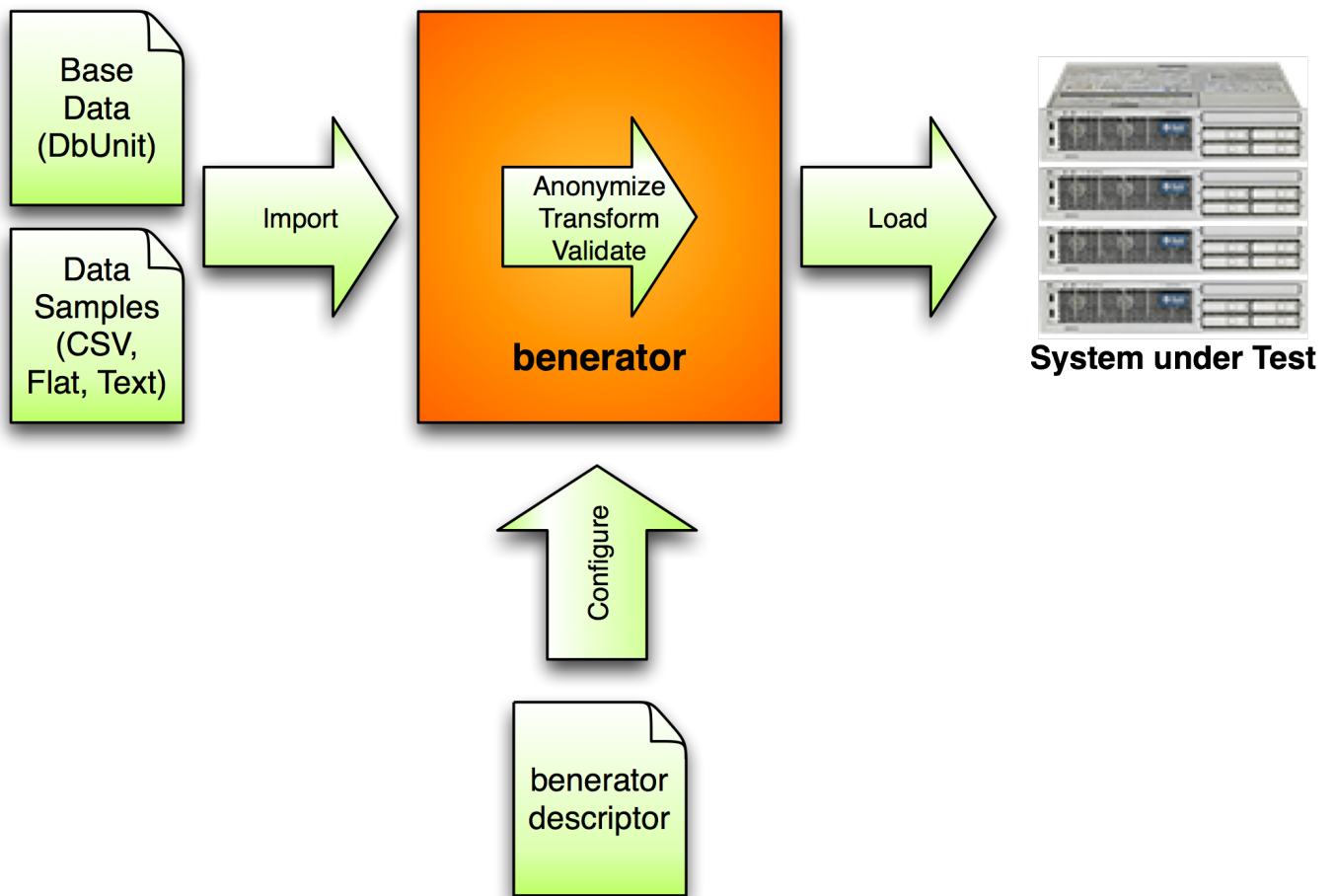
The core goals of Benerator are

- Generation of data that satisfies complex data validity requirements
- Anonymization of production data for showcases and serious performance testing projects
- Efficient generation of large data volumes, scaling up to companies with billions of customers and Big Data projects
- Early applicability in projects
- Little maintenance effort with ongoing implementation through configuration by exception
- Wide and easy customizability
- Applicability by non-developers
- Intuitive data definition format
- Satisfying stochastic requirements on data
- Extraction and anonymization of production data
- Supporting distributed and heterogeneous applications
- Establishing a common data generation platform for different business domains and software systems

2.2 Features

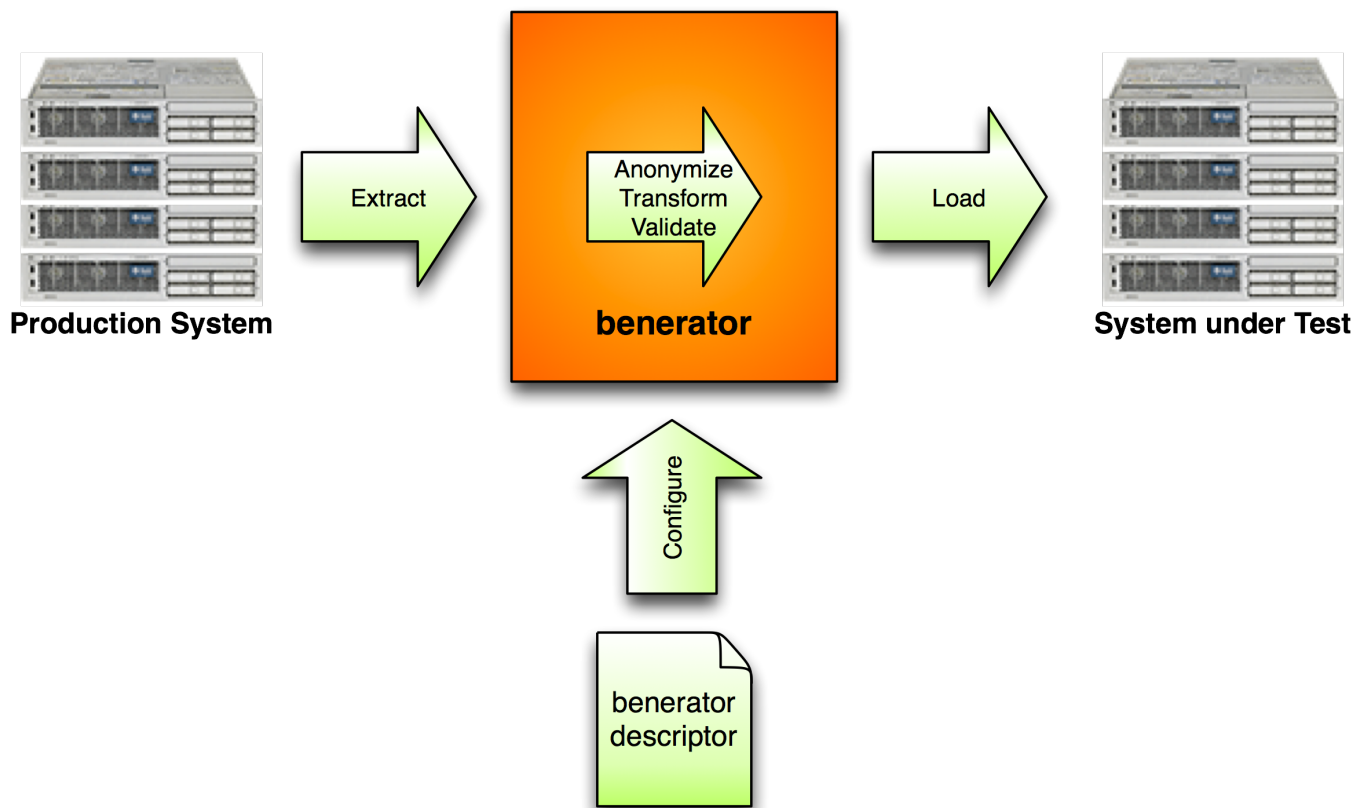
2.2.1 Data Synthesization

Performance test data can be completely synthesized. A basic setup can be imported e.g. from DbUnit files, CSV files and fixed column width files. A descriptor file configures how imported data should be processed and adds completely synthesized data. The processed or generated data finally is stored in the system under test.



2.2.2 Production Data Anonymization

Production data can be easily extracted from production systems. Tables can be imported unmodified, filtered, anonymized and converted.

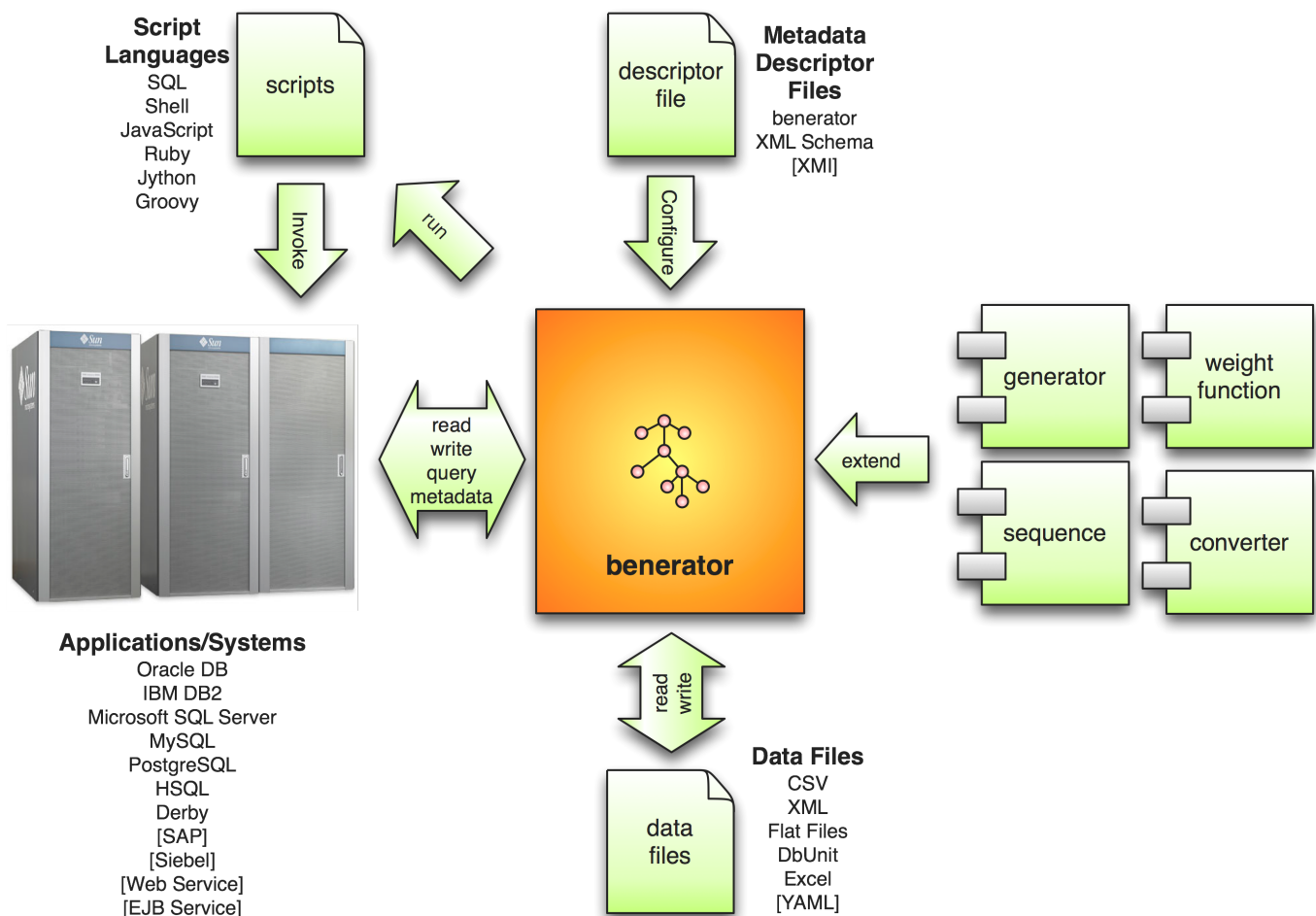


2.3 State of the Benerator

Benerator is developed and continuously extended and improved since June 2006. Benerator is mainly used and tested best for the data file and database data generation, for these applications Benerator should help you with almost all your data generation needs out of the box - and extending Benerator for specific needs is easy.

XML-Schema, on the other hand, allows for an extraordinarily wide range of features. Benerator's XML support is limited to features that are useful for generating XML data structures (no mixed content) and does not yet support all variants possible with XML schema. The elements `<unique>`, `<key>` and `<keyRef>` cannot be handled automatically, but require manual configuration. The following features are not yet implemented: `<group>`, `<import>`, `<all>` and `<sequence>` with `minCount != 1` or `maxCount != 1`. If you need support for some of these, please [contact us](#).

2.4 Building Blocks



2.5 Database Support

All common SQL data types are supported.

Benerator was tested with and provides examples for

- Oracle 19c (thin driver)
- DB2
- MS SQL Server
- MySQL 5
- PostgreSQL 12
- HSQL 2.x
- H2 1.2
- Derby 10.3
- Firebird

2.6 Benerator Editions

Benerator comes in different editions which differ by feature set, scalability and performance:

2.6.1 Performance Comparison

The results below show Benerator's generation and anonymization performance on a plain MacBook Air (2020) with standard equipment and Azul Java Virtual Machine (CE = Community Edition, EE = Enterprise Edition):

Benchmark	CE 1.1.2	CE 2.0.0	EE 2.0.0 / 1 Thread	EE 2.0.0 / 4 Threads
gen-string.ben.xml	37	58	336	1,095
gen-person-showcase.ben.xml	26	119	111	327
anon-person-showcase.ben.xml	31	120	113	328
anon-person-regex.ben.xml	346	537	838	1,381
anon-person-hash.ben.xml	386	500	1,299	1,287
anon-person-random.ben.xml	576	838	1,514	1,736
anon-person-constant.ben.xml	2,210	2,745	2,646	2,162

Note

The numbers are million entities generated/anonymized per hour.

Compared to CE 1.1.2's generation engine,

CE 2.0.0 is **1.5-2 times faster**

and EE 2.0.0 with 4 threads is roughly **4 times faster**

and **scales further** with the number of CPUs on your machine.

2.6.2 Benerator Community Edition (CE)

Benerator started as an open-source project and is committed to further improve and extend with and from the feedback of its user base and its contributors. It is the most powerful open-source data generator and is competitive with all commercial products.

However, it has two historic limitations:

- No neat graphical user interface
- Only single-threaded generation and anonymization

Though, Benerator Community Edition still has an impressive performance.

2.6.3 Benerator Professional & Full Edition

The rapiddweller Benerator Professional & Full Editions aim at providing an easy and effortless start for using Benerator and to generate realistic data in simple as well as complex scenarios. This includes:

- Benerator UI
- Ready-to-use templates
- Demos
- Best practices from countless benerator projects
- A powerful RESTful / JSON API
- Container support for Docker / Kubernetes
- And much more.

Benerator UI

Benerator UI is an integrated graphical development environment with

- Editing support by context-sensitive auto-completion
- Code snippets for many use-cases
- Templates for projects and files
- Project management and project collaboration
- Task management
- Data preview
- Log analysis and guidance on errors
- And much more.

2.6.4 Benerator Enterprise Edition (EE)

Extends Benerator Community Edition and improves it in many respects. With highly-optimized engine and generation-related components and with multithreaded execution support, its performance on a single machine is about 10x the performance of the Community Edition and Benerator can easily scale over multiple machines in your private cloud / cluster setup.

Improvements against the Community Edition are

- Improved performance on single threading
- Multithreaded data generation and anonymization
- Anonymization Reporting supports you in compliance checking
- Benerator UI: An integrated graphical development environment with editing support, project and task management
- JSON support
- JMS support: ActiveMQ, RabbitMQ and more
- Kafka support
- Industry modules: Logistics, Insurance, Finance, ...



Learn more about all editions and its features on www.benerator.de.

3. Installation

3.1 Download the distribution binary

Download Benerator from:

<https://github.com/rapiddweller/rapiddweller-benerator-ce/releases/>

You should download the most recent version of the rapiddweller-benerator-dist archive from the download page, e.g. *rapiddweller-benerator-ce-2.0.0-jdk-11-dist.zip*.



Warning

We highly recommend downloading a prebuild binary from our release tags as above to ease your start. If you clone our GitHub repository, there are no binaries included and you need to build benerator yourself. Building benerator requires a proper java/maven setup on your system. Learn more below [Development Quick Start](#).

3.2 Unzip Benerator

Unzip the downloaded file in an appropriate directory, e.g. `/Developer/Applications` or `C:\Program Files\Development`.

3.3 Set BENERATOR_HOME

Create an environment variable `BENERATOR_HOME` that points to the path you extracted benerator to.

Windows Details: Open the System Control Panel, choose Advanced Settings - Environment Variables. Choose New in the User Variables section. Enter `BENERATOR_HOME` as name and the path as value (e.g. `C:\Program Files\Development\rapiddweller-benerator-ce-2.0.0`). Click OK several times.

Mac/Unix/Linux Details: Open the file `.profile` in your user directory. Add an entry that points to benerator, e.g.: `export BENERATOR_HOME=/Developer/Applications/rapiddweller-benerator-ce-2.0.0`

3.4 Optional: Install JDBC drivers

Benerator comes with open-source JDBC drivers (for connecting to a database). No extra installation is necessary for them:

- Microsoft SQL Server Driver (MS SQL Server)
- MySQL Connector
- HSQL DB
- H2 DB
- Derby Client
- PostgreSQL
- Jaybird (Firebird DB)

However, if you need to use a closed source database driver, the vendor usually requires you to accept usage conditions before you can download and install their driver. So, if you are using Oracle DB or DB2, get the JDBC drivers from these sites:

- Oracle http://www.oracle.com/technology/software/tech/java/sqlj_jdbc/index.html: Click on "Oracle Database 11g Release 2 (11.2.0.1.0) drivers". Download `ojdbc6.jar` and put it into Benerator's lib directory. You will need to create a free Oracle account.
- DB2 <http://www-306.ibm.com/software/data/db2/java>: Follow the link 'IBM Data Server Driver for JDBC and SQLJ', download the driver archive and copy the contained file `db2jcc.jar` to Benerator's lib directory.

3.5 Optional: Set up log4j2

Troubleshooting is simpler if you make use of log4j's configuration capabilities. If you do not know (or care) about logging, simply skip this step. Otherwise, put a custom log4j2.xml file into the BENERATOR_HOME/lib directory.

3.6 Mac OS X configuration

On Mac OS X you need to provide benerator with an explicit configuration of the JAVA_HOME path. See <http://developer.apple.com/qa/qa2001/qa1170.html> for a good introduction to the OS X way of setting up Java. It is based on aliases conventions. If you are not familiar with that, you should read the article. If Java 6 (or newer) is the default version you will use, you can simply define JAVA_HOME by adding the following line to your .profile: in your user directory:

```
export JAVA_HOME=/Library/Java/Home
```

If it does not work or if you need to use different Java versions, it is easier to 'hard-code' JAVA_HOME like this:

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/adoptopenjdk-11.jdk/Contents/Home/
```

3.7 Verifying the settings

On any OS, open a console window (shell,cmd or powershell) and type ...

```
$ benerator --version

Benerator Community Edition 2.1.0-jdk-11
Java version: 11.0.11
JVM product: OpenJDK 64-Bit Server VM 11.0.11+9 (AdoptOpenJDK)
System: Mac OS X 10.16 x86_64
CPU & RAM: 8 cores and 8 GB RAM, max 2 GB of RAM for this process
```

Benerator will then launch and print out version information about itself, the Java version it uses, the operating system.

3.8 Development Quick Start (Advanced)

If you want to start development or use the maven project to build Benerator by yourself, on Linux or Mac OS X you can also try the quickstart by using the helper scripts. Checkout the project from <https://github.com/rapiddweller/rapiddweller-benerator-ce/>.



Following instructions only work on Linux or Mac OS X. Scripts are not supported on Windows.



It might be required to run the scripts with `sudo`.

If you want to use the command `benerator` in your shell session, you have to execute `source script/2_setup_benerator.sh`

If you want to install Benerator permanently into your system, you have to modify your environment file or your `~/.profile` and add ENV variable `BENERATOR_HOME` and `PATH=$BENERATOR_HOME/bin:$PATH`

to execute the scripts you can do ...

```
$ bash script/1_install_mvn_dependencies.sh
$ source script/2_setup_benerator.sh
$ bash script/3_execute_demos.sh
```

... alternatively, you can also set execute permissions like ...

```
$ chmod +x script/1_install_mvn_dependencies.sh
$ chmod +x script/2_setup_benerator.sh
$ chmod +x script/3_execute_demos.sh
```

... and execute scripts like this

```
$ .script/1_install_mvn_dependencies.sh
$ .script/2_setup_benerator.sh
$ .script/3_execute_demos.sh
```

- *1_install_mvn_dependencies.sh* : This script is checking prerequisites for you, cloning all rapiddweller-benerator-ce sub-projects and installing them via maven locally.
- *2_setup_benerator.sh* : This script is building on script no. 1 and using installed dependencies and packed jar, assembling it to a rapiddweller-benerator-ce.tar.gz and is setting up Benerator locally into your user home directory.
- *3_execute_demos.sh* : This script is building on script no. 2 and is using the unpacked and configured rapiddweller-benerator-ce application to execute existing demo files.

4. The Benerator Project Wizard

The Benerator Project Wizard helps you to easily and quickly set up any kind of Benerator project:

- Descriptor file-driven data generation
- XML Schema-driven XML file generation
- Creation of database snapshots

4.1 Starting the wizard

Start the Project Wizard on the command line by typing

[on Windows:]

```
benerator-wizard
```

[or Mac/Unix/Linux/Solaris:]

```
benerator-wizard
```

Depending on your language settings, the GUI welcomes you with an English or German dialog (contributions of translations are welcome). You can override the default language settings by using Java's `user.language` system property, e.g.

```
benerator-wizard -Duser.language=en
```

The dialog looks like this:

Visit www.benerator.de for ...

- * user friendly UI to use benerator
- * smart autocompletion
- * project management and setup
- * improved error handling
- * extended examples
- * multithreading
- * db connection pooling

Project Name: Project Type:

Project Folder:

(Maven) Group Id: Version Number:

Options: ☐ Create Eclipse Project (requires Maven) ☐ Overwrite ☐ Offline

Database Product: Driver:

URL: Schema:

User: Password:

Catalog:

Snapshot:

'create tables' script: 'drop tables' script:

Encoding: Line Separator:

Locale: Dataset/Country:

4.2 Configuring the project

Enter the project name and choose an appropriate directory.

Then you can choose among a list of predefined project types:

- **Hello World example:** Prints 'Hello World'-style output to the console
- **Simple Project:** Simple project definition for custom generation projects
- **Generate CSV file:** Defines generation of a simple CSV file
- **Populate database:** Small project that defines a database, creates a table, and populates it
- **Shop example database:** Performs Definition and population of the Benerator shop database example for all database systems supported by Benerator: DB2, Derby, Firebird, HSQL, H2, Oracle, PostgreSQL and SQLServer.

You can optionally make the wizard create an Eclipse project configuration, too. This requires you to have Maven installed.

If necessary, specify the database connection settings and test if they are right by clicking `Test Connection`.

When reproducing an existing database or creating a database snapshot, choose a snapshot file format: `DBUnit file`, `Excel(TM) document`, or `SQL file`. In all other cases, choose `None`.

When reproducing an existing database, you need to provide Benerator with the DDL files for creating and dropping the related tables and sequences ('create tables' script and 'drop tables' script).

Finally, you can explicitly choose system-dependent settings. If you leave these fields empty, Benerator will always take the individual settings of the system it is executed on.

4.3 Creating and running the project

Finally, press `Create` for creating the project. The wizard then configures the project in the specified project folder.

Look into the project folder and read the `readme.txt` file. This contains individual instructions for your project: What further configuration steps are eventually necessary (e.g. proprietary database drivers), how to invoke Benerator for this special project type, and how to go on with customizations.

The easiest way to run the generated project is to run the Maven Benerator Plugin as described in the generated `readme.txt` files. The project wizard creates all necessary files for you to execute the created projects immediately (except for proprietary database drivers: Oracle or DB2). However, you can also invoke Benerator directly from the command line, but you have to take care of compiling custom Java classes in the project for yourself.

5. Quick tour through the descriptor file format

5.1 <setup>

The benerator configuration file is XML based. An XML schema is provided. The document root is a setup element:

```
<?xml version="1.0" encoding="utf-8"?>
<setup xmlns="https://www.benerator.de/schema/2.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://www.benerator.de/schema/2.0.0 https://www.benerator.de/schema/rapiddweller-benerator-ce-2.0.0.xsd">
  <!-- content here -->
</setup>
```

benerator descriptor files are supposed to be named `benerator.xml` or end with the suffix `.ben.xml`.

5.2 benerator properties

Several global benerator properties allow for customization of its behavior:

name	description	default setting
defaultEncoding	the default file encoding to use for reading and writing text files	the system's file encoding
defaultLineSeparator	the line separator to use by default	the system's line separator
defaultTimeZone	The time zone to use	The system's time zone
defaultLocale	The locale to use if none has been specified explicitly	The system's language code, e.g. 'de'
defaultDataset	The dataset to use if none has been specified explicitly	The system's country's two-letter ISO code, e.g. 'US'
defaultPageSize	the number of entities to create in one 'run', typically a transaction	1
defaultScript	The default script engine to use for evaluating script expressions	ben (rapiddwellerScript)
defaultNull	tells if nullable attribute should always be generated as null by default	true
defaultSeparator	the default column separator to use for csv files	,
defaultErrorHandler	the default error handling mechanism to use	fatal
validate	Boolean flag to turn off validation (e.g. of XML validity and type definition consistency).	true
maxCount	limits the maximum cardinality of all entity and association generations. If set to 0, cardinalities will not be limited.	-1
defaultOneToOne	When set to true Benerator assumes each relation is one-to-one.	false
acceptUnknownSimpleTypes	When set to true, Benerator accepts unknown simple data types from its DescriptorProviders, relying on the user to choose the correct data type when generating.	false

You can configure them in the `<setup>` element, e.g.

```
<setup xmlns=...
  defaultEncoding="utf-8"
  defaultPageSize="1000">
```

5.3 <include>

5.3.1 Inclusion of properties files

An alternative way to specify the Benerator properties from the previous chapter is to specify them in a properties file, e.g.

```
context.defaultEncoding=UTF-8
context.defaultPageSize=1000
```

and include the properties file in the benerator descriptor file:

```
<include uri="my.properties"/>
```

This way you can easily use different settings in different environments (see '[Staging](#)').

File entries that do not begin with 'benerator' are simply put into the generation context and can be used to configure generation behavior.

5.3.2 Sub-Invocation of descriptor files

Besides properties files, Benerator descriptor files can be included too, e.g.

```
<include uri="subgeneration.ben.xml"/>
```

5.4 Global settings

benerator supports global settings. They can be evaluated using script expressions, e.g. {user_count}. This way, different types of settings may be evaluated:

- system environment
- Java virtual machine parameters
- context variables

A setting is explicitly defined using a setting element:

```
<setting name="threshold" value="5"/>
```

5.5 <import>

Benerator has lots of plugin interfaces but is agnostic of most implementors. So you need to explicitly import what you need.

The following packages are imported by default (providing, for example, the ConsoleExporter):

com.rapiddweller.benerator.consumer	General-purpose consumer classes
com.rapiddweller.benerator.primitive	Generators for primitive data types
com.rapiddweller.benerator.primitive.datetime	Generators for date, time and timestamp data
com.rapiddweller.benerator.distribution.sequence	Distributions of 'Sequence' type
com.rapiddweller.benerator.distribution.function	Distributions of 'Function' type
com.rapiddweller.benerator.distribution.cumulative	Distributions of type 'CumulativeDistributionFunction'
com.rapiddweller.benerator.sample	Generator components that use sample sets or seeds
com.rapiddweller.model.consumer	ConsoleExporter and LoggingConsumer
com.rapiddweller.common.converter	Converter components from rd-lib-common
com.rapiddweller.common.format	Format components from rd-lib-common
com.rapiddweller.common.validator	Validator components from rd-lib-common
com.rapiddweller.platform.fixedwidth	Fixed column width file importer and exporter
com.rapiddweller.platform.csv	CSV file importer and exporter
com.rapiddweller.platform.dbunit	DbUnit file importer and exporter
com.rapiddweller.platform.xls	Excel(TM) Sheet importer and exporter

Benerator extensions can be bundled as domains (logical extensions) or platforms (technical extensions). You can export different bundles as comma-separated lists:

```
<import domains="address, net"/>
<import domains="organization"/>
<import platforms="csv, db"/>
```

Imports must be the first elements used in a descriptor file.

When using a Benerator plugin or another library, you need to make sure that Benerator finds its binary. There are three alternatives:

1. Putting the associated jar file(s) into the lib folder of your Benerator installation. This way it is available for all data generation projects on your machine. If you work in a team where everyone is familiar with Benerator and the toolset is not based on Maven, this is generally the preferred approach.
2. Create a subfolder named lib under the data generation project folder and put the jar file(s) there. When distributing the project to be executed on machines with plain Benerator installations, distribute the full folder content including the lib subfolder.
3. When using Maven to run Benerator, simply create the necessary Maven dependencies and Maven will acquire all needed libraries dynamically. Read more about this in '[Maven Benerator Plugin](#)'

5.6 <generate>

<generate> elements are used to generate data from scratch. There are lots of configuration options. The minimal configuration specifies the type of data to be generated. For now, all generated data are 'entities' (composite data).

```
<generate type="Person" count="10" consumer="ConsoleExporter"/>
```

This will make Benerator generate 10 'Person' Entities and send them to a ConsoleExporter that prints out the persons to the console. But what is a Person? Benerator will figure it out by itself if it knows e.g. a database with a 'PERSON' table, an XML schema with a 'Person' element, or any other 'DescriptorProvider'. Benerator will generate database-valid or XML-Schema-valid data automatically. More about this later.

Let us start without DescriptorProviders, manually putting together what we need.

Entities consist of members, e.g. `<attribute>`s, `<id>`s or `<reference>`s. I will concentrate on attributes in the following sections and explain ids and references later.

5.6.1 "constant"

The simplest way to define data generation is using the same value for all generated data:

```
<generate type="Person" count="10" consumer="ConsoleExporter">
  <attribute name="active" type="boolean" constant="true"/>
</generate>
```

So we define, that all Person entities are generated with an 'active' attribute of type 'boolean' that is set to 'true'.

5.6.2 "values"

Attributes may be randomly set from a list of comma-separated values

```
<generate type="Person" count="10" consumer="ConsoleExporter">
  <attribute name="firstName" type="string" values="Alice', 'Bob', 'Charly'"/>
  <attribute name="rank" type="int" values="1,2,3"/>
</generate>
```

So we define, that Person entities have a 'firstName' attribute that is 'Alice', 'Bob' or 'Charly' and a rank of 1, 2 or 3. Note that string literals must be 'quoted', while number or Boolean literals do not.

5.6.3 "pattern": Generation by Regular Expression

String attribute generation can be configured using the "pattern" attribute with a regular expression, for example:

```
<generate type="Person" count="10" consumer="ConsoleExporter">
  <attribute name="salutation" type="string" pattern="(Mr|Mrs)"/>
  <attribute name="postalCode" type="string" pattern="[1-9][0-9]{4}"/>
</generate>
```

You can find a detailed description of Benerator's regular expression support in [Regular Expression Support](#).

5.7 <iterate>

The `<iterate>` element is used to iterate through pre-existing data, e.g. in a data file or database. The general form is

```
<iterate type="Person" source="persons.csv"/>
```

which iterates through all Persons defined in a CSV-file called 'persons.csv'.

Note

By default, iteration goes once from beginning to the end. Consider using the parameter `cyclic="true"` for iterating repeatedly and check this manual for applying distributions or filter the data to iterate through. Learn more in [Relational Databases->Determining attribute values](#)

5.7.1 "offset"

In whatever type of data generation or iteration, an **offset** can be applied to skip the heading entries of a data source, e.g.

```
<iterate type="Person" source="persons.csv" offset="10"/>
```

leaves out the first ten entries of the `persons.csv` file.

5.8 <echo>

The meaning of the `<echo>` element is similar to the `echo` command in batch files: Simply writing information to the console to inform the user what is happening, e.g.

```
<echo>Running...</echo>
```

For Mac OS X users there is a nice extra feature: When using `type='speech'`, Benerator uses Mac OS X's speech facility to speak the text. When executed on other operating systems, the text is only printed to the console:

```
<echo type="speech">Generation Finished</echo>
```

5.9 <beep/>

makes Benerator emit a short beep

5.10 <comment>

The `<comment>` element also prints output, not to the console, but to a logger. Thus you have the option of configuring whether to ignore the output or where to send it to.

```
<comment>`Here we reach the critical part...`</comment>
```

Using XML comments `<!-- -->` instead of comment descriptors would make it harder for you to comment out larger portions of a file for testing and debugging.

5.11 <execute type="shell">

The `<execute>` element serves to execute different kinds of code. One option is the execution of shell commands:

```
<execute type="shell">start-database.sh</execute>
```

The program output is printed to the console.



Note that some windows shell commands are only available in the command-line interpreter. In order to invoke them, you need to call `cmd /C`, e.g.

```
<execute type="shell">cmd /C type myGeneratedFile.csv</execute>
```

You can use `<execute>` for invoking scripts too (SQL, `rapiddwellerScript`, JavaScript, FreeMarker and more), but that will be explained later.

5.12 <wait>

The `<wait>` element makes Benerator wait for a fixed or a random amount of time.

A fixed amount of time is useful, e.g. for waiting until a system is initialized:

```
<wait duration="20000"/>
```

The duration is the time in milliseconds.

Random periods of wait time are useful when using Benerator to simulate client activity on a system. For this, you can nest `<wait>` elements in `<generate>` elements. More about this later.

5.13 <error>

You can make Benerator signal an error with a message and code:

```
<error code="-3">An error has occurred</error>
```



Note

If Benerator is not configured to do otherwise, it prints out the error message, cancels execution, finishes the process and returns the exit code to the operating system. If no exit code is specified, Benerator uses -1.

5.14 <if>

Evaluates a script expression and executes sub-elements depending on the result.

Either a decision to execute something or not:

```
<if test="com.rapiddweller.common.SystemInfo.isWindows()">
  <echo>Running under Windows</echo>
</if>
```

or a decision between alternatives:

```
<if test="com.rapiddweller.common.SystemInfo.isWindows()">
  <then>
    <execute type="shell">cmd /C type export.csv</execute>
  </then>
  <else>
    <execute type="shell">cat export.csv</execute>
  </else>
</if>
```

A typical application of the <if> element is to check if a required configuration is defined, and if not, to fall back to a default...:

```
<if test="!context.contains('stage')">
  <echo>No stage defined, falling back to 'dev'</echo>
  <setting name="stage" value="dev"/>
</if>
```

...or to report an error:

```
<if test="com.rapiddweller.common.SystemInfo.isWindows()">
  <error>No stage has been set</error>
</if>
```

5.15 <while>

The <while> element executes sub elements as long as a boolean 'test' expression resolves to **true**:

```
<generate type="test" count="10" consumer="ConsoleExporter">
  <id name="identifier" type="long"/>
  <while test="this.identifier == 5">
    <echo>add 10 to this.identifier</echo>
    <execute type="ben">this.identifier = this.identifier + 10</execute>
    <wait duration="1000"/>
  </while>
</generate>
```

5.16 <id> - Generating unique identifiers

For marking an entity member as an identifier, it is declared with an <id> element, e.g.

```
<id name="identifier" type="long"/>
```

There are several special id generators available. If you do not specify one explicitly, Benerator takes the `IncrementalIdGenerator`.

For explicitly choosing or initializing an id generator, use the generator attribute, e.g.:

```
<id name="identifier" type="long" generator="new IncrementalIdGenerator(100)"/>
```

for using an `IncrementalIdGenerator`, that starts with the value 100.

See [Common ID Generators](#) for a complete ID generator reference and [Using Relational Databases](#) for database-related id generators.

Instead of using a generator, you can as well use other `<attribute>`-like features, e.g. scripts:

```
<id name="id" type="long" script="parent.id"/>
```

5.17 Naming Conventions

For automatic support of special file content, the following naming conventions apply:

File Name	File Type
*.ben.xml	benerator descriptor file
*.dbunit.xml	DbUnit data field
*.csv	CSV file with data of simple type
*.ent.csv	CSV file with entity data
*.wgt.csv	CSV file with weighted data of simple type
*.fcw	Fixed column width files with entity data
*.set.properties	Dataset nesting definition

6. Data Generation Concepts

Now that you have mastered the first tutorials and have had a glance on Benerator's features, it is time for an in-depth introduction to data generation:

6.1 Naming

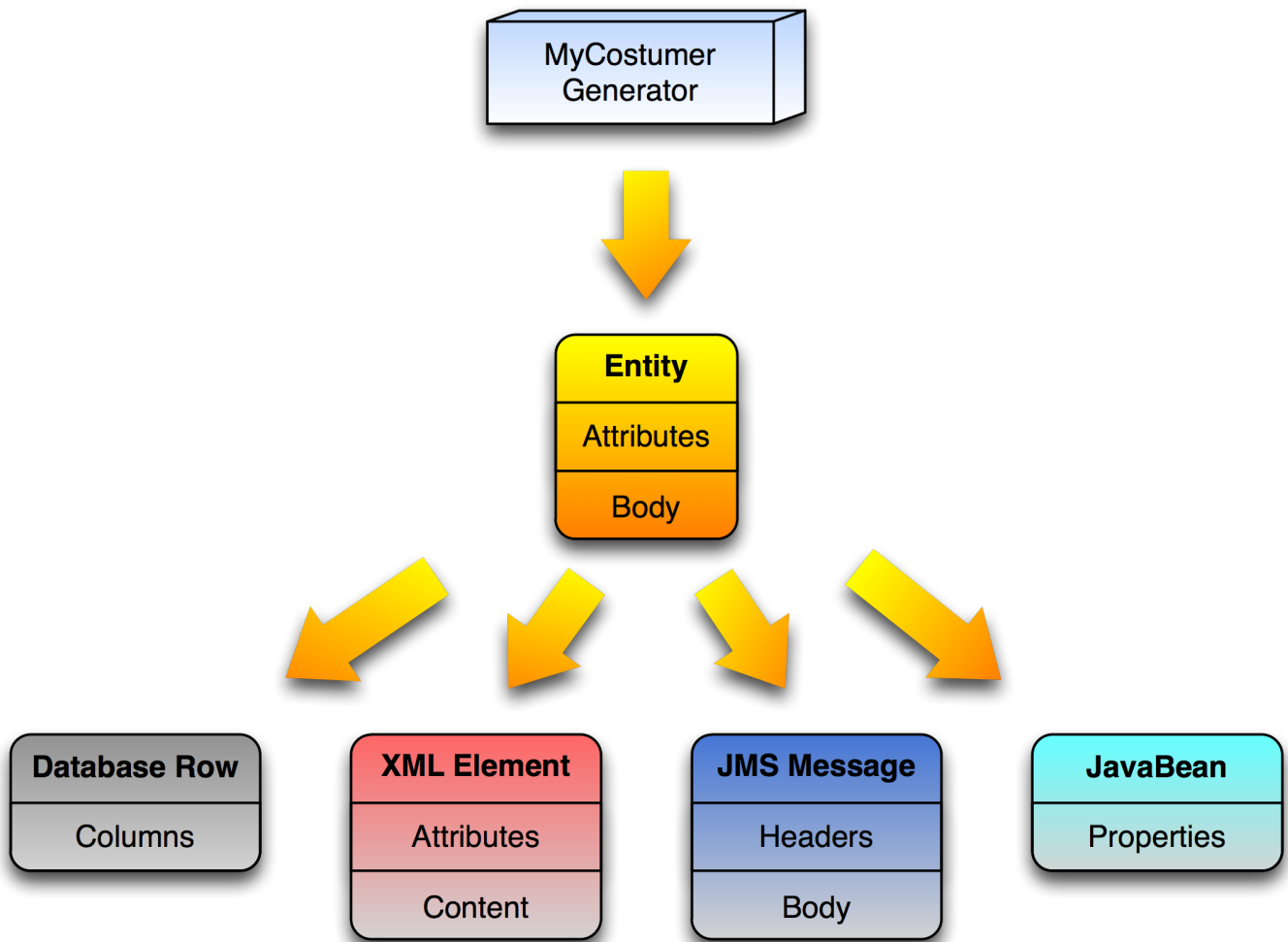
Business objects are called *entity* in this manual, their contained simple type data are *attributes*.

6.2 Entity Data

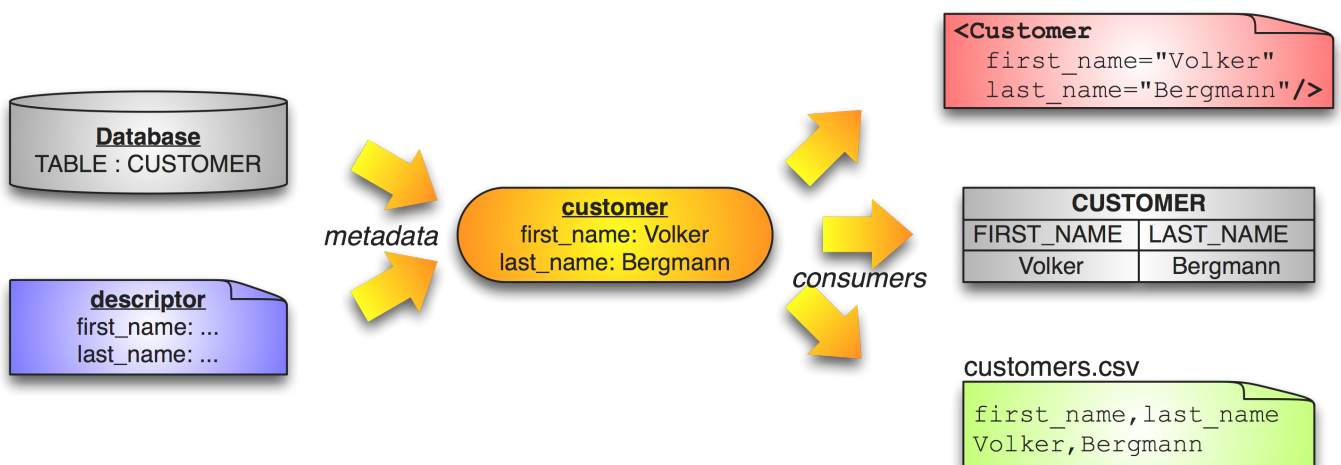
Benerator generates entities in a platform-independent manner (internally using the class `com.rapiddweller.model.data.Entity`). An entity will be interpreted individually depending on the target system. It can be mapped to

- relational data (DB)
- hierarchical data (XML)
- graphs (JavaBeans)
- attributed payload holders (File, JMS Message, HTTP invocations)

So you can use an abstract, generic way of defining and generating business objects and reuse it among colleagues, companies and target platforms. An address generator defined once can be used for populating customer tables in a database or creating XML order import batch files.



Metadata is platform-neutral too. So Benerator can import metadata definitions from a database and use it for generating XML data:



Entities can have

- an arbitrary number of simple-type attributes (like database tables or XML attributes). They can have a cardinality > 1 , too (like arrays of simple types)
- sub-components of entity type (like XML sub-elements)
- a body of simple type (like XML simpleType elements, files or JMS messages)

6.3 Simple Data Types

Benerator abstracts simple types too. These are the predefined simple types:

Benerator type	JDBC type name	JDBC type code	Java type
byte	Types.BIT	-7	java.lang.Byte
byte	Types.TINYINT	-6	java.lang.Byte
short	Types.SMALLINT	5	java.lang.Short
int	Types.INTEGER	-5	java.lang.Integer
big_integer	Types.BIGINT	-5	java.lang.Long
float	Types.FLOAT	6	java.lang.Float
double	Types.DOUBLE	8	java.lang.Double
double	Types.NUMERIC	2	java.lang.Double
double	Types.REAL	7	java.lang.Double
big_decimal	Types.DECIMAL	3	java.math.BigDecimal
boolean	Types.BOOLEAN	16	java.lang.Boolean
char	Types.CHAR	1	java.lang.Character
date	Types.DATE	91	java.lang.Date
date	Types.TIME	92	java.lang.Date
timestamp	Types.TIMESTAMP	93	java.lang.Timestamp
string	Types.VARCHAR	12	java.lang.String
string	Types.LONGVARCHAR	-1	java.lang.String
string	Types.CLOB	2005	java.lang.String
object	Types.JAVA_OBJECT	2000	java.lang.Object
binary	Types.BINARY	-2	byte[]
binary	Types.VARBINARY	-3	byte[]
binary	Types.VARBINARY	-4	byte[]
binary	Types.BLOB	2004	byte[]
(heuristic)	Types.OTHER	1111	(heuristic)

Oracle's `NCHAR`, `NVARCHAR2` and `NCLOB` types are treated as strings.

The following JDBC types are not supported: `DATALINK` (70), `NULL` (0), `DISTINCT` (2001), `STRUCT` (2002), `ARRAY` (2003), `REF` (2006). If you need them, create an issue or get in touch with the rapiddweller Benerator team.

6.4 Data Characteristics

6.4.1 Distribution Concept

There are two special issues that often remain unaddressed in testing:

- using realistic probability distributions (e.g. popularity of shop items)
- creating unique values (e.g. IDs or unique phone numbers for fraud checking)

For these purposes, rapiddweller Benerator provides several interfaces, which extend a common interface, Distribution. The most important ones are

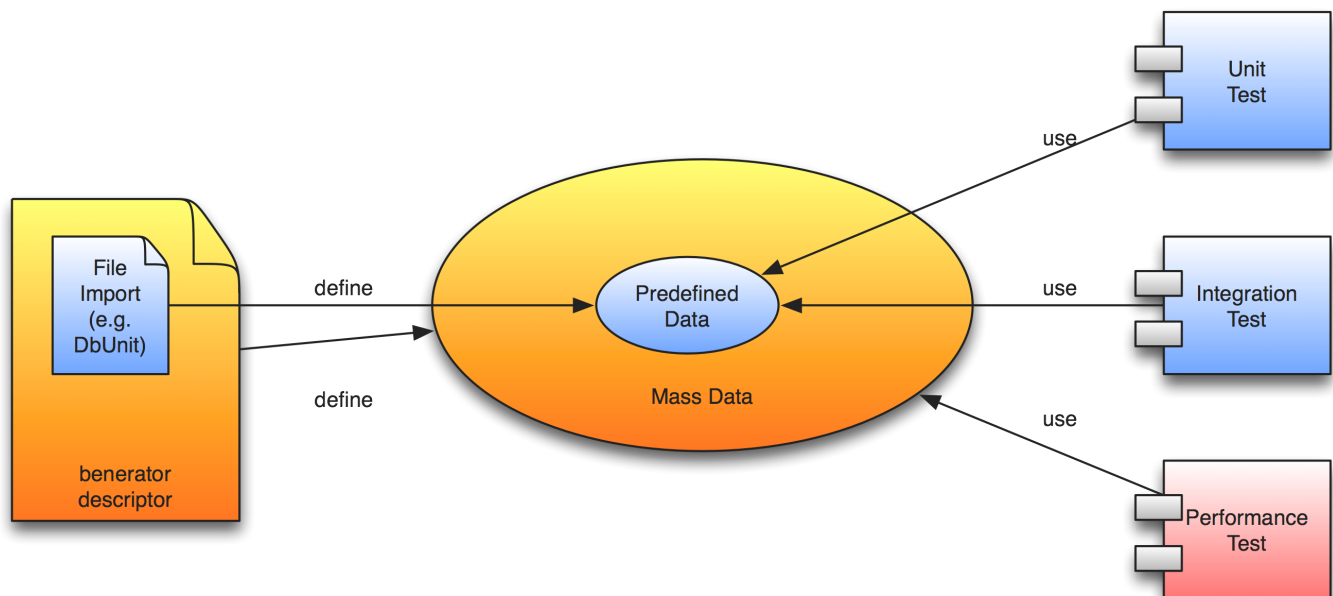
- WeightFunction
- Sequence

For a list of predefined distributions, see [Distributions](#).

6.5 Generation Stages

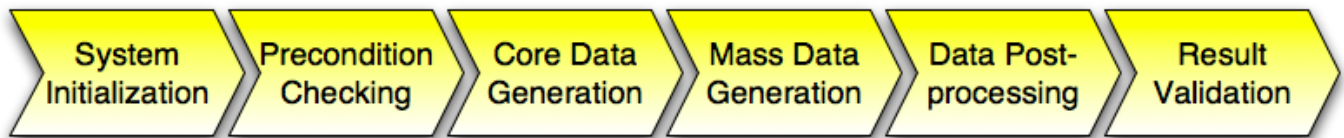
The result of data generation typically consists of

- a small predefined and well-known core data set
- large data volumes that are generated randomly and extends the core data



This approach has the advantage of supporting different test types with the same generation setup: It is essential for performance tests to have a unit- and integration-tested system, and you can strongly simplify the testing procedure by reusing data definitions from unit- and integration tests as core data for a mass data generation.

Regarding the technical steps involved, a generation process employs up to six stages for each system involved:



- **System Initialization**, e.g. by start scripts and DDL scripts
- **Precondition Checking** for verifying that data required for data generation is available
- **Core Data Generation** for creating a predefined data set
- **Mass Data Generation** for scaling data amounts of large volume
- **Data Postprocessing** for performing complex operations
- **Result Validation** for verifying the generated data

6.5.1 System Initialization Stage

In the system initialization stage you typically use scripts for starting and initializing the systems involved, e.g.

- start database by shell script
- run SQL script
- start application server

For starting a database with a shell script and initializing it with a SQL script, you could write:

```

<setup>
  <execute type="shell">`sh ./startdb.sh &&`</execute>
  <execute target="db" type="sql" onError="warn">
    DROP TABLE db_user;
    CREATE TABLE db_user (id int NOT NULL,
      name varchar(30) NOT NULL,
      PRIMARY KEY (id),
    );
  </execute>
</setup>
  
```

As you see, scripts can be inlined or imported from files. See [Scripting](#) for a full introduction.

6.5.2 Precondition Checking Stage

Complex data generation is often split up into several stages of which each has its own preconditions. For example, if you want to generate orders for all kinds of products, you may want to assure that at least one product of each category is defined in the system.

evaluate

The simplest way to perform precondition checks is the `<evaluate>` element, e.g. checking for categories without product:

```

<evaluate assert="{js:result == 0}" target="db">select count(*) from db_category left join db_product on
  db_product.category_id = db_category.id where db_product.category_id is null
</evaluate>
  
```

The `<evaluate>` element works as follows:

First, it evaluates a script the same way as an `<execute>` element does.

In the above example, the **select** query is performed on the database. Then the result is written into a variable named `result` and the `assert` condition is evaluated (cp. `result == 0`) which checks the value of the result and returns `true` or `false`. If the assertion resolves to `false`, Benerator raises an error.

In the above example, an error is raised if there is a category without any product assigned.

You can use an arbitrary expression language for performing the check. Like in `<execute>`, a prefix with a colon can be used to indicate the script language. You can optionally add an `id` attribute which will make Benerator put the evaluation result into the context with this id.

```
<evaluate id="product.cat_id_max" target="db">
  select max(db_product.category_id) from db_category
</evaluate>
```



Retrieving and persisting an id in the benerator context might be a useful approach for more complex foreign key constraints. However, try to check the built-in [Database-related Id Generators](#) first.

You can also call [DB Sanity](#) for verifying the preconditions.

6.5.3 Core Data Generation Stage

For predefined data generation it is most convenient to import core data from a file - this gives you full control and easy configuration by an editor and is the most reliable source for reproducible data. Currently, the most convenient file formats for this task are DbUnit XML files (one file with several tables) and CSV (one file per table).

```
<setup>
  <!-- import integration test data for all tables from one DbUnit file -->
  <iterate source="core.dbunit.xml" consumer="db"/>
  <!-- import predefined products from a CSV file -->
  <iterate type="db_product" source="demo/shop/products.import.csv" encoding="utf-8" consumer="db"/>
</setup>
```

Fixed column width files and SQL files can be used too. If you need to import data of other formats you can easily write a parser and use it directly from Benerator (See [Custom EntitySources](#)).

6.5.4 Mass Data Generation Stage

Mass data generation is the primary goal of Benerator. It is mainly performed by `<generate>` descriptors, which describe the creation of synthetic data, but may also include the import and reuse of information from external sources by an `<iterate>` descriptor. See [Descriptor File Format](#) and [Advanced Topics](#) for a description.

6.5.5 Data Postprocessing Stage

If your system has complex business logic (typically workflows), you will encounter generation requirements that are easier to satisfy by calling application business logic than by a pure descriptor-based generation.

For example, you might need to generate performance test data for a mortgage application: People may apply for a mortgage, enter information about their house, incomes and expenses, their application is rated by some rule set, and the mortgages finally are granted or rejected. Then you have complex logic (rating) that is not necessarily useful to be reproduced for data generation. It is easiest to call the business logic directly.

This can be done in two ways:

- Scripts : Having script commands inlined in the Benerator descriptor file or called from external files, e.g. `rapiddwellerScript`, JavaScript, Groovy, Ruby, Python. See [Scripting](#)
- Tasks : Programming own Java modules that are invoked by Benerator. See [Tasks](#)

6.5.6 Result Validation Stage

Data generation may become quite tricky. For improving maintainability it is recommended to perform validations after data generation:

- Checking the number of generated objects
- Checking invariants
- Checking prerequisites for specific performance tests

You can do so with the `<evaluate/>` element, e.g. checking the number of generated customers

```
<evaluate assert="{js:result = 5000000}" target="db"> select count(*) from db_user </evaluate>
```

The `<evaluate>` element was described above in the [Precondition Checking](#) section.

You can also use DB Sanity for verifying the preconditions; see [DB Sanity](#).

6.6 Metadata Concepts

Benerator processes metadata descriptors that can be imported from systems like databases and can be overwritten manually. Benerator automatically generates data that matches the (e.g. database) constraints. So, when it encounters a table defined like this:

```
CREATE TABLE db_user (
...
active SMALLINT DEFAULT 1 NOT NULL,
...
);
```

When generating data for the user table, Benerator will automatically generate all users with active set to 1:

```
<generate type="db_user" count="100" consumer="db"/>
```

If you specify active as an attribute, you inherit a new setting from the parent descriptor, dropping the parents configuration of values=1 and adding a new one, e.g. the configuration

```
<generate type="db_user" count="100" consumer="db">
  <attribute name="active" values="0,1"/>
</generate>
```

will cause generation of 50% 0 and 50% 1 values.

6.6.1 Case Sensitivity

Benerator has heuristic case sensitivity: It needs to combine metadata from different types of systems of which some may be case-sensitive, some may not. So Benerator first assumes case sensitivity when looking for a type. If the type is found in the same capitalization, this information is used. If it is not found, Benerator falls back to searching the type in a case-insensitive manner.

6.6.2 Namespaces

Benerator has a heuristic namespace support, similar to case-sensitivity handling: when looking up a descriptor by name, Benerator first searches the name in its assigned namespace. If the type is found there, this information is used. If it is not found, Benerator falls back to searching the type in all available namespaces.

6.6.3 `<setting>` and Benerator identifiers

A Benerator identifier (variable, entity or bean name) may contain only ASCII letters, numbers and underscores (no dot !) and is defined using a `<setting>` element - either in the descriptor file, e.g.

```
<setting name="user_count" value="1000000"/>
```

or in a properties file, e.g. myproject.properties:

```
user_count="1000000"
```

which is then included in the descriptor file:

```
<include uri="myproject.properties"/>
```

Of course you can evaluate variables for defining other variables as well by using a script expression:

```
<setting name="event_count" value="{user_count * 10}"/>
```

A property can also refer to another element of the generation context:

```
<setting name="limit" ref="maxCount"/>
```

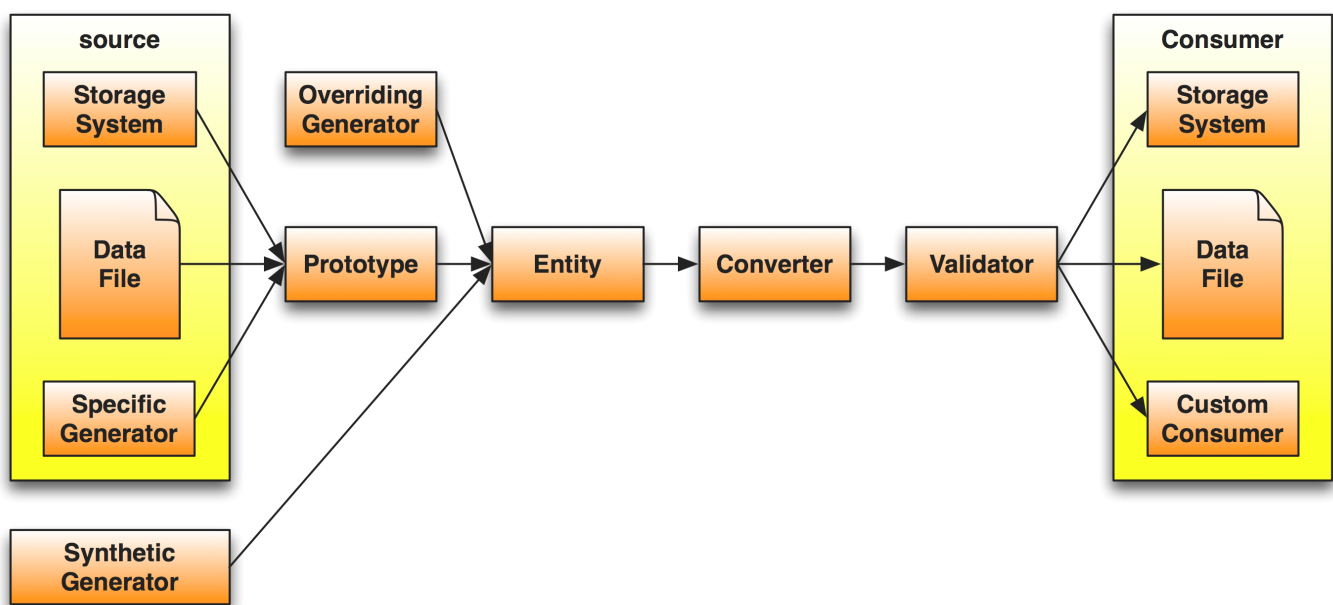
And it can be set calling a generator object:

```
<setting name="fileNumber" source="new DBSequenceGenerator('my_seq', db)"/>
```

You can define default values for properties. If no property value has been defined before, the property is set to this value:

```
<setting name="stage" default="development"/>
```

6.7 Benerator Components



Entities, as well as their attributes, can be imported from storage systems, data files, or specific generators. They then serve as prototypes of which attributes may be overwritten by another generator (overriding generator), e.g. for anonymization.

Alternatively, entities may be generated completely synthetically.

Entities and each entity attribute can be converted by a specific Converter object.

Validators assure the validity of the generated entities and attributes. All entities that fail validation are discarded.

Finally, generated data is consumed by storing it in a storage system (e.g. database), writing it to a data file or using it in a custom Consumer implementation.

6.8 Instantiating Global Components

You can define global components in a Spring-like syntax:

```
<bean id="helper" class="com.my.Helper">
  <property name="min" value="5"/>
  <property name="max" value="23"/>
</bean>
```

For details on this syntax and other variants, see the section [JavaBeans and the Benerator Context](#). You can refer to such an object by its id (`helper` in this case).

6.9 Instantiating Local Components

The following chapters will introduce you to the usage of each component type available in Benerator. They have common styles of definition and referral. If a component needs to be reused in different places, you would create it with a `<bean>` element and apply a referral to use it. If you do not need to reuse one component in different places, there are more concise inline instantiation styles available:

- default construction
- parameterized construction
- property-based construction

6.9.1 Referral

Any class can be instantiated and made available to Benerator by using a bean element, e.g. the `helper` instance above, you can use it like this:

```
<attribute name="number" generator="helper"/>
```

This is called *referral*.

6.9.2 Default Construction

If you specify just a class name, Benerator will create an instance of the class by invoking the default constructor. Be aware that the class needs a public no-argument constructor for being instantiated this way:

```
<attribute name="number" generator="com.my.Helper"/>
```

6.9.3 Parameterized Construction

You can as well specify the `new` keyword, a class name and constructor parameters. Benerator will then search a constructor with matching parameters and invoke it. If the class has several constructors with the same number of parameters Benerator might choose the wrong one, so it is good practice having just one constructor for each possible number of parameters.

```
<attribute name="number" generator="new com.my.Helper(5, 23)"/>
```

6.9.4 Property-based Construction

This is the most elegant and maintainable inline construction style, you specify the `new` keyword, the class name and, in square brackets, a comma-separated list of name-value pairs for each JavaBean property. Benerator uses a default constructor and the corresponding `set...()` methods to initialize the object.

```
<attribute name="number" generator="new com.my.Helper{min=5, max=23}"/>
```

6.10 Descriptive Data Generation

Descriptive Data Generation means defining elementary data restrictions, e.g. nullability and string lengths. For example, an attribute may have only one of an enumeration of values. They can be defined as a comma-separated list:

```
<attribute name="issuer" values="'AMEX','VISA'" />
```

For a list of descriptive attribute metadata, see [Attribute Metadata Reference](#) on how Descriptive metadata can be imported automatically from database schema metadata and be used for automatic database-valid data generation:

6.11 Default Data Generation

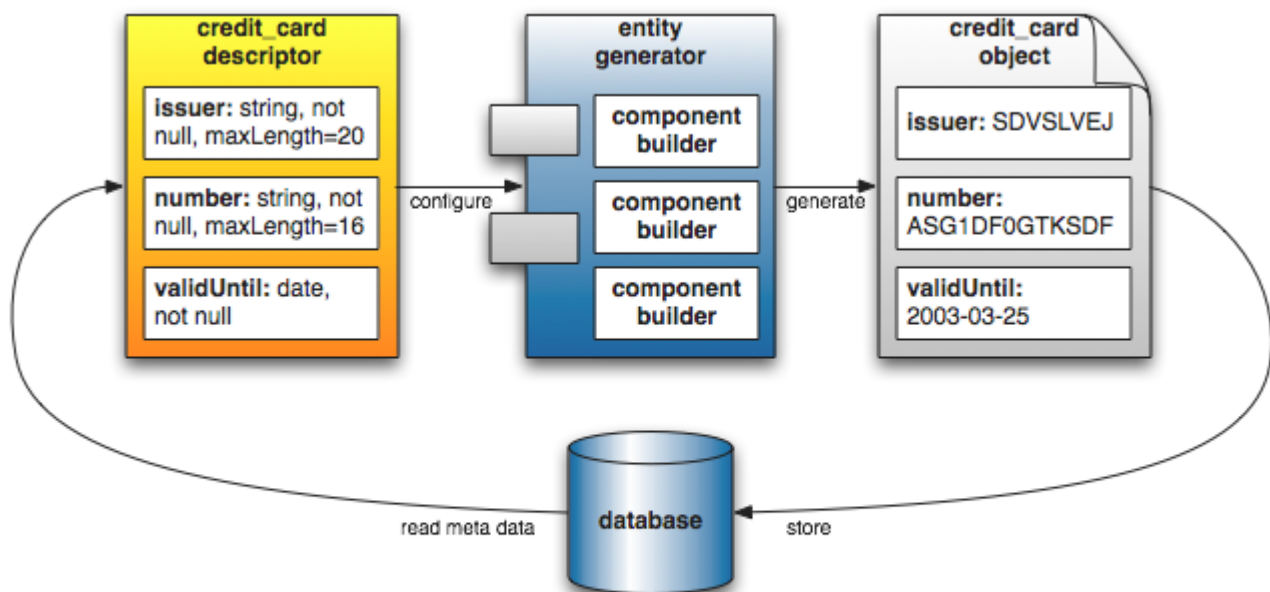
Based on descriptive metadata, Benerator applies several defaults for generating database-valid data.

All nullable attributes are generated as null by default.

Primary keys are generated as integral numbers by default, starting from 1 and increased by 1 consecutively. Primary keys of string type are handled similarly.

Foreign keys are resolved automatically. For avoiding illegal generation cases, Benerator assumes any foreign key relation to be one-to-one by default. Many-to-one relationships need to be configured manually.

Now have a look at an example for generating credit cards in a database:



```
<generate type="credit_card" count="500000" consumer="db"/>
```

Benerator reads the metadata for the table `credit_card` from a database. This results in descriptive metadata, saying that a `credit_card` entity has three attributes: `issuer` and `number` of type string and `validUntil` of type date. All of them may not be null and the `issuer` attribute has a maximum length of 20 characters, the `number` of 16 characters.

This is enough information to make Benerator generate, e.g. 50000 credit cards with a trivial setup:

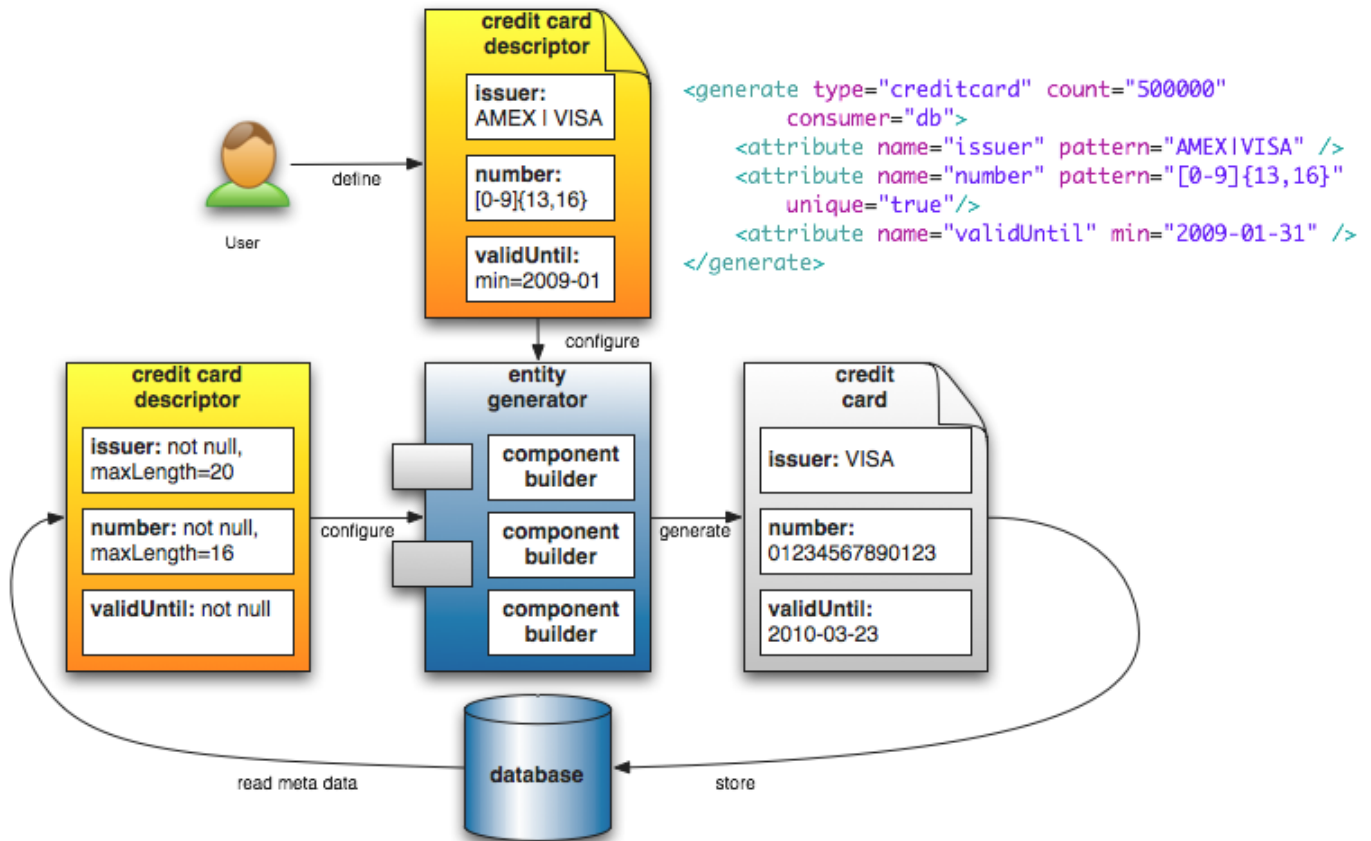
```
<generate type="credit_card" count="500000" consumer="db"/>
```

The resulting entries are database-valid automatically.

6.12 Constructive Data Generation

Constructive metadata describes methods of data generation, e.g. import from a data source or stochastic number generation.

We can improve the credit card example from above by adding own, constructive metadata to the descriptive ones imported from the database:

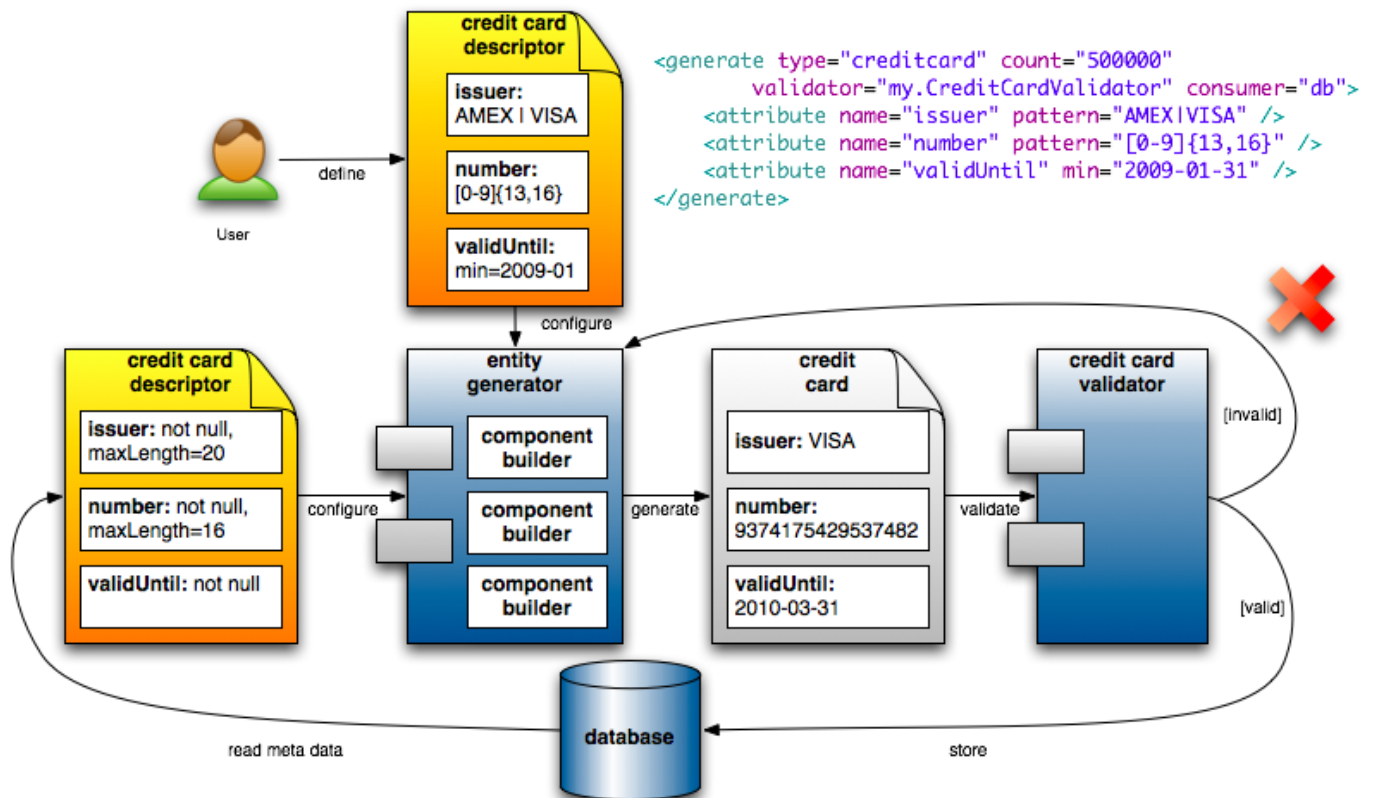


This way we can already satisfy simple validation algorithms, but not yet sophisticated ones that perform a checksum validation.

For a complete reference of metadata configuration, see [Attribute Metadata Reference](#), [ID Generators](#) and [Resolving Database Relations](#).

6.13 Validating Data Generation

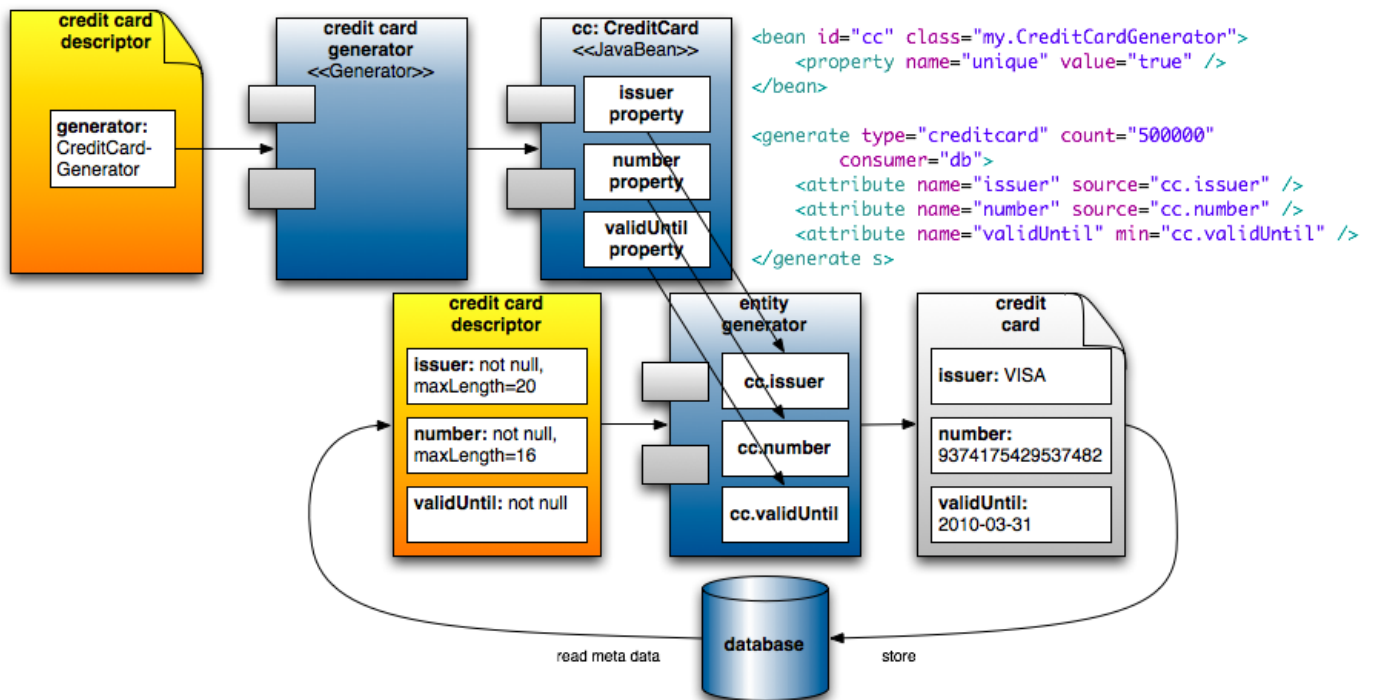
Suppose you have a validation component available, but do not know all details necessary for constructing valid data. In such a case, you can set up a constructive data generation and combine it with the validation module. So take the setup from the chapter before, write an adapter to your validation component and include it in Benerator's data generation:



Your setup will then create random credit card setups and the credit card validator will discard the invalid ones. For the definition of custom validators, see [Custom Validators](#).

6.14 Prototype-based Data Generation

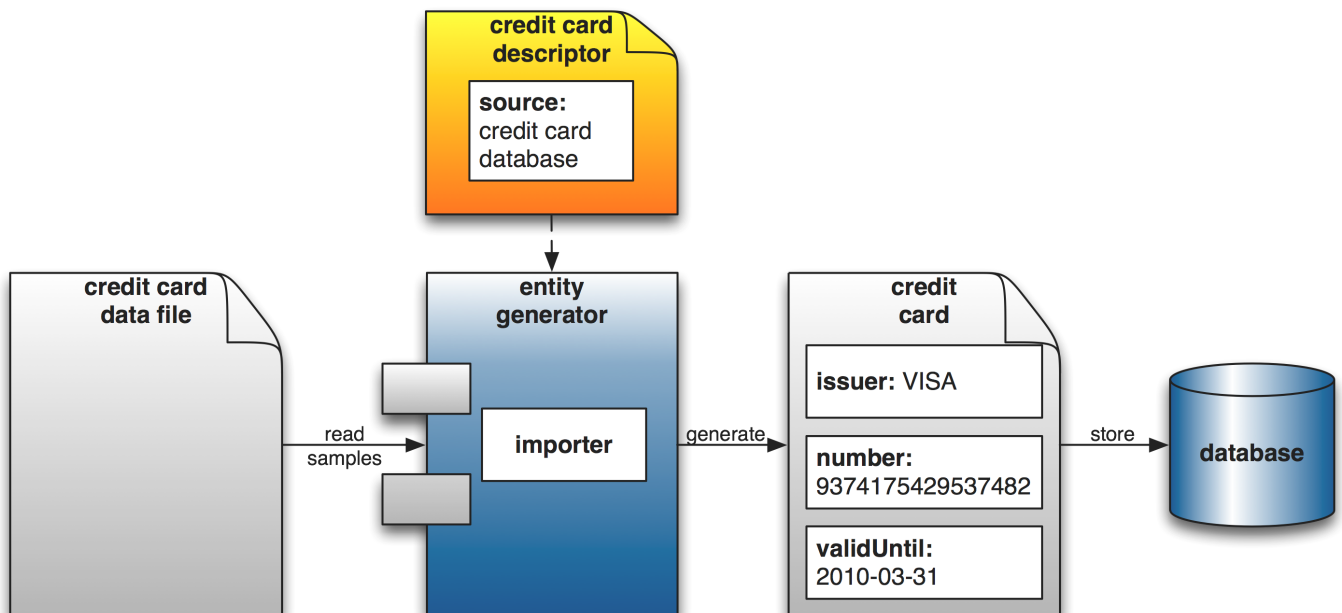
The examples above are satisfactory for almost all cases, but if you need to satisfy very difficult validity conditions you need ultimate control over a generation. For our credit card example, you might have a validator module that connects to the credit card company and validates if the account really exists



You can generate prototypes with custom generators or import them as samples, see [Sample-based Data Generation](#).

6.15 Sample-based Data Generation

The examples above are satisfactory for almost all cases, but there are cases in which you need to use a predefined set of entities. For our credit card example, the tested application might check credit cards by connecting to the credit card company and query if the account really exists. In such a case you typically define a file with known credit card numbers to use:



```
<iterate type="credit_card" source="credit_cards.csv" consumer="db" />
```

You can use different types of data sources for templates:

- Files: CSV, fixed column width files, DbUnit. For importing data of custom file formats or from other sources, see [Custom EntitySources](#)
- Storage Systems: Relational databases For importing data of proprietary storage systems, see e.g. [Memstore](#)

6.16 Variables

When importing entities from a data source you will need to map data in some way. This is where the variable concept comes in: You can define a `<variable>` as an auxiliary generator inside a `<generate>` descriptor and assign it a name (e.g. `person`). In each entity generation this generator will provide a newly generated object under the assigned name (`person`). So, if you want to access a part of a composite generated object you can query it e.g. by a script expression like `person.familyName`:

```
<generate type="customer" consumer="ConsoleExporter">
  <variable name="person" generator="PersonGenerator"/>
  <attribute name="lastName" script="person.familyName"/>
</generate>
```

For defining a variable, you can use the same syntax elements as for an attribute. But the type of data that the variable can generate is much less restricted. A variable may

- use an EntitySource or Generator that creates entity objects.
- use a Generator that creates Maps or JavaBean objects. Their map values or bean properties can be queried from a script the same way as for an entity
- execute a SQL query (e.g. `name="c_customer" source="db" selector="select id, name from customer where rating = 0"`) of which column values may be accessed by a script (e.g. `script="{c_customer[0]}"` for the id).

6.17 Combining components and variables

Starting with Benerator 0.7, sub-elements of a `<generate>` loop are evaluated in the order in which they appear in the descriptor file, in earlier versions they were reordered before processing. When nesting `<generate>` loops be aware, that each instance of the outer loop is consumed before a sub-generate is called, so it does not make sense to define an `<attribute>`, `<id>` or `<reference>` after the sub-generate statement.

6.18 Referring Files

In most cases, Files are referred by URIs. A URI may be

- a simple local (`data.csv`) or
- an absolute filename (`C:\datagen\data.csv`) or a
- a URL (`http://my.com/datagen/data.csv`).

For FTP access, use RFC1738 for encoding username, password and file format, e.g. `ftp://user:password@server/dir/file?type=i`

6.19 Protocols

Currently, Benerator supports only file URIs for reading and writing and HTTP and FTP URIs for reading. Support of further protocols is possible and planned for future releases.

6.20 Relative URIs

Relative URIs are resolved in a HTML hypertext manner: A relative URL is interpreted relative to a *base URI* which is the path of the Benerator descriptor file. If file lookup fails, Benerator searches the file relative to the current directory. If that fails, Benerator tries to retrieve the file from the Java classpath.

Benerator recognizes absolute paths under Windows (e.g. `C:\test`) and Unix (`/test` or `~/test`).

When in doubt, mark the URL as file URL: `file:///C:/test` or `file:///test`.

6.21 Importing Entities

Entities can be imported from systems, files or other generators. A typical application is to (re)use a DBUnit setup file from your (hopefully existing ;-) unit tests:

```
<iterate source="shop/shop.dbunit.xml" consumer="db"/>
```

For importing DbUnit files, follow the naming conventions using the suffix `.dbunit.xml`.

Each created entity is forwarded to one or more consumers, which usually will persist objects in a file or system, but might also be used to post-process created entities. The specified object needs to implement the Consumer or the system interface. When specifying a system here, it will be used to store the entities. File exporters (for CSV and fixed column width files) implement the Consumer interface.

6.22 Custom Importers

New import formats can be supported by implementing the EntitySource interface with a JavaBean implementation, instantiating it as bean and referring it by its id with a `source` attribute, e.g.

```
<setup>
  <bean id="products_file" class="com.rapiddweller.platform.fixedwidth.FixedWidthEntitySource">
    <property name="uri" value="shop/products.import.fcw"/>
    <property name="entity" value="product"/>
    <property name="properties" value="ean_code[13],name[30],price[8r0]"/>
  </bean>

  <iterate type="product" source="products_file" consumer="ConsoleExporter"/>
</setup>
```

6.23 Consumers

Consumers are the objects that finally receive the data after creation, conversion and validation. Consumers can be files, storage systems or custom JavaBeans that implement the Consumer interface. They are not supposed to mutate generated data. That is reserved for Converters.

6.23.1 Specifying Consumers

A `<generate>` element may have consumers in `<consumer>` sub-elements or in a comma-separated list in a consumer attribute, e.g. `consumer="a,b"`. A consumer sub element has the same syntax as a `<bean>` element, e.g.

```
<generate type="db_product">
  <consumer class="com.my.SpecialConsumer">
    <property name="format" value="uppercase"/>
  </consumer>
</generate>
```

A consumer attribute may hold a comma-separated list consisting of

- names of previously defined beans
- fully qualified class names of consumer implementations

Examples:

```
<database id="db" .../>
```

The `<database>` declaration will be described later.

```
<setup>
  <bean id="special" class="com.my.SpecialConsumer">
    <property name="format" value="uppercase"/>
  </bean>

  <generate type="db_product" consumer="db,special"/>
</setup>
```

or

```
<setup>
  <bean id="special" class="com.my.SpecialConsumer">
    <property name="format" value="uppercase"/>
  </bean>

  <generate type="db_product" consumer="com.my.SpecialConsumer"/>
</setup>
```

6.23.2 Consumer Life Cycle

In most cases, Consumers relate to heavyweight system resources, so it is important to know their life cycle. There are two different types of the life cycle:

- **Global Consumer:** A consumer defined as a `<bean>` has global scope (thus is called **global consumer**) and is closed when Generator finishes. So you can use the same consumer instance for consuming the output of several `<generate>` and `<iterate>` blocks.
- **Local Consumer:** A consumer-defined *on the fly* in a generate/iterate block (by `new`, class name or `<consumer>`) has a local scope and is immediately closed when its generate/iterate block finishes. If you want to generate a file and iterate through it afterward, you need to have it closed before. The most simple way to assure this is to use a local consumer in file generation.

6.24 Exporting Data to Files

You will need to reuse some generated data for setting up (load) test clients. You can export data by simply defining an appropriate consumer:

```
<setup>
  <import platforms="xls"/>

  <generate type="db_product" consumer="new XLSntityConsumer('prod.xls')">
    ...
  </generate>
</setup>
```

6.25 Post Processing Imported or Variable Data

When importing data or using helper variables, you may need to overwrite imported attributes. You can do so by

- **overwriting** them (e.g. with a generated ID value) or
- manipulating the imported attribute value with a **script** (e.g. replacing CARD=Y/N with 0/1) or
- using a **map** to convert between predefined values or
- using a **converter** to transform attributes individually (e.g. for converting strings to uppercase)

You could also combine the approaches

6.25.1 overwriting post-processing

```
<iterate type="TX" source="tx.ent.csv">
  <id name="ID" generator="IncrementalIdGenerator"/>
</iterate>
```

6.25.2 "script" post-processing

```
<iterate type="TX" source="tx.ent.csv">
  <attribute name="CARD" script="TX.CARD == 'Y' ? 1 : 0"/>
</iterate>
```

6.25.3 "map" post-processing

For mapping imported (or generated) values, you can use a convenient literal syntax, listing mappings in a comma-separated list of assignments in the form `original_value -> mapped_value`.

!!! danger Values need to be literals here too, so don't forget the quotes around strings and characters!

This is a postprocessing step, so it can be combined with an arbitrary generation strategy.

Example

```
<iterate type="db_user" source="db">
  <variable name="p" generator="person"/>
  <attribute name="gender" script="p.gender.name()" map="'MALE' -> 'm', 'FEMALE' -> 'f'"/>
</iterate>
```

In a script, the keyword `this` refers to the entity currently being generated/iterated.

6.26 Converters

Converters are useful for supporting using custom data types (e.g. a three-part phone number) and common conversions (e.g. formatting a date as string). Converters can be applied to entities as well as attributes by specifying a converter attribute:

```
<generate type="TRANSACTION" consumer="db">
  <id name="ID" type="long" strategy="increment" param="1000"/>
  <attribute name="PRODUCT" source="{TRANSACTION.PRODUCT}" converter="CaseConverter"/>
</generate>
```

For specifying Converters, you can

- use the class name
- refer a JavaBean in the Benerator context
- provide a comma-separated Converter list in the two types above

Benerator supports two types of converters:

1. Classes that implement Benerator's service provider interface (SPI) `com.rapiddweller.common.Converter`
2. Classes that extend the class `java.text.Format`

If the class has a `pattern` property, Benerator maps a descriptor's pattern attribute to the bean instance property.

See [Component Reference -> Converters](#) for a list of available converters and implementation examples.

6.27 Validators

Validators assist you in assuring the validity of generated data. Validators can be applied to attributes and full entities. They intercept in data generation: If a generated item is invalid, it will be discarded and regenerated transparently. This is a cheap way of fulfilling complex constraints which are only partially known: If you have a class or system that can validate this data, you can set up a heuristic generation that has a high probability of succeeding and simply discard the invalid ones. If the ratio of invalid objects is more than 99%, Benerator will give you a warning since this is likely to impact generation performance. If the ratio rises to 99.9%, Benerator will terminate with an exception.

For specifying Validators, you can

- use the class name
- refer a JavaBean in the Benerator context
- provide a comma-separated Validator list in the two types above

6.28 Creating random Entities

Entities can be generated without any input files - Benerator provides a rich set of Generator implementations. When using `<generate>` without a `source` attribute, the registered systems (e.g. the database) are requested for metadata. From the metadata, attributes are generated that match the metadata (e.g. database) constraints, as column length, referenced entities and more. By default, associations are treated as one-to-one associations.

With Benerator's many useful defaults, you have a minimum effort on initial configuration:

```
<generate type="db_user" count="1000" consumer="db" />
```

Id generation defaults to an increment strategy and for all other columns useful defaults are chosen.

Entities are generated as long as each attribute generator is available and limited by the number specified in the `count` attribute. The `pageSize` defines the number of creations after which a `flush()` is applied to all consumers (for a database system this is mapped to a commit).

6.29 Entity Count

There are different ways of determining or limiting the number of generated entities:

- the `count` attribute specifies a fixed number of instances to create
- the `minCount`, `maxCount` and `countDistribution` attributes let Benerator choose an instance count with the specified characteristics.
- availability of the component generators

Data generation stops if either the limit count is reached or a component generator becomes unavailable.

If you have problems with unexpectedly low numbers of generated entities you can set the log category `com.rapiddweller.benerator.STATE` to debug level.

6.30 Nesting Entities

Entities can form composition structures, which are generated most easily by recursive `<generate>` structures.

Consider a database schema with a `db_user` and a `db_customer` table. Each row in the `db_customer` table is supposed to have a row with the same primary key in the `db_user` table. So an easy way to implement this is to nest `db_customer` generation with `db_user` generation and use the outer `db_user`'s id value for setting the `db_customer` id:

```
<generate type="db_user" count="10" consumer="db">
  <id name="id" strategy="increment"/>
  ...
  <generate type="db_customer" count="1" consumer="db">
    <attribute name="id" script="{db_user.id}"/>
    ...
  </generate>
</generate>
```

6.31 Imposing one-field business constraints

Simple constraints, e.g. formats can be assured by defining an appropriate Generator or regular expression, e.g.

```
<setup>
<import domains="product" consumer="db"/>
<!-- create products of random attribs and category -->
<generate type="db_product" count="1000" pageSize="100">
  <attribute name="ean_code" generator="EANGenerator"/>
  <attribute name="name" pattern="[A-Z][A-Z]{5,12}"/>
</generate>
</setup>
```

6.32 Imposing multi-field constraints

For supporting multi-field constraints, you can use a prototype-based approach: Provide a Generator by a variable element. This generator creates prototype objects (or object graphs) that are used as prototypes. They may be Entities, JavaBeans or Maps. For example, this may be an importing generator. On each generation run, an instance is generated and made available to the other sub generators. They can use the entity or sub-elements by a source path attribute:

```
<setup>
<import domains="person"/>
<generate type="db_customer" consumer="db">
  <variable name="person" generator="PersonGenerator"/>
  <attribute name="salutation" script="person.salutation"/>
  <attribute name="first_name" script="person.givenName"/>
  <attribute name="last_name" script="person.familyName"/>
</generate>
</setup>
```

The source path may be composed of property names, map keys and entity features, separated by a dot.

6.33 Default Attribute Settings

Usually most entities have common attribute names, e.g. for ids or audit data. You can specify default settings by column name:

```
<defaultComponents>
<id name="ID" type="long" source="db" strategy="sequence" param="hibernate_sequence"/>
<attribute name="VERSION" values="1"/>
<attribute name="CREATED_AT" generator="currentDateGenerator"/>
<attribute name="CREATED_BY" values="rapiddweller"/>
<attribute name="UPDATED_AT" generator="currentDateGenerator"/>
<attribute name="UPDATED_BY" values="rapiddweller"/>
</defaultComponents>
```

If a table has a column that is not configured in the Benerator descriptor but as defaultComponent, Benerator uses the defaultComponent config. If no defaultComponent config exists, Benerator falls back to a useful standard setting.

6.34 Settings

You can define global settings in the descriptor file:

```
<setting name="my_name" value="Volker" />
```

or import several of them from a properties file:

```
<include uri="my.properties" />
```

6.35 Querying Information from a System

Arbitrary information may be queried from a system by a `selector` attribute, which is system-dependent. For a database SQL is used:

```
<generate type="db_order" count="30" pageSize="100">
  <reference name="customer_id" source="db" selector="select id from db_customer" cyclic="true"/>
  <consumer ref="db"/>
  <!-- automatically chosen by Benerator -->
</generate>
```

Using `cyclic="true"`, the result set will be re-iterated from the beginning when it has reached the end.

You may apply a distribution as well:

```
<generate type="db_order_item" count="100" pageSize="100">
  <attribute name="number_of_items" min="1" max="27" distribution="cumulated"/>
  <reference name="order_id" source="db" selector="select id from db_order" cyclic="true"/>
  <reference name="product_id" source="db" selector="select ean_code from db_product" distribution="random"/>
  <consumer ref="db"/>
</generate>
```



The result set of a selector might be quite large, so take care, which distribution to apply, see [Distribution Concepts](#).

You can use script expressions in your selectors, e.g.

```
selector="{{ftl:select ean_code from db_product where country='${country}'}"}
```

The script is resolved immediately before the first generation and then reused. If you need dynamic queries, that are re-evaluated, you can specify them with double brackets:

```
selector="{{ftl:select ean_code from db_product where country='${shop.country}'}"}
```

Example:

```
<generate type="shop" count="10">
  <attribute name="country" values="DE,AT,CH"/>
  <generate type="product" count="100" consumer="db">
    <attribute name="ean_code" source="db"
      selector="{{ftl:select ean_code from db_product where country='${shop.country}'}"}/>
  </generate>
</generate>
```

Also see the information about [Querying to Variables in Advanced Topics](#).

6.36 Attribute Metadata Reference

6.36.1 Descriptive Attribute Metadata

name	description	default
name	name of the feature to generate	
type	type of the feature to generate	string
nullable	tells if the feature may be null	true
mode	controls the processing mode: (normal	ignored
pattern	uses a regular expression for String creation or date format pattern for parsing Dates.	
values	provides a comma-separated list of values to choose from	
unique	whether to assure uniqueness, e.g. unique="true". Since this needs to keep every instance in memory, use is restricted to 100.000 elements. For larger numbers you should use Sequence-based algorithms.	false
min	the minimum Number or Date to generate	1
max	the maximum Number or Date to generate	9
granularity	the resolution of Numbers or Dates to generate	1
minLength	the minimum length of the Strings that are generated	
maxLength	the maximum length of the Strings that are generated	

6.36.2 Constructive Attribute Metadata

name	description	default
generator	uses a Generator instance for data creation	
nullQuota	the quota of null values to create	0
converter	the class name of a Converter to apply to the generated objects	
dataset	a (nestable) set to create data for, e.g. dataset="US" for the United States	
locale	a locale to create data for, e.g. locale="de"	default locale
source	A system, EntityIterator or file to import data from.	
selector	A system-dependent selector to query for data.	
trueQuota	the quota of true values created by a Boolean Generator.	0.5
distribution	the distribution to use for Number or Date generation. This may be a Sequence name or a WeightFunction class name.	
cyclic	auto-resets the generator after it has gone unavailable	false

6.37 Scripts

Scripts are supported in

- descriptor files
- properties files
- DbUnit XML files
- CSV files
- Fixed column width files

A script is denoted by curly braces, e.g. `{'Hi, I am ' + my_name}`. This syntax will use the default script engine for rendering the text as, e.g. 'Hi, I am Volker'. The default script engine is set writing `<setup defaultScript="...">` in the descriptor file's root element. If you want to use different script engines at the same time, you can differ them by prepending the scripting engine id, e.g. `{ftl:Hi, I am ${my_name}}` or `{ben:Hi, I am ' + my_name}`.

Scripts in Benerator descriptors are evaluated while parsing.

If you need to dynamically calculate data at runtime, use a `script` attribute, e.g.:

```
<attribute name="message" script="'Hi, ' + user_name + '!'" />
```



In the `script` attribute, curly braces are not necessary.

Using scripts you can access

- environment variables, e.g. `JAVA_HOME`
- JVM parameters, e.g. `benerator.validate`
- any JavaBean globally declared in the Benerator setup, e.g. `db`
- the last generated entity of each type, e.g. `db_user`
- the entity currently being generated and its attributes, e.g. `this.id`
- entities generated in outer `<generate>` elements
- helper variables in the `<generate>` element, e.g. `person.familyName`
- predefined or custom FreeMarker methods (when using FreeMarker as script language)
- Static Java methods and attributes, e.g. `System.getProperty('user.home')`
- instance methods and attributes on objects in the context, e.g. `db.system`

Warning

Variable names used in scripting may **not** contain points. A point always implies resolution of a local feature of an object, e.g. `person.familyName` resolves the `familyName` attribute/property/key of a person.

6.37.1 this

In a script, the keyword `this` always refers to the entity currently being generated. You can use this to construct attributes which have dependencies to each other:

```
<generate type="product">
  <id name="id"/>
  <attribute name="code" script="'ID#' + this.id"/>
</generate>
```

6.38 Handling Errors

6.38.1 onError

Several descriptor elements support an `onError` attribute. It determines an error severity and how Benerator should behave in case of errors.

The default severity is `fatal`, which causes Benerator to stop execution.

Other available severities are `ignore`, `trace`, `debug`, `info`, `warn`, `error`, which mainly influence the log level in which errors are reported, but do not stop execution.

```
<generate type="product" count="1000" onError="fatal" consumer="db">
  <!-- component setup here -->
</generate>
```

6.38.2 BadDataConsumer

For errors that are raised by a consumer, you have the alternative option to catch them and write the data which has caused the error to an alternative consumer. For example, you can write the problematic data to a CSV file named `errordata.csv` and post process it:

```
<generate type="product" count="1000"
  consumer="new BadDataConsumer(new CSVExporter('errors.csv'), db.inserter())">
  <!-- component setup here -->
</generate>
```



Note that this cannot work properly with a database that uses batch processing. See [Using Databases](#).

7. Data Anonymization Concepts

7.1 Basic Concepts

The approach for anonymizing (also obfuscating) production data is to `<iterate>` over existing data and overwrite data fields with privacy concerns making use of all the features you learned in [Data Generation Concepts](#).

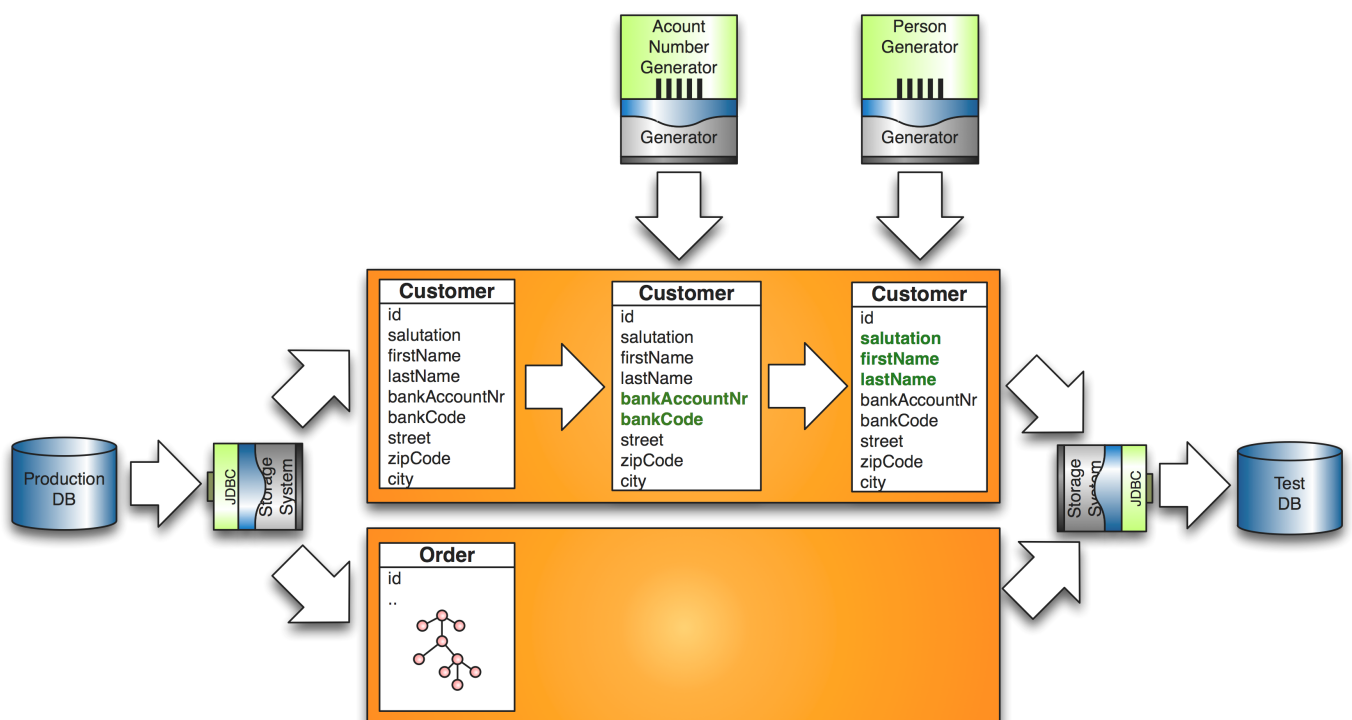
7.1.1 Prototype-based anonymization

If you need to assure multi-field-dependencies in anonymization or just overwrite data with real-looking substitutes, you can choose a prototype-based approach: import data from one source and merge it with prototypes that are generated or imported from another source. Benerator comes with many predefined prototype generators for different domains, and you can easily set up custom prototype-based anonymization approaches.

When importing data for functional and performance testing, you may need to add a [Data Postprocessing Stage](#).

In the following example, customers are imported from a database table in a production database (prod_db), anonymized and exported to a test database (test_db). All attributes that are not overwritten, will be exported as is. Since customer names and birthdates need to be anonymized, a prototype generator ([PersonGenerator](#)) is used to generate prototypes (named person) whose attributes are used to overwrite production customer attributes:

```
<iterate source="prod_db" type="db_customer" consumer="test_db">
  <variable name="person" generator="com.rapiddweller.domain.person.PersonGenerator"/>
  <attribute name="salutation" script="person.salutation" />
  <attribute name="first_name" script="person.givenName" />
  <attribute name="last_name" script="person.familyName" />
  <attribute name="birth_date" nullable="false" />
</iterate>
```



7.1.2 Data Masking

The following ones are Converters useful for data masking, making the manipulation obvious:

Classes	Description	Example
Mask	Replaces each character of a string with an asterisk '*' or another configurable character.	*
MiddleMask	Replaces each character of a string with an asterisk '*' or another configurable character, leaving a configurable number of characters unmasked at the beginning and/or the end of the string.	38*****329
MD5Hash, SHA1Hash, SHA256Hash	Convert any data to a hexadecimal hash code	D41D8CD98F00B204E9800998ECF8427E
MD5HashBase64, SHA1HashBase64, SHA256HashBase64	Convert any data to a hash code in Base64 format	1B2M2Y8AsgTpgAmY7PhCfg==
JavaHash	Convert any data to a hexadecimal hash code. This implementation is faster than the hash converters above	0027b8b2

The following example shows you a mix of prototype usage and masking:

Imagine having a set of personal data, like this:

given Name	family Name	alias	street	house Number	postal Code	city	accountNo	ssn
Hannah	Hopkins	hahop	Main Street	3456	9876	Central City	1234567890	1234567890
Bob	Baker	bobo	Orchard Lane	12	65395	Countryside	2345678901	2345678901
Alice	Anderson	alan	Shiny Street	123	93748	Gloryville	3456789012	3456789012

Applying the Benerator setup

```
<?xml version="1.0" encoding="UTF-8"?>
<setup xmlns="https://www.benerator.de/schema/2.1.0" defaultDataset="US">

  <import domains="address"/>

  <iterate source="persons.csv" type="data" consumer="ConsoleExporter">
    <variable name="addr" generator="AddressGenerator"/>
    <attribute name="familyName" converter="new CutLength(3)"/>
    <attribute name="givenName" converter="Mask"/>
    <attribute name="alias" converter="new Append('_demo')"/>
    <attribute name="street" script="addr.street"/>
    <attribute name="houseNumber" script="addr.houseNumber"/>
    <attribute name="postalCode" script="addr.postalCode"/>
    <attribute name="city" script="addr.city"/>
    <attribute name="country" constant="US"/>
    <attribute name="accountNo" converter="SHA256Hash"/>
    <attribute name="ssn" converter="new MiddleMask(2,3)"/>
    <attribute name="creditCardNo" converter="SHA1HashBase64"/>
    <attribute name="secret1" converter="MD5Hash"/>
    <attribute name="secret2" converter="MD5HashBase64"/>
    <attribute name="secret3" converter="SHA1Hash"/>
    <attribute name="secret4" converter="SHA1HashBase64"/>
    <attribute name="secret5" converter="JavaHash"/>
  </iterate>
</setup>
```

Yields the following result:

given Name	family Name	alias	street	house Number	postal Code	city	country	accountNo
**	Hop	hahop_demo	Williams Street	24	11940	EAST MORICHES	US	C775E7B751AB38829C
***	Bak	bobo_demo	Ridge Street	42	57373	SAINT LAWRENCE	US	4191597AA77C3AA38B7
*	And	alan_demo	Lincoln Street	47	88006	LAS CRUCES	US	26CC49F1A05B5413C7

7.2 Import filtering

You can choose a subset of the available data by using - features of the source system like a select statement for a `<database>` or `memstore` - a 'filter' expression for any kind of data source (Enterprise Edition only).

7.2.1 <Iterate filter> Option (Enterprise Edition)

In order to filter data import from any source, use a filter expression. Within it, you can use the variable `_candidate` to access the data entity that is a candidate to import and return true in order to accept it or false in order to drop it.

An example for iterating through orders only of high priority:

```
<iterate source="orders.xls" type="order" filter="_candidate.priority == 'high'" consumer="ConsoleExporter"/>
```

7.3 Anonymization Report (Enterprise Edition)

For compliance checking, Enterprise Edition provides an anonymization Tracker. It is activated on the command line by calling

```
benerator --anonReport 100
```

and creates a report listing each field name and which percentage of its occurrences were changed in anonymization. The report is exported to a tab-separated CSV file named `anonymization-report.csv` and to the console:

```
+-----+
| Anonymization Report: |
| Date/Time 2021-09-16 21:59:58 MESZ |
| Anonymized 500000 entities |
| Checked 500000 entities (100%) |
+-----+
| Field | # Checked | % Changed |
+-----+
| order.street | 500,000 | 100.0% |
+-----+
| order.city | 500,000 | 99.3% |
+-----+
| order.cardno | 500,000 | all |
+-----+
| order.date | 500,000 | none |
+-----+
| order.express | 500,000 | none |
+-----+
| order.gift | 500,000 | none |
+-----+
```

When anonymizing large amounts of complex data structures, you will notice that this tracking has a substantial performance impact, which may slow down performance by a factor of 10 or more. There are two combinable approaches to address this:

1. reducing the check to the fields which are relevant for privacy
2. Partial anonymization checking

7.3.1 Reducing the check to relevant fields

Anonymization is not done for fun but for privacy protection, so it is legitimate to reduce the anonymization check to the fields which are relevant for this purpose. In most applications, sensitive data is only a small part of all data, and anonymization checking overhead is strongly reduced.

This restriction is defined by listing the fields to be checked in an `<anon-check>` element:

```
<anon-check>street, city, cardno</anon-check>
```

You will then notice faster execution and a shorter report (restricted to the fields above):

```
+-----+
| Anonymization Report: |
| Date/Time 2021-09-16 21:59:58 MESZ |
| Anonymized 500,000 entities |
| Checked 500,000 entities (100%) |
+-----+
| Field | # Checked | % Changed |
+-----+
| order.street | 500,000 | 100.0% |
+-----+
| order.city | 500,000 | 99.3% |
+-----+
| order.cardno | 500,000 | all |
+-----+
```

7.3.2 Anonymization Sample Checking

If the reduction of fields to check is not fast enough (or undesired), you can additionally (or alternatively) reduce the checks to random samples of the anonymized data. In order to do so, you specify the percentage of data samples to be checked as command line argument on the Benerator call, for example a sample size of 10% of all data would be checked when specifying

```
benerator --anonReport 10
```

Then your report may look like this (when combined with field reduction):

```

+-----+
| Anonymization Report: |
| Date/Time 2021-09-16 21:59:58 MESZ |
| Anonymized 500,000 entities |
| Checked 50,075 entities (10%) |
+-----+
| Field | # Checked | % Changed |
+-----+
| order.street | 50,075 | 100.0% |
+-----+
| order.city | 50,075 | 99.3% |
+-----+
| order.cardno | 50,075 | all |
+-----+

```

7.3.3 Comparing Anonymization Tracking Performance

Which approach performs better is heavily dependent on the properties of your individual data structures.



A rule of thumb: For projects with large data structures, the fields restriction approach performs better, for projects with smaller data structures the sampling approach.

A non-representative example project with large data objects of which only a small number of fields needed to be anonymized, exhibited the following performance:

Method	Performance
No anonymization tracking	120 ME/h
Full anonymization tracking	10 ME/h
10% samples tracking	56 ME/h
Restricted fields tracking	65 ME/h
Restricted fields and 10% samples	75 ME/h



ME/h stands for a million entities per hour.

7.4 'condition'

When anonymizing or importing data, you may need to match multi-field constraints of the form **"if field A is set then field B must be set and field C must be null"**. In many cases, an easy solution is to import data, mutate only non-null fields and leave null-valued fields as they are.

A shorter syntax element to do so is the `condition` attribute.

It contains a condition and when added to a component generator, the generator is only applied if the condition resolves to true:

```

<iterate source="db1" type="customer" consumer="">
  <attribute name="vat_no" condition="this.vat_no != null" pattern="DE[1-9][0-9]{8}" unique="true" />
</iterate>

```

8. Regular Expression Support

Generator supports most common regular expression features and given a regular expression, it is able to generate strings that match the regular expression – even in a unique manner if required. Here is a full description of regular expression features recognized for data generation setup:

8.1 Characters

Digits (0-9) and US-ASCII letters (A-Z, a-z) are supported as they are. Special characters can be defined by their UNICODE number in octal or hexadecimal form:

```
\0n The character with octal value 0n (0 <= n <= 7)
\0nn The character with octal value 0nn (0 <= n <= 7)
\0mnn The character with octal value 0mnn (0 <= m <= 3, 0 <= n <= 7)
\xhh The character with hexadecimal value 0xhh
\uhhhh The character with hexadecimal value 0xhhhh
```

For control codes in general there is a special notation:

```
\cx The control character corresponding to x (e.g. \cA for Ctrl-A)
```

and certain control codes have own representations:

```
\t The tab character ('\u0009')
\n The newline (line feed) character ('\u000A')
\r The carriage-return character ('\u000D')
\f The form-feed character ('\u000C')
\a The alert (bell) character ('\u0007')
\e The escape character ('\u001B')
```

Some characters have a special meaning in regular expressions, so if you want to use them as a plain character (and not in their regex-meaning), you need to escape them with a backslash:

```
\. Dot
\~ Minus sign
\^ Circumflex
\$ Dollar
\' 'Or' sign
\( Left parenthesis
\) Right parenthesis
\[ Left bracket
\] Right bracket
\{ Left curly brace
\} Right curly brace
\\ Backslash character
```

8.2 Character Classes

A 'character class' defines a set of characters from which one can be chosen and is marked by surrounding brackets: []

[abc]	a, b, or c (simple class)
[^abc]	Any character except a, b, or c (negation)
[a-zA-Z]	a through z or A through Z, inclusive (range).
[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
[a-z&&[def]]	d, e, or f (intersection)
[a-z&&[^abc]]	a through z, except for b and c: [ad-z] (subtraction)
[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z] (subtraction)

There are some popular predefined character classes:

.	Any character (may or may not match line terminators)
\d	A digit: [0-9]
\D	A non-digit: [^0-9]
\s	A whitespace character: [\t\n\x0B\f\r]
\S	A non-whitespace character: [^\s]
\w	A word character: [a-zA-Z_0-9]
\W	A non-word character: [^\w]

Quantifiers can be used to specify how many characters of a class (or other regular expression constructs) should appear:

```
X?    X, once or not at all
X*    X, zero or more times
X+    X, one or more times
X{n}  X, exactly n times
X{n,} X, at least n times
X{n,m} X, at least n but not more than m times
```

8.3 Operators

```
XY    X followed by Y
X|Y    Either X or Y
(X)    X, as a group
```

8.4 Frequently Asked Questions

For generating characters which appear in your language, but not in English (like German umlauts), you can use their unicode representation (e.g. `\u00FC` for `ü`).

Different implementations of regular expression parsers exist and many have slight differences. So, if you take a regular expression that worked on one parser and run it on another one, you may get an error message. Benerator users that do not construct a regular expression by themselves, but simply take on 'from the internet' observe the same effect: The most frequent fault is if someone wants to generate a character that has a special meaning in a regular expression and does not escape it with a backslash, e.g. `\.`, `\\`, `\-`, `\|`, `\[`, `\]`, `\{`, `\}`, ...

An example: Some regex parsers recognize that the expression `[A-]X` could resolve to `AX` or `A-`. While others (like Benerator's parser) diagnose a malformed character class (a missing character behind the minus) and report an error. You can resolve this by escaping the minus sign: `[A\-]X`.

9. Data File Processing

9.1 CSV Import and Export

9.1.1 Iterating entity data from a CSV file

You can iterate entity data from a CSV file by assigning the file with the extension '.ent.csv' and specifying the file name as 'source' in an `<iterate>` statement, e.g. for printing the data to the console:

```
<iterate type="user" source="user.ent.csv" consumer="ConsoleExporter"/>
```

This way, you need to have a CSV file which uses column headers and the default column separator (which is comma by default and can be set globally in the root element's `defaultSeparator` attribute, e.g. to a semicolon: `<setup defaultSeparator=";">`)

If the CSV file does not have headers or uses another separator or file encoding that deviates from the default, you need to configure the CSV import component (CSVEntitySource) explicitly with a `<bean>` statement and refer it later:

```
<setup>

  <bean id="in" class="CSVEntitySource">
    <property name="uri" value="headless-in.csv"/>
    <property name="separator" value=";" />
    <property name="encoding" value="UTF-8"/>
    <property name="columns" value="name,age"/>
  </bean>

  <iterate type="user" source="in" consumer="ConsoleExporter"/>
</setup>
```

For CSV files without a header, you need to specify a comma-separated list of column names in the 'columns' property.

9.1.2 Creating CSV files

For creating a CSV file you must always take the same approach as above: Defining a bean with its properties and referring it as consumer:

```
<setup>

  <bean id="out" class="CSVEntityExporter">
    <property name="uri" value="target/headless-out.csv"/>
    <property name="columns" value="name, age, check"/>
  </bean>

  <generate type="product" count="200" consumer="out"/>
</setup>
```

See the [Component Reference](#) for more details.

9.2 Excel™ File Processing

When choosing to include Excel files in your data generation/anonymization, always be aware, that its data capacity is limited to 65,535 rows.

9.2.1 Iterating Entity Data from an Excel™ File

```
<iterate type="product" source="products.xls" consumer="ConsoleExporter"/>
```

9.2.2 Creating Excel™ Files

```
<generate type="product" count="7" consumer="new XLSExporter('products.xls')">
  <attribute name="ean_code" pattern="[0-9]{13}" />
  <attribute name="name" pattern="[A-Z][a-z]{5,12}" />
```

```
<attribute name="price" type="double" min="0.25" max="100.00" granularity="0.25"/>
</generate>
```

creates an Excel file like this:

	A	B	C
1	ean_code	name	price
2	5434103340665	Wibcoi	66,25
3	6699915746859	Mxjnknqhw	88,75
4	3883187344869	Bvlnkdr	20,
5	0920591729413	Amemfnf	51,75
6	7600616248648	Gcpfanhv	60,
7	0135505461756	Ehvmphge	72,75
8	8199813652144	Aqfxrrkfz	55,5

9.3 Fixed Column Width File Processing

A Fixed Column Width (FCW) files contain data in a text table format easily readable for humans. A product list may look like this:

```
0539401811799 Evian Water      1.25
1347354927249 Lindt Chocolate  2.75
2335882183280 Sanbitter       6.75
9071782830384 Beluga Caviar   100.00
9965715556715 Black Truffles  85.75
```

Note that, since a column header may require the column to be too wide, it is often left out. Beyond its width, each column may have additional formatting requirements: - Alignment: Left, right, centered - Fixed number of fraction Digits - Padding character (for example '0' for numbers)

Another way to format the above data may be

```
0539401811799 Evian Water_____ 001.25
1347354927249 Lindt Chocolate__ 002.75
2335882183280 Sanbitter_____ 006.75
9071782830384 Beluga Caviar____ 100.00
9965715556715 Black Truffles___ 085.75
```

Generator supports column format definition in a short syntax:

```
<width>[.<fraction digits>]<alignment>[<pad character>]
```

Field	Description	Default
width	The total number of characters for the column	
fraction digits	The number of fraction digits to display	<not used>
alignment	l for left, c for centered, r for right	l
pad character	a character used to reach the column width, usually a space character or sometimes 0 for numbers	<space>

Examples:

- `name[16l]` : Print the name left-aligned with 16 characters (padding spaces at the right)
- `price[8.2r0]` : Print the price right-aligned with a total of 8 characters (5 integer digits + the decimal point + 2 fraction digits), padded at the left using zeros. Example: `00033.25`

The complete columns' configuration is expressed as a comma-separated list of column specifications.

The FCW support classes `FixedWidthEntitySource` and `FixedWidthEntityExporter` are in the platform package 'fixedwidth' and make use of a similar configuration API:

```
<import platforms="fixedwidth"/>

<bean id="products_file" class="FixedWidthEntityExporter">
  <property name="uri" value="shop/products.import.fcw"/>
  <property name="properties" value="ean_code[13],name[30],price[8r0]"/>
</bean>
```

9.3.1 Iterating Entity Data of a Fixed Column Width File

For iterating data of Fixed Column Width files, you need to provide one more configuration detail than the example above: the 'entity' property tells Benerator as which entity type to interpret the file content.

```
<setup>
  <import platforms="fixedwidth"/>

  <bean id="products_file" class="FixedWidthEntitySource">
    <property name="uri" value="shop/products.import.fcw"/>
    <property name="entity" value="product"/>
    <property name="properties" value="ean_code[13],name[30],price[8r0]"/>
  </bean>

  <iterate type="product" source="products_file" consumer="ConsoleExporter"/>
</setup>
```

9.3.2 Creating Fixed Column Width File

```
<setup>
  <import platforms="fixedwidth"/>

  <bean id="products_file" class="FixedWidthEntityExporter">
    <property name="uri" value="export.fcw"/>
    <property name="columns" value="ean_code[14],name[16l],price[8.2r]"/>
  </bean>

  <generate type="product" count="7" consumer="products_file,ConsoleExporter">
    <attribute name="ean_code" pattern="[0-9]{13}"/>
    <attribute name="name" pattern="[A-Z][a-z]{5,12}"/>
    <attribute name="price" type="double" min="0.25" max="100.00" granularity="0.25"/>
  </generate>
</setup>
```

creates a file like this:

```
0539401811799 Sekybauld      90.75
1347354927249 Flntqqwkq     55.00
2335882183280 Wdlbjkv       9.75
9071782830384 Snhhlqoggabo   89.00
9965715556715 Exqygfllwbzgkn 31.75
```

```
5500154197720 Ctiskrmjjps      5.00
4379247967662 Xmuudbkpyz      39.00
```

9.4 JSON File Generation and Anonymization (Enterprise Edition)

9.4.1 Iterating entity data from a JSON file

You can iterate entity data from a JSON file by assigning the file with the extension '.json' and specifying the file name as 'source' in an `<iterate>` statement, e.g. for printing the data to the console:

```
<iterate type="user" source="users.json" consumer="ConsoleExporter"/>
```

An example source file 'users.json' may look like this:

```
[
  {
    "name": "Alice",
    "age": 23
  },
  {
    "name": "Bob",
    "age": 34
  },
  {
    "name": "Charly",
    "age": 45
  }
]
```

9.4.2 Creating JSON files

Generated data is exported to a JSON file by defining a consumer like this: `consumer="new JsonFileExporter('gen-persons.json')"`

```
<?xml version="1.0" encoding="UTF-8"?>
<setup>
  <generate type="person" count="5" consumer="new JsonFileExporter('gen-persons.json')">
    <attribute name="name" pattern="Alice|Bob|Charly"/>
    <attribute name="age" type="int" min="18" max="67"/>
  </generate>
</setup>
```

10. Generating XML Files

Benerator offers several options to generate XML files.

They come in two different flavors:

1. Explicit XML construction: You manually compose and configure XML generation
2. Implicit (schema-based) XML definition: You provide an XML Schema file and Benerator figures out the rest

10.1 Explicit XML construction

With explicit XML construction, you manually define data generation and nesting. An XML consumer then simply outputs them as XML file.

Two such XML consumers are provided with Benerator:

- DbUnitEntityExporter
- XMLEntityExporter

Each one of both uses a fix defined format to render data as XML. If both formats do not meet your needs, you can apply an XML transformation afterwards, for example invoked with an `execute` directive:

```
<execute type="shell">tx-xml transactions.dbunit.xml tx-trans.xslt tx.xml</execute>
```

10.1.1 DbUnitEntityExporter

A consumer writes any created entity to a file in DbUnit XML format. Use this if you only need a DbUnit file or want a simple, flat XML-based export for import in other tools. See the '[Component Reference](#)' for more information.

```
<setup>
  <generate type="order" count="7" consumer="new DbUnitEntityExporter('tx.dbunit.xml')">
    <id name="id" type="int"/>
    <attribute name="items" type="short" min="1" max="27" distribution="cumulated"/>
    <attribute name="price" type="double" min="0.01" max="100.00" increment="0.01"/>
    <attribute name="comment" pattern="[A-Z][a-z]{5,12}"/>
    <attribute name="ean_code" pattern="[0-9]{13}"/>
  </generate>
</setup>
```

Running this setup creates an XML file like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<dataset>
  <order id="1" items="11" price="78.17" comment="Drwhcqpfbga" ean_code="9919144617610"/>
  <order id="2" items="12" price="58.36" comment="Loxzhlvgs" ean_code="6603356278824"/>
  <order id="3" items="15" price="19.41" comment="Lpphxx" ean_code="6843307402222"/>
  <order id="4" items="18" price="62.45" comment="Gfotkrhegmbbx" ean_code="8264224421213"/>
  <order id="5" items="10" price="12.08" comment="Sdhvne" ean_code="5261265548872"/>
  <order id="6" items="18" price="27.81" comment="Svtrtrhlwmdu" ean_code="3677815958330"/>
  <order id="7" items="17" price="75.28" comment="Helqgd" ean_code="5931484187271"/>
</dataset>
```

Note that the DbUnit file format is limited to flat data structures. For nested XML elements use the following consumer:

10.1.2 XMLEntityExporter

A consumer which is more powerful than the DbUnitEntityExporter: It renders each simple-type entity attribute as an XML attribute and each sub-entity as a nested XML element. See the [Component Reference](#) for more information.

The example shows how to nest generation configuration:

```
<setup>
  <import platforms="xml"/>
  <bean id="xml" spec="new XMLEntityExporter('tx.xml')"/>
```

```

<generate type="orders" count="1" consumer="xml">
  <generate type="order" count="7" consumer="xml">
    <id name="id" type="int"/>
    <attribute name="items" type="short" min="1" max="27" distribution="cumulated"/>
    <attribute name="price" type="double" min="0.01" max="100.00" granularity="0.01"/>
    <attribute name="comment" pattern="[A-Z][a-z]{5,12}"/>
    <attribute name="ean_code" pattern="[0-9]{13}"/>
  </generate>
</generate>
</setup>

```

for the following result:

```

<?xml version="1.0" encoding="UTF-8"?>
<orders>
  <order id="1" items="11" price="77.99000000000001" comment="Rvjloach" ean_code="9780700577140"/>
  <order id="2" items="22" price="62.13" comment="Ezviominvbvol" ean_code="2224896066588"/>
  <order id="3" items="19" price="84.25" comment="Zkrryifmoo" ean_code="1193496860138"/>
  <order id="4" items="12" price="72.53" comment="Ufecgkxxqyen" ean_code="3292695014212"/>
  <order id="5" items="8" price="68.67" comment="Mrxisnwbdyoe" ean_code="8835687800915"/>
  <order id="6" items="7" price="20.610000000000003" comment="Rdrutwfkpuly" ean_code="4455177396063"/>
  <order id="7" items="12" price="24.630000000000003" comment="Puecbk" ean_code="8930345219912"/>
</orders>

```

10.1.3 Using data types from XML schema files

Including an XML schema in a classic descriptor file makes its data types available for explicit data generation. This can be used with the DbUnitExporter as well for the XmlExporter:

```

<include uri="shop.xsd" />

<generate type="product" count="5" consumer="ConsoleExporter"/>

```

10.2 Implicit (schema-based) XML definition

This is the most powerful XML generation approach: An XML schema file is used as the central definition for data generation. Benerator is able to automatically generate data purely based on a plain schema file, but, of course, you will want more detailed control over data generation. You get this by inserting generation descriptors in the well-known XML descriptor syntax. This makes data generation almost as versatile as with the classic descriptor-file-based approach. More than that, you can generate an arbitrary number of XML files with one program call.

Since XML schema offers a very wide range of options, Benerator concentrates on features usually needed for data files but does not yet support the complete schema feature set. If you are interested in improvements, please email us at team@rapiddweller.com.

10.2.1 Introduction

For the first trials, use a simple XML Schema file. We are beginning without annotations and save the following XML Schema with the name `transactions.xsd`:

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:ben="https://www.benerator.de/schema/benerator-2.1.0.xsd"
  xmlns="https://www.benerator.de/shop-0.8.1.xsd"
  targetNamespace="https://www.benerator.de/shop-0.8.1.xsd"
  elementFormDefault="qualified">

  <xs:element name="transactions">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="transaction" minOccurs="0" maxOccurs="unbounded" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="transaction">
    <xs:complexType>
      <xs:attribute name="id" type="xs:long" use="required" />
      <xs:attribute name="comment" type="xs:string" use="required" />
      <xs:attribute name="ean_code" type="xs:string" use="required" />
      <xs:attribute name="price" type="price-type" use="required" />
      <xs:attribute name="items" type="xs:integer" />
    </xs:complexType>
  </xs:element>

```

```

<xs:simpleType name="price-type">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="0"/>
    <xs:totalDigits value="8" />
    <xs:fractionDigits value="2" />
  </xs:restriction>
</xs:simpleType>
</xs:schema>

```

This defines an XML file format which has a `<transactions>` attribute as root element which contains an arbitrary number of `<transaction>` elements. Each `<transaction>` has the attributes 'id', 'comment', 'ean_code', 'price' and 'items'. The 'price-type' specifies decimal values with a total of 8 digits, 2 of which are decimal digits.

You can invoke XML file generation using Benerator directly or using the Maven Benerator Plugin. Let's call Benerator directly from the shell for now. Open a console, go to the directory which contains your schema file and invoke (under Windows):

```
createXML transactions.xsd transactions tx-{0}.xml 2
```

On Unix systems type:

```
createXML transactions.xsd transactions tx-{0}.xml 2
```

This tells Benerator to generate 2 xml files named `tx-1.xml` and `tx-2.xml` based on the schema file `transactions.xsd` using the `transactions` element as root element.

Open one of the generated files, and you will see the following content:

```

<?xml version="1.0" encoding="UTF-8"?>
<transactions elementFormDefault="unqualified"/>

```

So, what did Benerator do wrong? Nothing, it is a perfectly schema-valid document. Since minOccurs of the transaction ref is zero, Benerator takes the easy choice: Remember: One of Benerator's strengths is to configure the generation of valid data as easily as possible and in project stages as early as possible. With the chosen approach, you need to spend less time explicitly configuring element removal which is not yet supported by your application.

For configuring the generation of `<transaction>` elements, you need to add an annotation to your schema. The `ref` configuration in the `sequence` is the right place to configure cardinalities of included sub elements:

```

<xs:element name="transactions">
  <xs:complexType>
    <xs:sequence maxOccurs="unbounded">
      <xs:element _ref="transaction">
        <xs:annotation>
          <xs:appinfo>
            <ben:part minCount="5" maxCount="10"/>
          </xs:appinfo>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

Run Benerator again, and you will notice, that Benerator generated files like the following, each containing 5 to 10 transactions:

```

<?xml version="1.0" encoding="UTF-8"?>
<transactions elementFormDefault="unqualified">
  <transaction id="4" comment="OBNBHQWJYSPAHOCSBGDPGXUQK" ean_code="KJCDPUJNK" price="1" items="6"/>
  <transaction id="6" comment="UIXSXLGFBIIRP" ean_code="MM" price="3" items="7"/>
  <transaction id="4" comment="CRWGBGEKLRZQADE" ean_code="MXIESHSXQVLFJIBC" price="5" items="5"/>
  <transaction id="9" comment="FVBABHSYXZJHQYCVWJ" ean_code="FRPHJBOKUWHYKWHCWEIJBHVHIMV" price="1" items="9"/>
  <transaction id="9" comment="FZBNSLBEBZMTGPZJUG" ean_code="MNYYPKRM" price="7" items="5"/>
  <transaction id="7" comment="KIWPOOHNV" ean_code="CRXMHAGAC" price="3" items="7"/>
  <transaction id="9" comment="JETNYCMECHGUPSUKLSEA" ean_code="ICY" price="1" items="5"/>
</transactions>

```

Now we have `<transactions>`, but their attribute values are not necessarily meaningful for our application. We need to configure attribute generation, too. Note however, that Benerator understands the definition of custom data types like the 'price-type' and automatically generates valid data, though taking the easy way of defaulting to integral numbers.

10.2.2 Configuring Attribute Generation

Now it is time to configure the attribute details. Let us start by declaring the `id` attribute as ID:

```
<xs:attribute name="id" type="xs:long" use="required">
  <xs:annotation>
    <xs:appinfo>
      <ben:id/>
    </xs:appinfo>
  </xs:annotation>
</xs:attribute>
```

BTW: The XML Schema type ID is not yet handled automatically. You need to add an explicit `<ben:id/>` annotation for generating unique identifiers of the desired type.

Shorter random comments are generated based on a regular expression:

```
<xs:attribute name="comment" type="xs:string" use="required">
  <xs:annotation>
    <xs:appinfo>
      <ben:attribute pattern="[A-Z][a-z]{5,12}" />
    </xs:appinfo>
  </xs:annotation>
</xs:attribute>
```

You can configure number generation for the `items` attribute by setting `min`, `max`, `resolution` and `distribution` values:

```
<xs:attribute name="items">
  <xs:annotation>
    <xs:appinfo>
      <ben:attribute type="short" min="1" max="27" distribution="cumulated" />
    </xs:appinfo>
  </xs:annotation>
</xs:attribute>
```

This makes benenerator create `items` numbers from 1 to 27 with a cumulated distribution which has its maximum at 14.

10.2.3 Using `<variables>` in XML Schema

Now for more complex data generation: You can use `<variables>` like in descriptor files. They need to be placed inside an `<element>`. Let us, for example, use a CSV file with product definitions, containing EAN and price for each article. First, the variable declaration:

```
<xs:element name="transaction">
  <xs:annotation>
    <xs:appinfo>
      <variable name="product" source="products.ent.csv" distribution="random" />
    </xs:appinfo>
  </xs:annotation>
</xs:element>
...
```

For each generation of a transaction, the `<variable>` is called to generate a new helper object, in this case providing a CSV data line with product data. The contents of this data are mapped using script expressions:

```
...
<xs:attribute name="ean_code" type="xs:string" use="required">
  <xs:annotation>
    <xs:appinfo>
      <ben:attribute script="product.ean_code" />
    </xs:appinfo>
  </xs:annotation>
</xs:attribute>

<xs:attribute name="price" type="price-type" use="required">
  <xs:annotation>
    <xs:appinfo>
      <ben:attribute script="product.price" />
    </xs:appinfo>
  </xs:annotation>
</xs:attribute>
```

Using a CSV file with product definitions

```
ean_code,name,category_id,price,manufacturer
8000353006386,Limoncello Liqueur,DRNK/ALCO,9.85,Luxardo
```

```
3068320018430, Evian 1.0 l, DRNK/SOFT, 1.95, Danone
8076800000085, le Lasagnette, FOOD/MISC, 0.89, Barilla
7610400071680, Connaisseurs, FOOD/CONF, 16.95, Lindt
```

we finally get a satisfactory result:

```
<?xml version="1.0" encoding="UTF-8"?>
<transactions elementFormDefault="unqualified">
  <transaction id="1" comment="Sczeyozcpc" ean_code="8076800000085" price="0.89" items="9"/>
  <transaction id="2" comment="Nglija" ean_code="8000353006386" price="9.85" items="11"/>
  <transaction id="3" comment="Miejztqhgaoc" ean_code="7610400071680" price="16.95" items="14"/>
  <transaction id="4" comment="Tzoxzrupygjfi" ean_code="8000353006386" price="9.85" items="11"/>
  <transaction id="5" comment="Sufqdrku" ean_code="3068320018430" price="1.95" items="17"/>
  <transaction id="6" comment="Jbtqsft" ean_code="8000353006386" price="9.85" items="14"/>
  <transaction id="7" comment="Lvivrwwxmay" ean_code="8076800000085" price="0.89" items="10"/>
</transactions>
```

You might as well want to calculate the total price. You can easily do so using a script expression, e.g. `script="this.price * this.items"`. Note that the elements are evaluated and generated in the order in which they are declared, so the `total sum` field must be defined after the used terms `price` and `items`.

10.2.4 Importing Properties File Data

You can import settings from properties files by placing `<include>`s in the schema's root node annotation:

```
<xs:schema ...>
  <xs:annotation>
    <xs:appinfo>
      <ben:include uri="benerator.properties"/>
    </xs:appinfo>
  </xs:annotation>
  ...
</xs:schema>
```

10.2.5 Current Limitations

The current XML schema limitations are:

- No support for recursion of the same element type, e.g. categories containing other categories
- No support for mixed content. Benerator is concerned with the generation of data structures, while mixed-type documents generally apply to natural-language documents.
- groups are not supported
- sequences may not have `maxOccurs > 1`
- namespace support is only rudimentary, problems may arise on different types with equal names
- schema `include` is not supported yet
- ids and idrefs are not resolved automatically

10.3 Conclusion

Almost the full feature set of Benerator descriptor files is available for XML Schema-based file generation. If you know the [Benerator descriptor file](#) syntax well, it is a straightforward and relatively simple process to annotate XML Schema files.

However, if you just need to export XML-formatted data and write your own XML parser for importing the data somewhere else, you might prefer to use the [DbUnitEntityExporter](#) (flat structure) or [XMLEntityExporter](#) (hierarchical structure), possibly in combination with an XSL transformation.

You can as well combine the approaches: Annotating the XML Schema with Benerator descriptors and using its internal type definition in a Benerator setup with an [XMLEntityExporter](#).

11. Distribution Concepts

Distributions enable Benerator to generate numbers with desired distribution characteristics or following certain a sequence. A distribution may also be applied to groups of data objects to provide them with certain distribution characteristics.

Distributions come in two flavors:

- Sequences: Algorithm for generating numbers
- Weights: Functions which provide the probability of a certain number

For most common needs there exist predefined sequences and weights.

For special needs, you can define and add your own custom ones.

11.1 Basic Configuration

A Distribution is selected with a `distribution` attribute and can be parameterized with a `min` and `max` value and a `granularity`. The granularity is applied in a way, that any generated number is `min` plus an integer multiple of `granularity`.

Thus, a configuration

```
<attribute name="price" type="double" distribution="increment" min="0.25" max="100" granularity="0.25"/>
```

yields the numbers

```
0.25, 0.50, 0.75, 1.00, ..., 99.75, 100.00
```

11.2 Sequence

A Sequence is basically a number generator. It can provide a custom random algorithm, a custom weighted number generator or a unique number generation algorithm.

The Sequences used most often are `random`, `increment` and `cumulated`.

For a complete list of predefined sequences, see the '[Component Reference](#)', for defining a custom Sequence, see '[Extending Benerator](#)'.

11.3 Weight Functions

A weight function basically is a mathematical function that tells which weight to apply to which number. The most frequently used weight functions are `GaussianFunction` and `ExponentialFunction`.

For a complete list of predefined weight functions, see the [Component Reference](#), for defining a custom weight function, see [Extending Benerator](#).

11.4 WeightedNumbers

WeightedNumbers is a special component for creating a small set of numbers based on a weighted-number literal, for example `1^70, 3^30` for generating 70% `1` values and 30% `3` values.

This is a very convenient and simple approach for controlling parent-child cardinalities in nested data generation.

Example:

```
<attribute name="n" type="int" distribution="new WeightedNumbers('1^70,3^30')"/>
```

When using `WeightedNumbers` to determine the cardinality of an Entity part which is a container, then the container type must be declared. Typical settings are `container='array'` or, in some cases, `container='list'`:

```
<part name='y' container='array' countDistribution="new WeightedNumbers('0^70,1^20,2^10')">
  <attribute name='z' pattern='AAA' />
</part>
```

11.5 Distributing other data than numbers

'Other data' usually comes from a data source and is imported by an `<iterate>` or `<attribute>` element:

```
<attribute name="code" type="string" source="codes.csv"/>
```

When iterating through data (e.g. imported from file or database), Benerator's default behavior is to serve each item exactly once and in the order as provided. When the end of the data set is reached, Benerator stops.

When setting `cyclic="true"` Benerator serves the imported data consecutively too but does not stop when it reaches the end. Instead, it restarts iteration.

Beware: For SQL queries this means that the query is reissued, so it may have a different result set than the former invocation.

```
<attribute name="code" type="string" source="codes.csv" cyclic="true"/>
```

But that is not really a distribution. We can do better and get probability effects:

11.6 Importing Weights

When importing data from data sources, you can specify weights. They are different when importing simple data or entities:

11.6.1 Importing primitive data weights

When importing primitive data from a CSV file, each value is expected to be in an extra row. If a row has more than one column, the content of the second column is interpreted as weight. If there is no such column, a weight of 1 is assumed. Benerator automatically normalizes over all data objects, so there is no need to care about manual weight normalization. Remember to use a filename that indicates the weight character, using a suffix like `.wgt.csv` or `wgt.xls`.

If you, for example, create a CSV file `roles.wgt.csv`:

```
customer,7
clerk,2
admin,1
```

and use it in a configuration like this:

```
<generate type="user" count="100">
  <attribute name="role" source="roles.wgt.csv" />
</generate>
```

this will create 100 users of which about 70 will have the role `customer`, 20 `clerk` and 10 `admin`.

Alternative Delimiters for importing weights

By default, the semicolon is the delimiter between commands: Benerator splits imports commands by their delimiter. The default separator can be overwritten by the property `separator` e.g. `separator="|"`:

```
<generate type="user" count="100">
  <attribute name="role" source="roles.wgt.csv" separator="|" />
</generate>
```

It is also possible to specify the separator for the whole project in your `<setup>` node as

```
<setup defaultSeparator="|">

  <generate type="user" count="100">
    <attribute name="role" source="roles.wgt.csv" />
  </generate>

</setup>
```

11.6.2 Weighing imported entities by attribute

When importing entities, one entity attribute can be chosen to represent the weight by specifying `distribution="weighted[attribute-name]"`. Remember to indicate, that the source file contains entity data by using the correct file suffix, e.g. `.ent.csv` or `.ent.xls`.

Example: If you are importing cities and want to weigh them by their population, you can define a CSV file `cities.ent.csv`:

```
name,population
New York,8274527
Los Angeles,3834340
San Francisco,764976
```

and e.g. create addresses with city names weighted by population, when specifying

```
<generate type="address" count="100" consumer="ConsoleExporter">
  <variable name="city_data" source="cities.ent.csv" distribution="weighted[population]"/>
  <id name="id" type="long" />
  <attribute name="city" script="city_data.name"/>
</generate>
```

11.7 Distributing unweighted Data

If the imported data does not come with weight information, you can apply a Distribution to control probability:

```
<attribute name="code" type="string" source="codes.csv" distribution="random"/>
```

For WeightFunctions, all available data is loaded into RAM and then the Weight Function's number generation feature is used to generate indices of the data items.

Most Sequences implement data distribution as described above for Weight Functions, but can be programmed individually for each Sequence.

Attention: Most distributions load all available data to distribute into RAM.

Most sequences should not be applied to data sets of more than 100.000 elements, a weight function should be restricted to at most 10.000 elements.

'Unlimited' Sequences which are suitable for arbitrarily large data sets are

- expand
- randomWalk
- repeat
- step

For the complete list of predefined Distributions, see the '[Component Reference](#)', for programming custom Distributions, see '[Custom Sequences](#)' and '[Custom WeightFunctions](#)'.

12. Using Relational Databases

12.1 Import and `<database>`

For using database-related features, you must import the `db` package:

```
<import platform="db"/>
```

You can easily define a database:

```
<database id="db" url="jdbc:hsqldb:hsqldb://localhost" driver="org.hsqldb.jdbcDriver" user="sa" batch="false"/>
```

A database must have an `id` by which it can be referenced later. For starting a project, it is better to have `batch="false"`. In this mode, database errors are easier to track.

The following attributes are available in the `<database>` element:

name	description
<code>id</code>	identifier under which the database is accessible in the context (required)
<code>environment</code>	Identifier of a database repository entry
<code>url</code>	JDBC database url
<code>driver</code>	JDBC driver class
<code>user</code>	user name for login
<code>password</code>	password for login
<code>catalog</code>	Database catalog to use
<code>schema</code>	database schema to use
<code>includeTables</code>	Regular expression for tables to be used or <code>#all</code> for multi schema inclusion (table, index, keys)
<code>excludeTables</code>	Regular expression for tables to be ignored
<code>lazy</code>	boolean flag to enable lazy metadata parsing. This improves performance on large systems of which only a small number of tables are actually used in generation.
<code>metaCache</code>	boolean flag which can be activated on databases with slow database access to cache database metadata on the local file system instead of reparsing it on each run
<code>batch</code>	boolean flag to specify if batch inserts and updates shall be done
<code>fetchSize</code>	JDBC fetch size for query results
<code>readOnly</code>	indicates if only read access shall be allowed in order to protect sensitive data
<code>acceptUnknownColumnTypes</code>	If set to true, Benerator accepts exotic database column types without complaining and relies on the user to take care of the appropriate data type when generating values for the column.



Note

Benerator has some built-in knowledge about the most widely used database systems and their conventions. So in most cases, it is sufficient to provide url, driver, user, and password. In special cases, e.g. if you want to access a schema that is not the default schema of your user, you may have to set schema (and possibly catalog) explicitly.

12.2 Usual Database Settings

Database	URL Format	Driver Class	Default Catalog	Default Schema
DB2	jdbc:db2://host:50001/ dbname	com.ibm.db2.jcc.DB2Driver		<user_name>
Derby	jdbc:derby://host:1527/ dbname			<user_name>
Firebird	jdbc:firebirdsql:host/ 3050:dbname	org.firebirdsql.jdbc.FBDriver		
H2	Jdbc:h2:....	org.h2.Driver		PUBLIC
HSQL	jdbc:hsqldb:hsqldb://host:9001/ dbname	org.hsqldb.jdbcDriver		PUBLIC
MySQL	jdbc:mysql://host:3306/ dbname	com.mysql.jdbc.Driver	<user_name>	
Oracle	jdbc:oracle:thin:@host: 1521:SID	oracle.jdbc.driver.OracleDriver		<user_name>
Postgres	jdbc:postgresql://host:5432/ dbname	org.postgresql.Driver	<user_name>	public
SQL Server	jdbc:jtds:sqlserver://host: 1433/dbname	net.sourceforge.jtds.jdbc.Driver	master	dbo

12.3 Using Database Repositories

For frequently-used databases it is more convenient to use a central environment configuration repository and have Benerator look up all the system connection details by a simple system name instead of having to copy & paste JDBC settings again and again.

This way, a database definition turns from

```
<database id="db" url="jdbc:oracle:thin:@host:1521:SID"
  driver="oracle.jdbc.driver.OracleDriver"
  user="sam" password="sam123"
  catalog="CRM01" schema="CUSTS" readOnly="true"/>
```

to

```
<database id="db" environment="dev" system="crm"/>
```

where `environment` denotes the environment definition file and `crm` a system configuration within that file.

A global standard location for a system repository is located in a folder `rapiddweller` under your user home directory (like `c:\Documents and Settings\<user_name>\` on Windows)

You can as well define a project-specific environment repository in a sub directory `conf` under the project folder.

For details, see [Environment Files](#).

12.3.1 Overriding Environment Settings

When using a system definition in an environment file and specifying different attributes in the XML file, (like another user and password), these override the configuration details in the database repository. This way you can have a central and convenient database lookup and can access the database with different users in the same run. An example:

```
<database id="db1" environment="mydb" user="user1" password="password1"/>
```

```
<database id="db2" environment="mydb" user="user2" password="password2"/>
```

12.4 Iterating through database tables

For importing or anonymizing data, you need to iterate through database tables. This is configured easily by using the database id as `source` and the table name as `type`:

```
<database id="db" environment="dev" system="crm"/>
<iterate source="db" type="customers" consumer="ConsoleExporter"/>
```

For iterating over a data sub set, specify a `selector` which may take the content of a regular SQL 'where' clause:

```
<database id="db" environment="dev" system="crm"/>
<iterate source="db" type="customers" selector="level > 5 AND cust_type = 'PRIVATE'" consumer="ConsoleExporter"/>
```

12.5 Caching Database Metadata

On very large databases, especially when accessed remotely, database metadata retrieval may take several minutes. In this case, you can make use of the `metaCache` facility.

Two preconditions exist for using meta data caching: You need to

1. configure a database repository (`environment`) and
2. set `metaCache` to `true`

```
<database id="db2" environment="mydb" metaCache="true"/>
```

On the first run, you will not observe any speedup – metadata parsing may even take longer since the cache needs to be created on the first run and Benerator is likely to read (much) more metadata than you absolutely need for your specific data generation. When done so, Benerator saves the metadata in an XML file. On subsequent runs, Benerator notices the cache file and reads it within milliseconds.

Cache Invalidation: There are several reasons that make Benerator invalidate its cache information and reload it:

- **Execution of SQL code via `<execute>`**: Benerator is not so clever about interpreting SQL code. Thus, it interprets any executed SQL code as a potential metadata change and invalidates the cache.
- **Cache time out:** If the cache file is older than twelve hours, Benerator throws it away just in case. If you are sure that the database has not changed meanwhile, you can perform a 'touch' on the cache file.

Warning

If you change the database structure from another client system and Benerator is configured to cache metadata, there is no way to become aware of it and the old metadata cache file is used which has become obsolete. You need to delete the cache file manually in such cases!

12.6 Executing SQL statements

SQL code can be executed, e.g. from a file:

```
<execute uri="drop-tables.sql" target="db" onError="warn"/>
```

Uris are resolved relative to the benerator file that declares them (as common in HTML). If the file is not found locally, it is searched relative to the current working directory.

You can inline SQL code as well:

```
<execute target="db" onError="warn">
  CREATE TABLE db_role (
    id int NOT NULL,
    name varchar(16) NOT NULL,
    PRIMARY KEY (id)
  );
</execute>
```

12.6.1 Alternative Delimiters

By default, the semicolon is the delimiter between commands: Benerator splits SQL commands by their delimiter and sends one after the other to the database. In some cases, you need different behavior, e.g. if a procedure should be defined and/or called. In such cases, you can specify an alternative delimiter in an `<execute>` statement:

```
<execute target="db" separator="/">
  declare output_var varchar(500);
  begin
    EXECUTE_PROC_A(output_var);
    EXECUTE_PROC_B(output_var);
  end;
</execute>
```

12.7 Inserting entities into a database

When using a database as consumer in a `<generate>` or `<iterate>` element, the elements are *inserted* by default. For information, how to *update* entries, see the next chapter.

```
<generate type="db_user" count="50000" consumer="db"/>
```

If primary key generation should be performed by the database, you need to tell Benerator to ignore the field, setting the mode to 'ignored'

```
<generate type="db_user" count="50000" consumer="db">
  <id mode="ignored"/>
</generate>
```

12.8 Database-related Id Generators

The following id generators make use of database features:

- **DBSequenceGenerator:** Retrieves id values from a database sequence. With default settings, it operates quite slowly, since it incurs an additional database call for obtaining the id value for each generated entity. When setting its property `cached` to `true`, it fetches the current value of its database sequence, creates ids offline in Benerator RAM and updates the database sequence in the end. Of course, this requires Benerator to run in a single instance and no other client may be writing data to the system while Benerator is generating – otherwise, a primary key conflict may arise.
- **DBSeqHiLoGenerator:** Combines a value retrieved from a database with a local counter to create unique values (with a strongly reduced performance burden compared to the DBSequenceGenerator)
- **QueryGenerator:** Uses a database query to calculate id values
- **QueryLongGenerator:** Uses a database to calculate id values of number type
- **QueryHiLoGenerator:** Works like a DBSeqHiLoGenerator, but based on a query instead of a sequence
- **SequenceTableGenerator:** Lets you read and increment values from database tables

The best performance with cluster-safe generators is achieved with the `DBSeqHiLoGenerator`, followed by the `QueryHiLoGenerator`.

12.8.1 SequenceTableGenerator

The SequenceTableGenerator lets you create unique long values from a database table. Depending on the table structure this can be trivial or tricky.

You always need to specify a **database**, a **table** and a **column** from which to read the value, in non-trivial cases, you also need a selector.

Single-Value-Table

In the simplest case, you have a table which stores a single row with a single value:

```
<setup>
  <database id="db" environment="mydb"/>

  <bean id="sg" class="SequenceTableGenerator">
    <property name="database" ref="db"/>
    <property name="table" value="MY_TABLE"/>
    <property name="column" value="SEQ_VALUE"/>
  </bean>
  <generate type="PERSON" count="100" consumer="db">
    <id name="id" type="long" generator="sg"/>
    ...
  </generate>
</setup>
```

Name-Value-Pair Table

In a slightly more difficult case, you have name-value-pairs of 'sequence identifier' and 'sequence value'. Then you must specify a selector, that tells Generator which row to use. For example, if the sequence for the PERSON table is specified by a row in which the SEQ_ID column has the value PERSON :

```
<setup>
  <database id="db" environment="mydb"/>

  <bean id="sg" class="SequenceTableGenerator">
    <property name="database" ref="db"/>
    <property name="table" value="MY_TABLE"/>
    <property name="column" value="'SEQ_VALUE'"/>
    <property name="selector" value="SEQ_ID = 'PERSON'"/>
  </bean>

  <generate type="PERSON" count="100" consumer="cons">
    <id name="id" type="int" generator="sg"/>
  </generate>
</setup>
```

Arbitrary Table

You can support arbitrary complex sequence tables with a **parameterized selector**. It marks each parameter with a question mark (?) and must be invoked differently than the examples above, using a **script** that calls the **generateWithParams(...)** method:

```
<setup>
  <database id="db" environment="mydb"/>

  <bean id="sg" class="SequenceTableGenerator">
    <property name="database" ref="db"/>
    <property name="table" value="MY_TABLE"/>
    <property name="column" value="'SEQ_VALUE'"/>
    <property name="selector" value="MOD_ID = ? and ITEM_ID = ?"/>
  </bean>

  <generate type="PERSON" count="100" consumer="cons"> <!-- calculate mid and iid as required -->
    <id name="id" type="int" script="sg.generateWithParams(mid, iid)"/>
  </generate>
</setup>
```

12.9 Handling of common Columns

In many databases, you encounter common columns like auditing information `created_by`, `created_at`, `updated_by`, `updated_at` or optimistic locking columns.

See [Default Attribute Settings](#) for instructions on how to define common default generation settings for these.

12.10 Determining attribute values by a database query

You can use database queries to determine column values. A **source** attribute identifies the database to query and a selector the SQL query to perform:

```
<attribute name="user_rank" source="db" selector="select rank from ranks where active = 1"/>
```

You can use **source** and selector in `<attribute>`, `<id>`, `<reference>` and `<variable>` statements.

Warning

With this syntax, the query's result set is iterated throughout the `<generate>` loop until its end is reached. In the example above, a result set with the rows [1], [2], [3] will result in the `user_rank` values 1 for the first generated entry, 2, for the second and 3 for the third. After that the end of the result set is reached, the component signals, that it is unavailable and Benerator will terminate the generation loop. If you configured more than 3 objects to be generated, you will get an exception that Benerator was not able to provide the requested number of data sets. You have the following alternatives:

1. Cycling through the result set again and again (`cyclic="true"`)
2. Apply a distribution choose a mode for selecting elements of the result set repeatedly or randomly
3. If the query should be performed for each generated entity and is supposed to provide a single result, this is called sub-query and is supported by a 'subQuery' attribute

12.10.1 Cycling through query result sets

When using a selector in combination with `cyclic="true"`, the query is automatically repeated when the end of the result set is reached:

```
<attribute name="user_rank" source="db" selector="select rank from ranks where active = 1" cyclic="true"/>
```

12.10.2 Applying a distribution to a query

When using a selector in combination with **distribution**, the query's result set is processed by the selected distribution algorithm. Depending on the distribution, the result set may be buffered. Result set elements may be provided uniquely or repeatedly in an ordered or a random fashion. See the [Distributions Reference](#). The most common distribution is the `random` distribution which buffers the full result set and then provides a randomly chosen entry on each invocation.

```
<attribute name="user_rank" source="db" selector="select rank from ranks where active = 1" distribution="random"/>
```

12.10.3 Sub selectors

Frequently you will encounter multi-field constraints in an entity which can be matched by a query. Usually this means to first generate a random value and then to impose a database query with the value generated before. The query's results are valid only for the currently generated entity, and in general only one query result row is expected. You have a kind of „sub-query“ which is handled best by using a **subSelector**.

For example, you might offer products (table 'product') in different geographical regions and have a cross-reference table `product_region` that describes, which products are available in which region:

```
<generate name="example" type="ex" count="5" consumer="ConsoleExporter">
  <attribute name="region" values="'americas','emea','asia'"/>

  <reference name="product" source="db"
    subSelector="{{'select product_id from product_region where region_id = ' + this.region }}" />
</generate>
```



In the example, you need double brackets `{{...}}` in order to signal that **this.region** should be reevaluated on each invocation. When using a single bracket, the query memorizes the value of **this.region** at the first invocation and reuses it on each subsequent call.

12.11 Resolving Database Relations

12.11.1 Automatic referencing

By default, Benerator assumes that all relations are **one-to-one** as the most defensive choice. Thus, the following setup in which a table `db_user` references a table `db_role` will cause an error:

```
<generate type="db_role" count="10" consumer="db"/>
```

```
<generate type="db_user" count="100" consumer="db"/>
```

This is because, assuming a **one-to-one** relationship, you **can only** generate as many users as unique roles are available! ...and you have generated only 10 roles before. In other words, in fully automatic data generation, the number of user entries will be the number of role entries.

In most cases you actually deal with many-to-one relationships and thus need to specify its characteristics explicitly, typically by using a distribution. Basically, a reference is defined by (column) **name**, (database) **source** and **targetType** (referenced table):

```
<setup>
  <generate type="db_role" count="10" consumer="db"/>

  <generate type="db_user" count="100" consumer="db">
    <reference name="role_fk" targetType="db_role" source="db" distribution="random"/>
  </generate>
  ...
</setup>
```

This will cause the creation of 100 users which are evenly distributed over the roles.

12.11.2 Null references

If you want to generate only null values, you can reduce the declaration to a `name` and `nullQuota="1"` element:

```
<generate type="db_user" count="100" consumer="db">
  <reference name="role_fk" nullQuota="1"/>
</generate>
```

12.11.3 Selective Referencing

For restricting the objects referred to or using an individual way to construct the reference values, you can specify a selector which will be evaluated by the target system and return a reference value. For databases the selector needs to be a SQL where clause or complete query:

```
<setup>
  <generate type="db_role" count="10" consumer="db"/>

  <generate type="db_user" count="100" consumer="db">
    <reference name="role_fk" targetType="db_role" source="db" selector="role_name != 'admin'"
      distribution="random"/>
  </generate>
  ...
</setup>
```

12.11.4 Other referencing options

Besides selective referencing, you can use (almost) the full feature set of `<attribute>` elements to generate references, e.g. `constant`, `pattern`, `values`, `script`, etc. You could, e.g., configure the use of each role type by itself:

```
<setup>
  <generate type="db_user" count="5" consumer="db">
    <reference name="role_fk" constant="'admin'"/>
    ...
  </generate>

  <generate type="db_user" count="95" consumer="db">
    <reference name="role_fk" constant="'customer'"/>
    ...
  </generate>
  ...
</setup>
```

12.12 Composite Keys

Generator does not provide an **automated** composite key handling, but you can configure it to handle them **explicitly**. The typical approach for this is a prototype query.

12.13 Prototype Queries

For the general idea of prototype-based generation, see [Prototype-based Data Generation](#). In addition to the core features, the prototype approach is a good way to handle composite primary keys and composite foreign keys, since their components are available in combination.

12.13.1 Prototype Queries on Entities

When querying entities, you specify the database to query as `source`, the where clause of the select as `selector`, and the table which is queried as `type`. After that, you can access the results' attributes by their names:

```
<generate name="example" type="ex" count="5" consumer="ConsoleExporter">
  <variable name="_product"
    type="MY_PRODUCT"
    source="db"
    selector="sysdate between VALID_FROM and VALID_TO"
    distribution="random"/>

  <reference name="PRODUCT_ID" script="_product.PRODUCT_ID"/>

  <attribute name="ROUTING_TYPE" script="_product.ROUTING_TYPE"/>
</generate>
```

Note

When aggregating data with a general query that is not (or cannot be) mapped to a type, you can access the results' column values as array elements (indices are 0-based):

```
<variable name="_product" source="db"
  selector="select PRODUCT_ID, ROUTING_TYPE from MY_PRODUCT where sysdate between VALID_FROM and VALID_TO"
  distribution="random"/>

<generate name="example" type="ex" count="5" consumer="ConsoleExporter">
  <reference name="PRODUCT_ID" script="_product[0]"/>

  <reference name="ROUTING_TYPE" script="_product[1]"/>
</generate>
```

12.14 Exporting Database Content

The rows of a database table can be iterated simply. Here's an example for writing all users of a table `db_user` to a file `users.csv`:

```
<iterate source="db" type="db_user" consumer="new CSVEntityExporter('users.csv')"/>
```

You can as well select a subset to iterate:

```
<iterate source="db" type="db_user" selector="active = 1" consumer="new CSVEntityExporter('users.csv')"/>
```

12.15 Updating Database Entries

When using a database, you can update existing entries.

For example, if you have `db_orders` that refer (one-to-many) `db_order_items`, you can calculate the sum of the `db_order_items`' `total_price` values and write it to the `db_order`'s `total_price` column:

```
<iterate type="db_order" source="db" consumer="db.updater()">

  <attribute name="total_price" source="db"
    selector="{{ftl:select sum(total_price) from db_order_item where order_id = ${db_order.id}}}"
    cyclic="true"/>

</iterate>
```

The described update mechanism can also be used to anonymize production data – see [Production Data Anonymization](#). For transferring user data from a source database `sourcedb` to a target database `testdb`, you would write

```
<iterate source="sourcedb" type="db_user" consumer="testdb">

  <!-- anonymize here -->

</iterate>
```

If you have duplicated the database and want to anonymize the copy by updating the tables, you would write

```
<iterate type="db_user" source="db" consumer="db.updater()">

  <!-- anonymize here -->

</iterate>
```

12.15.1 Overwrite output table name with inserter

If you want to read data from one table, anonymize it and write it to a different table in the same database, you can use a special inserter:

```
<iterate type="prod_user" source="db" consumer="db.inserter('anon_user')">

  <!-- anonymize here -->

</iterate>
```



`db.inserter()` is the only way to overwrite the target table name for the consumer. Otherwise, Benerator will always take the type as name and assumes both source and target table have the same name.

Using `db.inserter()` might also be helpful for creating staging tables from multiple sources and persisting these in e.g. H2 databases.

12.16 Controlling Transactions

By default, Benerator performs one transaction per data set that is generated. When generating huge amounts of data, transaction overhead quickly becomes significant, and you will want to insert several data set in a common transaction. You can use the `pageSize` argument to configure the number of data elements per transaction. For most databases and tasks, `pageSize="1000"` is a reasonable setting:

```
<generate type="user" count="1000000" pageSize="1000" consumer="db"/>
```

For further hints on improving performance, refer to the [Performance](#) section.

If you are nesting creation loops, you can set the transaction control for each level separately:

```
<generate type="user" count="1000" pageSize="100" consumer="db">
  <generate type="order" count="50" pageSize="500" consumer="db"/>
</generate>
```

But an 'inner' transaction commit will commit the outer elements too, so you may get more transactions than you expect. The inner pageCount in the descriptor example makes the outer pageSize 10 effective, since there is a commit after 500 orders. With 50 orders per customer, it is a commit for every 10th customer.

In most cases it is feasible and more intuitive to make the sub creation loops simply join the outer transaction control, by setting their `pageSize` to zero:

```
<generate type="user" count="1000" pageSize="100" consumer="db">
  <generate type="order" count="50" pageSize="0" consumer="db"/>
</generate>
```

Any `<generate>` loop with `pageSize > 0` is flushed when finished. For databases this means a commit.

12.17 Transcoding Database Data

Benerator's transcoding feature enables you to

1. copy database entries from one database to another
2. assign new primary key values while copying
3. transcode relationships (automatically translate foreign key relationships to the new primary key values)
4. merge relationships (make copied data refer to pre-existing data in the target database)

Features 1-3 are can be performed easily, for feature 4 you need to fulfill some preconditions.

12.17.1 Copying database entries

A transcoding task that involves one or more database tables is wrapped with a `<transcodingTask>` and for each table to be transcoded, there needs to be a `<transcode>` element that specifies the table name. The `<transcode>` steps must be specified in the order in which they can be applied without violating foreign key constraints. For example, if you have a table USER which references a table ROLE, you need to transcode ROLE first, then USER.

```
<transcodingTask defaultSource="db1" target="db2">
  <transcode table="ROLE"/>
  <transcode table="USER"/>
</transcodingTask>
```

This copies the full content of the tables ROLE and USER from db1 to db2.

12.17.2 Restricting the copied Set

You can restrict the set of database entries to copy by using a `selector` attribute in the `<transcode>` element:

```
<transcodingTask defaultSource="s" target="t">
  <transcode table="ROLE" selector="ID = 1"/>
  <transcode table="USER" selector="ROLE_ID = 1"/>
</transcodingTask>
```

Only the ROLE with id 1 and the USERS that refer role #1 are copied.

12.17.3 Transcoding

You can overwrite primary key values and other attributes while you are transferring data using the normal Benerator syntax:

```
<transcodingTask defaultSource="s" target="t">

  <transcode table="ROLE">
    <id name="id" generator="IncrementalIdGenerator"/>
  </transcode>

  <transcode table="USER">
    <id name="id" generator="IncrementalIdGenerator"/>
  </transcode>

</transcodingTask>
```

Each ROLE and USER gets a new primary key value, and the foreign key references from USER to ROLE are reassigned to match the new id values.

12.17.4 Cascaded Transcoding

As an easy approach to transcode graphs of dependent objects along with their parent object, they can be transcoded in cascade along with their 'owner' object. This means, for each transcoded owner object, Benerator looks up, which database rows relate to it as defined in the cascade and transcodes them too. Thus, if you restrict the owners (e.g. company) to a subset of all available owners, the cascade statement assures that only database rows (e.g. department), which relate to this subset (company), are transcoded.

A cascade statement consists of `<cascade>` element nested in a `<transcode>` element specifying a ref that tells, which columns of which table make up the table relationship. The Syntax is `table(column1 [, column2 [, ...]])`, depending on the number of columns used as a foreign key. Benerator looks up the corresponding foreign key constraint in the database and finds out the type of relationship.

As an example, if you want to transcode the rows of a company table and cascade to their departments, you would write

```
<transcode table="company">
  <cascade ref="department(company_id)"/>
</transcode>
```

Cascades can be nested:

```
<transcode table="company">

  <cascade ref="department(company_id)">
    <cascade ref="employee(department_id)"/>
  </cascade>

</transcode>
```

In `<cascade>`, you can overwrite attributes, ids and references like in `<transcode>`:

```
<transcode table="company">

  <id name="id" generator="IncrementalIdGenerator"/>
  <cascade ref="department(company_id)"/>

</transcode>
```

12.17.5 Merging Foreign Key Relations

Benerator allows you to merge data from different databases. To continue the example above, you could have ROLES and USERS in different databases and merge them into one single target database. This introduces a new requirement: Since you might have automatically assigned technical ids, an object with a given 'business' identity (e.g. user 'Volker Bergmann') might have different 'technical' ids (primary keys of value e.g. 10 or 1000) in different databases. Thus, you need to provide Benerator with a description, which technical id relates to which business id in which database.

There are several alternatives available: Let us start with one of the simplest, and most widely used in order to give you an overview of the approach and then provide you with a complete list of possibilities.

Let's assume the tables `ROLE` and `USER` each have a `NAME` column with unique identifiers. In this case, you can apply the unique-key identity mapping and store it in a file with the suffix `.id.xml`, e.g. `identities.id.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>

<dbsanity>
  <identity table="ROLE" type="unique-key" columns="NAME"/>
  <identity table="USER" type="unique-key" columns="NAME"/>
</dbsanity>
```

This identity file enables Benerator to merge foreign key relationships in the transcoding process, for example:

```
<transcodingTask target="db3" identity="identity-def.xml">
  <transcode table="ROLE" source="db1"/>
  <transcode table="USER" source="db2"/>
</transcodingTask>
```

12.17.6 Defining Identities

Identities are defined in a file format taken from DB Sanity. Thus, it is XML and its root element is `<dbsanity>`. Under the root, all `<identity>` definitions are listed, each referring to a **table** and having a certain identity **type**, e.g.

```
<?xml version="1.0" encoding="UTF-8"?>

<dbsanity>
  <identity table="ROLE" type="..." />
  <identity table="USER" type="..." />
</dbsanity>
```

There are different types of entity definitions available:

natural-pk

The simplest type of identity applies if the primary key (PK) is a business key (natural key). In this case, use a `<natural-pk>` definition:

```
<identity table="CURRENCY" type="natural-pk"/>
```

unique-key

If a non-pk-column or a combination of several columns is unique, use the `<unique-key>` identity and list the key components in a comma-separated list in a `columns` attribute:

```
<identity table="PRODUCT" type="unique-key" columns="CATEGORY, CODE"/>
```

You may as well include foreign keys in the list: Benerator will use the business id of the referenced objects to determine the identity of the referer.

nk-pk-query

In more complicated cases, you might need a definition style that gives you more freedom. The `<nk-pk-query>` allows you to specify an arbitrary SQL query which is supposed to return a pair of natural key and primary key values for each table row:

```
<identity table="COUNTRY" type="nk-pk-query">

  select COUNTRY_NAME.NAME as NK, COUNTRY.ID as PK from COUNTRY_NAME join COUNTRY on COUNTRY_NAME.COUNTRY_ID =
  COUNTRY.ID where COUNTRY_NAME.LANGUAGE = 'EN'

</identity>
```

sub-nk-pk-query

A table's rows may have a complex identity definition, which is only unique in the context of a 'parent' row in another table. A good example is if a state name is only unique within a given country, not different countries may have a state of the same name, but different identities. The provided query must have the parent's primary key as ? parameter:

```
<identity type="sub-nk-pk-query" table="" parents="COUNTRY">

    select sn.STATE_NAME as SUB_NK, s.STATE_ID as PK from STATE s join STATE_NAME sn on sn.STATE_ID = s.STATE_ID where
    sn.LANGUAGE_ID = 'ENG' AND **s.COUNTRY_ID = ?**

</identity>
```

12.17.7 Limitations

Currently, the amount of data that can be transcoded is limited by the amount of available Java heap memory, but as long as you do not transcode billions of data sets you are not supposed to get problems. Composite primary keys are not supported.

12.17.8 Multi-schema references

Benerator is scanning data sources for references and validity and check if all data types can be handled by Benerator in general.

When you have a PostgreSQL database for example with the following data model:

```
CREATE SCHEMA schema1;
CREATE SCHEMA schema2;
CREATE SCHEMA schema3;
CREATE SEQUENCE schema1.seq_id_gen START WITH 10;
CREATE SEQUENCE schema2.seq_id_gen START WITH 10;
CREATE SEQUENCE schema3.seq_id_gen START WITH 20;

CREATE TABLE schema3.db_manufacturer (
id SERIAL PRIMARY KEY,
name varchar(30) NOT NULL,
description text NULL
);
CREATE TABLE IF NOT EXISTS schema1.db_category (
id SERIAL PRIMARY KEY,
name varchar(30) NOT NULL
);
CREATE TABLE schema2.db_category (
id SERIAL PRIMARY KEY,
name varchar(30) NOT NULL
);
CREATE TABLE schema1.db_product (
ean_code varchar(13) NOT NULL,
name varchar(30) NOT NULL,
category_id int NOT NULL,
manufacturer_id int NOT NULL,
price decimal(8,2) NOT NULL,
notes varchar(256) NULL,
description text NULL,
CONSTRAINT db_product_category_fk FOREIGN KEY (category_id) REFERENCES schema1.db_category (id),
CONSTRAINT db_manufacturer_fk FOREIGN KEY (manufacturer_id) REFERENCES schema3.db_manufacturer (id)
);
COMMIT;
```

you would have related data distributed into different schema. In Benerator script you would define 3 different data sources ...

```
<setup>

<database id="schema1"
url="{dbUrl}"
driver="{dbDriver}"
schema="schema1"
user="{dbUser}"
password="{dbPassword}"
batch="{dbBatch}"/>

<database id="schema2"
url="{dbUrl}"
driver="{dbDriver}"
schema="schema2"
user="{dbUser}"
password="{dbPassword}"
batch="{dbBatch}"/>

<database id="schema3"
url="{dbUrl}"
driver="{dbDriver}"
schema="schema3"
```

```

        user="{dbUser}"
        password="{dbPassword}"
        batch="{dbBatch}"/>
    ...
</setup>

```

when you try to fill one of the tables, Benerator scans your data source and checks if all references are accessible. In this case, data source **schema1** has one table **db_product** with a foreign key reference into schema 3. The new mechanism is getting the foreign keys from your actual scope ...

```

<database id="schema1"
  url="{dbUrl}"
  driver="{dbDriver}"
  schema="schema1"
  user="{dbUser}"
  password="{dbPassword}"
  batch="{dbBatch}"/>

```

schema1 in this case. It contains one foreign key into a foreign schema ('db_product'), based on this meta information, Benerator decides what else is necessary to import into your scope. In this case it would be **schema3**. The flag `includeTable="#all"` is not necessary anymore!

Known issue

There is a known limitation when it comes to multischema. If there are two tables with the same name in different schemas and both schemas are imported into your Benerator context, like

```

<database id="schema1" url="{dbUrl}" driver="{dbDriver}" schema="schema1" user="{dbUser}" password="{dbPassword}" />
<database id="schema2" url="{dbUrl}" driver="{dbDriver}" schema="schema2" user="{dbUser}" password="{dbPassword}" />

```

you might get an error because Benerator can't decide which table you want to access.

As workaround exclude one of the conflicting table by using `excludeTable="db_user"` as below:

```

<database id="schema1" url="{dbUrl}" driver="{dbDriver}" schema="schema1" user="{dbUser}" password="{dbPassword}" />
<database id="schema2" url="{dbUrl}" driver="{dbDriver}" schema="schema2" user="{dbUser}" password="{dbPassword}" excludeTable="db_user"/>

```

13. Message Queue Access (Enterprise Edition)

Virtually any system that supports access via a JMS interface, can be used for data import or export in Benerator. It has been tested with ActiveMQ and RabbitMQ.

13.1 Basic configuration

The base element of message queue access is a `<jms-destination/>`. It must have each of the following attributes set:

Name	Description	example
id	the internal identifier by which it can be accessed from other Benerator elements	'orderQueue'
factory	the Java class used as JMS entry point of the used JMS library	'org.apache.activemq.ActiveMQConnectionFactory'
url	a URL specific for the used JMS library	'tcp://localhost:61616'
name	the public name of the queue	'orderQueue'
type	the type of queue, either 'topic' or 'queue'	'queue'
format	the format used for the messages, currently, 'json' or 'map'	'json'

As an example, an ActiveMQ topic of name 'newOrders' at the local system which uses JSON-formatted messages may be configured as

```
<jms-destination id='userQueue' factory='org.apache.activemq.ActiveMQConnectionFactory'
url='tcp://localhost:61616' name='newUsers' type='topic' format='json' />
```

The internal id 'orderQueue' will later be used to access the queue for reading or writing.

Each JMS provider supports individual URL formats, some with variants for different purposes. Please refer to the JMS library documentation for details. Some JMS API providers use the word URI instead of URL, but don't let that confuse you.

For detailed information on ActiveMQ URL formats, see the '[ActiveMQ web page](#)', for RabbitMQ, see the '[RabbitMQ web page](#)'.

13.2 Export

In order to export data, the JMS destination defined above can be used as `consumer` :

```
<generate type='person' count='10' consumer='userQueue'>
  <attribute name='firstName' pattern='Alice|Bob|Charly' />
</generate>
```

13.3 Import

```
<iterate source='userQueue' type='person' count='10' consumer='ConsoleExporter' />
```

13.4 Playground

For getting acquainted with JMS access and play around on an offline system, you can use temporary ActiveMQ queues which are available without the hassle of installing ActiveMQ on your system. When using the URL `vm://localhost?broker.persistent=false`, the driver itself creates an in-process queue, and you can experiment with writing data to it and retrieving it again.

So, here is an example setup which

1. defines a queue
2. <generate>s person data and sends it to an ActiveMQ topic,
3. then <iterate>s the queue and prints the retrieved person data to the console

```
<setup>
  <jms-destination id='userQueue' factory='org.apache.activemq.ActiveMQConnectionFactory'
    url='vm://localhost?broker.persistent=false' name='newUsers' type='topic' format='json' />

  <generate type='person' count='10' consumer='userQueue'>
    <attribute name="name" pattern="Alice|Bob|Charly"/>
    <attribute name="age" type="int" min="18" max="67"/>
  </generate>

  <iterate source='myqueue' type='person' count='10' consumer='ConsoleExporter' />
</setup>
```

14. Kafka Access (Enterprise Edition)

Benerator can read from and write to Kafka queues.

14.1 Basic configuration

The basic configuration elements required for any kind of Kafka connector are

Name	Description
id	the internal id by which the Kafka connector can be accessed in Benerator
bootstrap.servers	See the original Kafka Documentation
topic	The name of the Kafka queue to connect to
format	The message format used in this queue. Currently, only 'json' is supported

Depending on the system you want to connect to, you might need to provide [Advanced configuration](#).

14.2 Export

For exporting data, a `<kafka-exporter>` is used:

```
<setup>
  <kafka-exporter id='exporter' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json' />
  <generate type='person' count='10' consumer='exporter'>
    <attribute name="name" pattern="Alice|Bob|Charly"/>
    <attribute name="age" type="int" min="18" max="67"/>
  </generate>
</setup>
```

14.3 Import

For importing data, a `<kafka-importer>` is used:

```
<setup>
  <kafka-importer id='importer' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json' />
  <iterate source='importer' type='person' count='10' consumer='ConsoleExporter' />
</setup>
```

14.4 Advanced configuration

14.4.1 Kafka properties

Benerator supports all configuration properties of Kafka 2.8. For an in-depth explanation of these, please have a look at the [Kafka Documentation](#).

14.4.2 encoding

By default, Benerator uses UTF-8 encoding to read and write JSON.

If you need to use another encoding like UTF-16, specify it as `encoding` parameter:

```
<kafka-exporter id='exporter' bootstrap.servers='localhost:9094' topic='kafka-demo'
  format='json' encoding='UTF-16' />
```

14.4.3 auto.offset.reset

When connecting to a topic, a client declares which group it belongs to and the group is assigned a message offset, which is stored on the server. When messages with a higher offset exist in the queue or are sent to it, the Kafka server sends them to the client and increases the saved offset.

The client declares its group using the setting `group.id`, which Benerator sets to `default`, if not provided explicitly.

When a client reconnects, the Kafka server remembers the client group's offset and delivers the new messages.

This is the good case. In some situations, this method has issues:

- New client group
- Topic was deleted and recreated
- messages have been deleted
- You name it

Therefore, a client must declare an offset reset strategy (`auto.offset.reset`) to be applied in such a case. The setting can have the following values

- `earliest`: automatically reset the offset to the earliest offset
- `latest`: automatically reset the offset to the latest offset
- `none`: throw exception to the consumer if no previous offset is found for the consumer's group

14.4.4 key.attribute

In order to tell a Kafka exporter which data to use as key to the exported message, specify the attribute name as `key.attribute`, for example using an order priority as key:

```
<setup>
<kafka-exporter id='exporter' bootstrap.servers='localhost:9094' key.attribute='priority' topic='kafka-demo' format='json' />
<generate type='order' count='100' consumer='exporter'>
  <attribute name='priority' type='string' script='(high|medium|low)' />
  <attribute name='orderId' type='long' distribution='increment' />
  <attribute name='productId' type='long' min='1' max='300' />
</generate>
</setup>
```

14.4.5 key.serializer and key.deserializer

For de/serializing of message keys, Kafka provides different classes, each defined in Kafka's package `org.apache.kafka.common.serialization`. For sending messages a serializer is used, for receiving messages a deserializer.

By default, Benerator uses Kafka's `StringSerializer` and `StringDeserializer`.

If necessary, you can use other de/serializer classes, specifying the `key.serializer` or `key.deserializer` setting with one of these:

- `org.apache.kafka.common.serialization.ByteArraySerializer` or `ByteArrayDeserializer`
- `org.apache.kafka.common.serialization.BytesSerializer` or `BytesDeserializer`
- `org.apache.kafka.common.serialization.DoubleSerializer` or `DoubleDeserializer`
- `org.apache.kafka.common.serialization.FloatSerializer` or `FloatDeserializer`
- `org.apache.kafka.common.serialization.IntegerSerializer` or `IntegerDeserializer`
- `org.apache.kafka.common.serialization.LongSerializer` or `LongDeserializer`
- `org.apache.kafka.common.serialization.ShortSerializer` or `ShortDeserializer`
- `org.apache.kafka.common.serialization.StringSerializer` or `StringDeserializer`

Alternatively, you can implement and provide your own de/serializer.

14.5 Performance

Kafka commits its messages received. For faster read access to a Kafka queue, you can use the Kafka-Settings `enable.auto.commit` and `auto.commit.interval.ms` to combine a larger number of objects in one transaction, thus reducing transaction overhead.

Example:

```
<kafka-importer id='importer' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json'
enable.auto.commit='true' auto.commit.interval.ms='5000' />
```

For a better match with Benerator's `pageSize` approach, you can use this in a Kafka-Importer too, e.g. for committing messages in groups of 10,000:

```
<kafka-importer id='importer' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json'
page.size='10000' />
```

14.6 Encryption and Authentication

Ask your Kafka administrator on details how to configure secure access. All settings you might need are available in the XML format, your configuration could look like this:

```
<kafka-importer id='importer' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json'
security.protocol='SSL'
ssl.truststore.location='/var/private/ssl/kafka.client.truststore.jks'
ssl.truststore.password='test1234'
ssl.keystore.location='/var/private/ssl/kafka.client.keystore.jks'
ssl.keystore.password='test1234'
ssl.key.password='test1234'
/>
```

If the Benerator file is shared, but your access individual, then remember that you can hide your password from the XML setup file by using the properties file mechanism in your user folder, see '[<setting>](#) and [Benerator identifiers](#)':

Local file `kafka.properties`:

```
truststore_password=test1234
keystore_password=test1234
ssl_password=test1234
```

Shared file `benerator.xml`:

```
<include uri="kafka.properties"/>
<kafka-importer id='importer' bootstrap.servers='localhost:9094' topic='kafka-demo' format='json'
security.protocol='SSL'
ssl.truststore.location='/var/private/ssl/kafka.client.truststore.jks'
ssl.truststore.password='{truststore_password}'
ssl.keystore.location='/var/private/ssl/kafka.client.keystore.jks'
ssl.keystore.password='{keystore_password}'
ssl.key.password='{ssl_password}'
/>
```

15. Environment Files

An environment file may define an arbitrary number of systems to connect from Benerator via a short reference. Currently, relational databases and Kafka clusters and topics are supported. An environment file has the suffix `.env.properties`. As an example, you may define the development environment as `dev.env.properties`.

15.1 System Definition

The settings for each system are listed in the environment file in the form:

```
<name>.<type>.<setting>=<value>
```

- `<name>` must be an identifier,
- `<type>` must be `db` or `kafka`,
- `<setting>` must be one of the settings supported by this system type. The name may contain `'.'` characters.

Example:

```
crm.db.url=jdbc:h2:mem:benerator
crm.db.driver=org.h2.Driver
```

15.2 Additional Settings

Additional individual settings that depend on the environment may be declared. Such settings must not have a `'.'` character in their name.

Example:

```
id_strategy=hilo
```

15.3 Comments

Lines starting with a hash `#` are considered comments and should be used to structure and explain the file's content:

```
# this is a comment line
```

15.4 Full Environment File Example

Complete environment file example: `dev.env.properties`

```
# In the dev environment, the CRM system is located in a H2 databas
crm.db.url=jdbc:h2:mem:benerator
crm.db.driver=org.h2.Driver
crm.db.schema=PUBLIC
crm.db.user=sa
crm.db.read.only=true

# The procurement is a Kafka cluster on x.de
procurement.kafka.bootstrap.servers=x.de:9092
procurement.kafka.topic=orders
procurement.kafka.format=json

# Use hilo as default id generation strategy
id_strategy=hilo
```

15.5 Placing and Locating Environment Files

You can put the environment file in different places. If it should be shared with other users of the same project, it is recommended to put it in a folder named 'conf' inside the project folder (example location `myproject/conf/dev.env.properties`).

For private system and password configurations, you can put an environment file into the folder `rapiddweller` under your user home directory (example location `~/rapiddweller/dev.env.properties` on a macOS/Unix system or `C:\Users\<your_user_name>\rapiddweller/dev.env.properties` on a Windows system).

Beware: For referencing an environment, have it in the proper folder and use just the environment name to reference it (not a path!)

15.6 Accessing a System in an Environment

In order to access the system, you need to reference it in the Benerator setup. The `crm` database above would be referenced by:

```
<database id="custs" environment="dev" system="crm"/>
```

or

```
<kafka-importer id="orders" environment="dev" system="procurement"/>
```

15.7 Planning for different environments

You will likely set up your data generation or anonymization task on a development environment, fine tune and test it and finally roll it out on a performance test, showcase or functional testing system. Consider these different stages of the Benerator project.

If you are planning and naming your environment definitions wisely, it is a trivial configuration change to switch from one stage to another:

1. Choose abstract names that describe the *roles* of the systems, *not* the names, eg. `crm`, `order`, `procurement` etc.
2. Define an environment file for each stage, eg. `dev`, `functest`, `perftest` etc.
3. In each environment files, assign each role identifier the physical system of that stage.
4. Create a Benerator setup in which `stage` occurs as a variable, eg. as setting, environment property or setting in an included configuration properties file
5. When declaring a system, use a script expression `{stage}` to reference the stage of a system

Now switching the stage means just switching the `stage` flag:

```
<setting name="stage" value="dev"/>
...
<database id="custs" environment="{stage}" system="crm"/>
...
<kafka-exporter id="orders" environment="{stage}" system="procurement"/>
```

15.8 Backwards Compatibility

The previous database-only environment format (before Benerator 2.1.0) is still supported, but will be abandoned in a future release.

16. Advanced Topics

16.1 JavaBeans and the Benerator Context

You can instantiate JavaBeans by an intuitive syntax like this:

```
<bean id="helper" class="com.my.Helper">
  <property name="min" value="5"/>
  <property name="max" value="23"/>
</bean>
```

The *class* attribute denotes which JavaBean class to instantiate (by the default constructor). The enclosed property tags cause the JavaBean's properties and attributes to be set to appropriate values. Benerator converts common types automatically. If Benerator cannot perform conversion of a custom type, you can define a custom ConverterManager setup (see `rd-lib-common`). Date and time formatting is supported according to ISO 8601 Conventions.

Objects are made available by exposing them in a context. The *id* attribute defines the name with which an object can be found, e.g. for a 'source' or 'ref' attribute of another element's setup.

So the example above creates an instance of a DBSystem JavaBean class, setting its properties to values for connecting a database. The object is retrievable by the context with the *id* 'db'.



Note

The class DBSystem implements the interface 'System' which provides (among other features) meta information about the entities (tables) contained in the database.

You can create references to other objects declared before by a 'ref'-attribute in the bean declaration. The following example shows this for a task setup, but this can be applied to beans and consumers as well.



Note

You may implement the System interface for connecting to other system types like SAP or Siebel systems.

JavaBeans may refer each other (see `proxy.target`) and may have collection or attribute properties (see `log_csv.components`) as shown in the following example:

```
<bean id="csv" class="CSVEntityExporter">
  <property name="uri" value="target/customers.csv"/>
  <property name="columns" value="salutation,first_name,last_name"/>
</bean>

<bean id="proxy" class="shop.MyProxy">
  <property name="target" ref="csv"/>
</bean>

<bean id="log_csv" class="ConsumerChain">
  <property name="components">
    <bean class="LoggingConsumer"/>
    <idref bean="proxy"/>
  </property>
</bean>
```

You can use all inline construction styles in a spec attribute, too.

You can invoke methods on beans using `rapiddwellerScript`:

```
<execute>myBean.init(47 + 11)</execute>
```

16.2 Importing Java classes

Benerator provides an import facility similar to the Java language. You can import classes, packages and domains.

So, instead of using the fully qualified name:

```
<bean id="special" class="com.my.SpecialGenerator" />
```

you can import the class and use the local name:

```
<import class="com.my.SpecialGenerator"/>
<bean id="special" class="com.my.SpecialGenerator"/>
```

The following alternative works as well and imports all classes of the com.my package:

```
<import class="com.my.*"/>
<bean id="special" class="com.my.SpecialGenerator"/>
```

16.2.1 Step-by-step instruction: Importing Custom Java classes (JAR)

Precondition: Verified local benerator installation (cp. [Installation](#)).

In this example, two jars from maven central are used for demo purposes:

1. Download following jars locally
2. <https://repo1.maven.org/maven2/org/apache/commons/commons-text/1.9/commons-text-1.9.jar>
3. <https://repo1.maven.org/maven2/org/apache/commons/commons-lang3/3.12.0/commons-lang3-3.12.0.jar>
4. Move the jars into the `lib\` folder of your benerator installation e.g. (e.g. `C:\Program Files\Development\rapiddweller-benerator-ce\lib\`). Avoid creating any subfolder(s) in the `lib\` directory and place the jar file(s) in the lib folders root level.
5. Given the custom libraries are provided in your ben lib folder, or you added them to the classpath of your Java Virtual Machine (JVM), the following benerator script shows you the available instantiation options:

```
<setup>

<!-- Use direct reference (OPTION A) -->
<generate type="optionA" consumer="ConsoleExporter" count="5">
  <attribute name="column"
    type="string"
    script="org.apache.commons.text.WordUtils.swapCase('A: benerator is a GREAT universal data tool')"/>
</generate>

<!-- Import multiple classes from custom jar (OPTION B) -->
<import class="org.apache.commons.text.WordUtils"/>

<!-- Import multiple classes from custom jar with wildcard (OPTION C) -->
<import class="org.apache.commons.text.*"/>

<!-- Use OPTION B/C in BENERATION -->
<generate type="optionBC" consumer="ConsoleExporter" count="5">
  <attribute name="column" type="string" script="WordUtils.swapCase('B/C: benerator is a GREAT universal data tool')"/>
</generate>

<!-- Instantiate single class as bean (OPTION D) -->
<bean id="customUtil" class="org.apache.commons.text.WordUtils">
  <!-- Allows to define properties for your constructor as below if required -->
  <!-- <property name="myProperty" value="12345"/> -->
</bean>

<!-- Use OPTION D in BENERATION -->
<generate consumer="ConsoleExporter" count="5" type="optionB">
  <attribute name="column" type="string" script="customUtil.swapCase('D: benerator is a GREAT universal data tool')"/>
</generate>

</setup>
```

16.2.2 Domain imports

Domains can be imported as well. For the built-in Benerator domains, only the domain name is necessary, for custom domains, the fully qualified name of the domain's top-level package. For a built-in domain:

```
<import domains="person"/>

<bean id="personGen" class="PersonGenerator"/>
```

For a custom domain:

```
<import domain="com.my"/>

<bean id="myGen" class="MyGenerator"/>
```

16.3 Looking up services/objects via JNDI

Benerator provides an `InitialContext` class in the JNDI platform package. It can be used to locate objects and make them available in Benerator:

```
<import platforms="jndi"/>

<bean id="ctx" class="InitialContext">
  <property name="factory" value="..."/>
  <property name="url" value="..."/>
  <property name="user" value="..."/>
  <property name="password" value="..."/>
</bean>

<bean id="ejb" spec="ctx.lookup('cons')"/>
```

16.4 Calculating global sums

Sometimes you need to calculate the total sum of one field over all generated instances, e.g. for calculating checksums. The `AddingConsumer` is there to help you with this task. Instantiate it as `<bean>`, specifying the field name to add and the number type to use, then use it as consumer and finally query its 'sum' property value with a script expression:

```
<bean id="adder" spec="new AddingConsumer('_txn_amount_', 'long')"/>

<generate type="deb_transactions" count="100" consumer="ConsoleExporter, adder">
  <attribute name="_txn_amount_" type="long" min="1" max="100" distribution="random"/>
</generate>

<generate type="trailer_record" count="1" consumer="ConsoleExporter">
  <attribute name="total" script="adder.sum"/>
</generate>
```

16.5 Querying data from a system to variables

Querying is discussed in [Data Generation Concepts](#).

Benerator supports to store multiple values in [Variables](#) at runtime that can be accessed array-like to use the data for multiple attributes i.e. fulfill multi-field constraints.

Given a variable named `productInfo` and a selector that queries 3 values (e.g. `select ean_code,product_name,product_no from ...`) from a product database

the values are accessed by `productInfo[0]`, `productInfo[1]`, `productInfo[2]` using the `script` element.

```
<generate type="shop" count="10">
  <attribute name="country" values="DE,AT,CH"/>
  <generate type="product" count="100" consumer="db">

    <variable name="productInfo"
      source="db"
      selector="{ftl: select ean_code,product_name,product_no from db_product where country='${shop.country}'}"
      cyclic="true"
    />

    <attribute name="ean_code"
```

```

        script="productInfo[0]"/>
    <attribute name="product_name"
        script="productInfo[1]"/>
    <attribute name="product_no"
        script="productInfo[2]"/>
</generate>
</generate>

```

Warning

Set **cyclic="true"** if not enough products for the given country value are available in your dataset.

16.6 The MemStore

The MemStore is a simple implementation of the StorageSystem interface which allows you to store entity data in RAM and perform simple queries on them. This is useful in cases where you need to generate data where there are internal dependencies but no database is involved, e.g. generating an XML file with products and orders.

A MemStore is declared with a `<memstore>` element:

```
<memstore id="store"/>
```

You can use it as a consumer for storing data (in this case products):

```

<generate type="product" count="10" consumer="store,ConsoleExporter">
  <id name="id" type="int"/>
  <attribute name="name" pattern="[A-Z][a-z]{4,12}"/>
</generate>

```

Afterwards you can query the generated products for referencing them in generated orders:

```

<generate type="order" count="10" consumer="ConsoleExporter">
  <variable name="product" source="store" type="product"/>
  <id name="id" type="int"/>
  <attribute name="product_id" script="product.id"/>
</generate>

```

Warning

Note that you can **only** query for entities – if you need a single attribute of an entity, you must first use a variable to get the entity and afterwards retrieve the required attribute by script.

You can use a distribution:

```
<variable name="product" source="store" type="product" distribution="random"/>
```

A simple form of query is supported by a `selector` element. Its content must be a script expression that serves as a filter. The expression is consecutively evaluated on each element (as candidate) and has to return `true` if the candidate is accepted, otherwise `false`.

Danger

IMPORTANT: The script can only access each candidate by using the keyword `_candidate`.

As an example, here is a query which only returns products whose name starts with 'A':

```
<variable name="product" source="store" type="product" selector="_candidate.name.startsWith('A')"/>
```

16.7 Datasets

You can define datasets and combine them with supersets. This mechanism lets you also define parallel and overlapping hierarchies of nested datasets.

Definition of hierarchies is separated from the definition of dataset values for a concrete topic. So you can define a dataset grouping for regions, mapping continents, countries, states and departments and apply this grouping to define and combine sets of e.g. cities, person names or products.

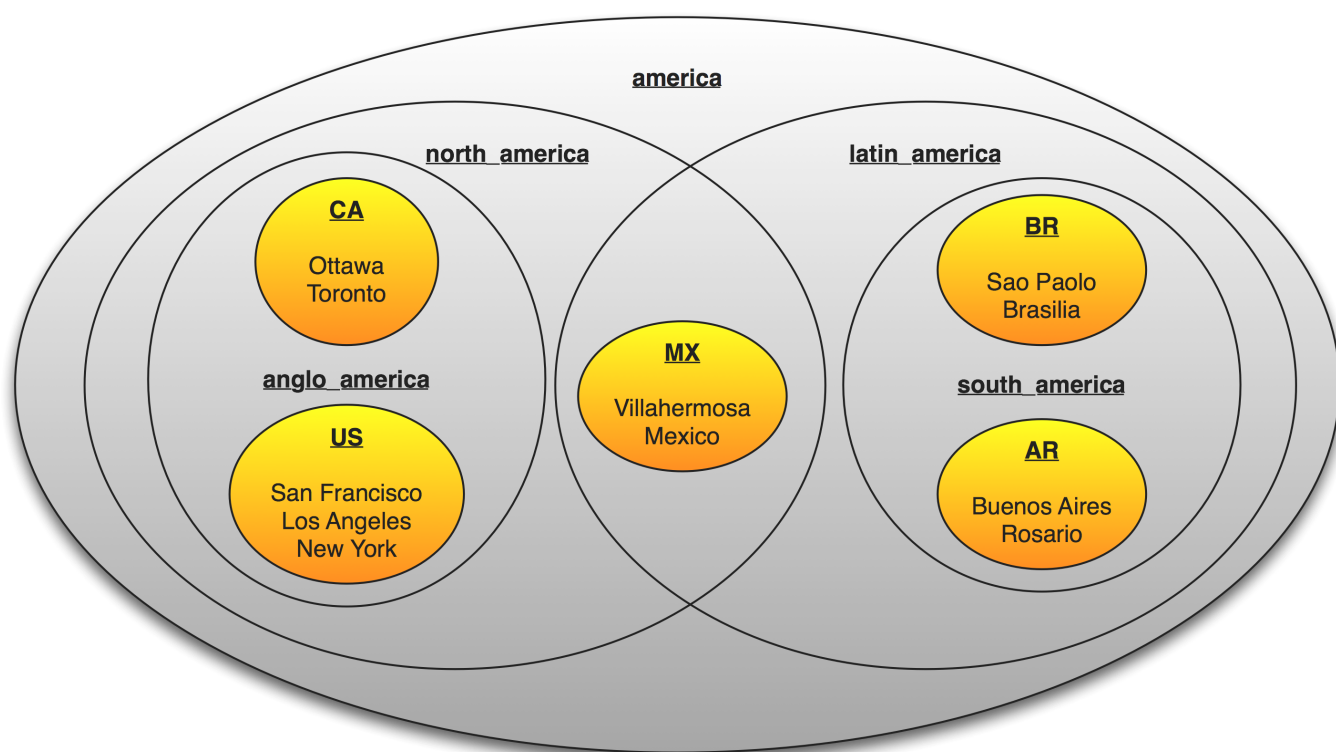
We will apply the mechanism here for cities in geographical regions. You can find the example files in the distribution's directory `demo/dataset/`

A dataset is identified by a code. For a country, its ISO code is an appropriate choice, but you are free to define and choose what is useful for your application.

Assume you wanted to process some American countries: US (USA), CA (Canada), MX (Mexico), BR (Brazil), AR (Argentina)

You could group them geographically (North America vs. South America) or by language (Latin America vs. Anglo America). You could do both in parallel by defining area sets in a file `area.set.properties` :

```
latin_america=MX,BR,AR
anglo_america=US,CA
north_america=US,CA,MX
south_america=BR,AR
america=north_america,south_america
```



The simple datasets would be defined in CSV files:

`cities_US.csv` :

```
San Francisco
Los Angeles
New York
```

`cities_CA.csv` :

```
Ottawa
Toronto

cities_MX.csv

Mexico
Villahermosa

cities_BR.csv

Sao P  lo

Brasilia

cities_AR.csv

Buenos Aires

Rosario
```

You can now use this setup to generate city names for any of the specified regions. For North American cities you could specify

```
<echo message="north american cities:"/>

<generate type="city" consumer="exporter" count="10">
  <attribute name="name" unique="true"
    source="city_{0}.csv" encoding="UTF-8"
    dataset="north_america" nesting="area"/>
</generate>
```

and generate the output:

north american cities:

```
city[name=Mexico]
city[name=Los Angeles]
city[name=San Francisco]
city[name=New York]
city[name=Villahermosa]
city[name=Ottawa]
city[name=Toronto]
```

16.7.1 Region nesting

Benenerator comes with a predefined nesting definition for geographical datasets, called 'region':

```
europe=western_europe,central_europe,
southern_europe,eastern_europe,northern_europe=
western_europe=PT,ES,AD,FR,MC
central_europe=BE,NL,LU,DE,CH,AT,LI
southern_europe=IT,SM,GR,CY,TR
eastern_europe=AL,SI,CZ,HU,PL,RU,RO,BG,HR,BA,EE,LT,LV,SK,UA
northern_europe=UK,IE,DK,SE,NO,FI,IS
near_east=AF,IR,IL,JO,KZ,PK,QA,SA,AE
africa=EG,GH,KE,ZA
north_america=US,CA
central_america=MX,BS
america=north_america,central_america,south_america
south_america=AR,BR,EC
asia=JP,IN,ID,KR,KP,MY,SG,TW,TH
australia=AU,NZ
```

16.8 Chaining generators

Generators can be chained, composed, or reused in different contexts. You can do so by instantiating a generator as JavaBean and referring it in properties of other JavaBean-instantiated generators or specifying it as `source` attribute like an importer.

```
<!-- creates a text generator -->
<bean id="textGen" class="RegexStringGenerator">
  <property name="pattern" value="([a-z]{3,8}[ ])*[a-z]{3,8}\."/>
</bean>

<!-- wraps the text generator and creates messages -->
<generate type="message" count="10" consumer="LoggingConsumer">
  <attribute name="text" source="textGen" converter="MessageConverter" pattern="Message: '{0}'"/>
</generate>
```

16.9 Invoking Benerator programmatically

For integrating Benerator with other applications, you can invoke it programmatically in a Java application too.

16.9.1 Invoking Benerator as a whole

For executing descriptor files do the following:

```
// create an instance of the Descriptor runner specifying the descriptor file
DescriptorRunner runner = new DescriptorRunner("path/to/file/benerator.xml");
BeneratorContext context = runner.getContext();
// use the BeneratorContext to set locale, file encoding, ... as you need
context.setValidate(false);
runner.run();
```

16.9.2 Making Benerator generate data and invoke a custom class with it

If you want to use Benerator for feeding a custom class with generated data, implement the Consumer interface in a way that connects to your program, instantiate it and register it with the BeneratorContext:

```
DescriptorRunner runner = new DescriptorRunner("path/to/file/benerator.xml");
BeneratorContext context = runner.getContext();
context.setValidate(false);
MyConsumer myConsumer = new MyConsumer();
context.set("myConsumer", myConsumer); // add a custom Consumer
runner.run();
```

A simplistic implementation could simply write entities to the console, e.g.

```
class MyConsumer extends AbstractConsumer<Entity> {
    List<Entity> products = new ArrayList<Entity>();
    public void startConsuming(Entity entity) {
        products.add(entity);
    }
}
```

16.9.3 Using generators defined in a descriptor file

You can define data generation in a Benerator descriptor file, make Benerator configure the generator and hand it out to your application.

First, define a descriptor file, e.g.

```
<?xml version="1.0" encoding="utf-8"?>
<setup>

    <generate type="address">
        <attribute name="city" values="'Munich', 'New York', 'Tokyo'"/>
    </generate>

</setup>
```

Then you can get the address generator from Benerator by calling:

```
BeneratorContext context = new BeneratorContext();
Generator<?> generator = new DescriptorBasedGenerator("benerator.xml", "address", context);
generator.init(context);
ProductWrapper wrapper = new ProductWrapper();
for (int i = 0; i < 10; i++)
    System.out.println(generator.generate(wrapper));
generator.close();
```

16.10 Tasks

In many cases, data generation based on the descriptor file format may be too complex and a script based generation too slow.

In these cases you can provide a custom class that implements the `com.rapiddweller.task.Task` interface and can be invoked from the Benerator descriptor file. For example, you could write a Task implementation that calls business logic for triggering complex operations on business objects.

Such a class can be instantiated and invoked with a similar syntax like any JavaBean, e.g.:

```
<run-task class="com.my.SpecialTask">
  <property name="uri" value="base.csv"/>
  <property name="db" ref="db"/>
</run-task>
```

You can instantiate and initialize a task like a `<bean>` (see '[JavaBeans and the Benerator Context](#)'). Tasks can be executed in pages, ensuring that the total number of execution over all threads is the page size. For this, The element `run-task` also supports the attributes

- `count` : the total number of times the Task is executed (defaults to 1)
- `pageSize` : the number of invocations to execute together (defaults to 1), e.g. for grouping `pageSize` invocations to a single transaction
- `pager`: injects a class (implementing the `PageListener` interface) to perform custom paging operations (like flushing data to file or database)

```
<run-task class="my.SpecialTask" count="1000" pageSize="100" pager="my.PagingStrategy">
  <property name="message" value="I'm special"/>
</run-task>
```

16.11 Staging

Combining scripting and property files, you get a staging mechanism. A typical example would be to use different setups for developing data generation on a local system and using it to produce mass data on a test environment. The basic approach is to extract variable properties to properties files, e.g. `development.properties`:

```
user_count=100
db_batch=false
pageSize=1
db_uri=jdbc:oracle:thin:@10.37.129.4:1521:XE
db_driver=oracle.jdbc.driver.OracleDriver
db_user={user.name}
db_password={user.name}
db_schema={user.name}
```

and `perftest.properties`:

```
user_count=1000000
db_batch=true
pageSize=1000
db_uri=jdbc:oracle:thin:@134.53.26.183:1521:MYAPP
db_driver=oracle.jdbc.driver.OracleDriver
db_user=myapp
db_password=myapp
db_schema=myapp
```

You can then decide which configuration to use by setting a stage setting as Java virtual machine parameter, e.g. write

```
<mvn benerator:generate -Dstage=development/>
```

16.12 Template data structures

You can use DbUnit import files for replicating entity graph structures many times on each generated object. Say, for each customer in a tested online shop, a default order structure should be created. You would then define the order structure in a DbUnit file.

```
<dataset>
  <db_order_item order_id="{db_order.id}" number_of_items="2" product_ean_code="8076800195057" total_price="2.40"/>
  <db_order_item order_id="{db_order.id}" number_of_items="1" product_ean_code="8006550301040" total_price="8.70"/>
</dataset>
```

and then create an order for each customer that imports its sub structure from the DbUnit file:

```
<generate type="db_order" consumer="db">
  <id name="id"/>
  <reference name="customer_id"/>
```

```

<iterate type="db_order_item" source="demo/shop/default_order.dbunit.xml" consumer="db">
  <id name="id"/>
  <reference name="order_fk" script="db_order.id"/>
</iterate>
</generate>

```

Of course, you have to care for appropriate ids yourself.

16.13 Generating arrays

Instead of entity data, you can generate arrays using `type="array"` for declaring the generated type and `<value>` elements for configuring the generated components:

```

<generate type="array" count="5" consumer="ConsoleExporter">
  <value type="string" pattern="[A-Z]{5}"/>
  <value type="int" min="1" max="42"/>
</generate>

```

This is useful for special consumers that require array types, e.g. for generating parameters for the `JavaInvoker` which calls Java methods.

16.14 Scoped Generation

All nested generator components have a scope which influences its life cycle: When generation in a scope is finished, its sub components are reset before the next invocation.

16.14.1 Default (implicit) scope

As an example, assume you are generating data for a shop application which takes orders composed of order items. Each order and order item must have its unique id, but the `order_item` should have a sequence number which represents its position number within its associated order:

```

<generate name="the_order" type="order" count="2" consumer="ConsoleExporter">
  <id name="id" type="int" generator="IncrementalIdGenerator"/>

  <generate name="the_item" type="order_item" minCount="1" maxCount="3" consumer="ConsoleExporter">
    <id name="id" type="int" generator="IncrementalIdGenerator"/>
    <reference name="order_id" script="the_order.id"/>
    <attribute name="item_no" type="int" distribution="increment"/>
    <attribute name="product" type="string" pattern="[A-Z][a-z]{3,8}"/>
    <attribute name="count" type="int" min="1" max="3"/>
  </generate>
</generate>

```

When running this descriptor, the output is (after indenting and formatting):

```

order[id=1]
order_item[id=1, order_id=1, item_no=1, product=Pgbxll, count=2]
order_item[id=2, order_id=1, item_no=2, product=Pmce, count=1]order[id=2]
order_item[id=3, order_id=2, item_no=1, product=Entlkzkjv, count=2]
order_item[id=4, order_id=2, item_no=2, product=Jgqp, count=3]

```

This exhibits the behaviour with default (implicit) scope, which is tied to the owner of a component. In this case, the scope of the `item_no` component is the `order_item` entity: When one generation loop of „the_item“ is through, the contained component generators are reset before running the next loop (belonging to a new instance of 'the_order').

16.14.2 Explicit scope

To understand explicit scoping, let us first extend the previous example, assuming that each single product instance ordered is shipped in an own package, which shall be tracked. Thus, you would add a sub `<generate>` for the packages:

```

<generate name="the_order" type="order" count="2" consumer="ConsoleExporter">
  <id name="id" type="int" generator="IncrementalIdGenerator"/>

  <generate name="the_item" type="order_item" minCount="1" maxCount="3" consumer="ConsoleExporter">
    <id name="id" type="int" generator="IncrementalIdGenerator"/>
    <reference name="order_id" script="the_order.id"/>

```

```

<attribute name="item_no" type="int" distribution="increment"/>
<attribute name="product" type="string" pattern="[A-Z][a-z]{3,8}"/>
<attribute name="count" type="int" min="1" max="3"/>

<generate type="package" count="{the_item.count}" consumer="ConsoleExporter">
  <id name="id" type="int" generator="IncrementalIdGenerator"/>
  <reference name="order_id" script="the_order.id"/>
  <attribute name="order_pkg_no" type="int" distribution="increment"/>
</generate>

</generate>

</generate>

```

Running this yields the output:

```

order[id=1]
order_item[id=1, order_id=1, item_no=1, product=Pgbxll, count=2]
package[id=1, order_id=1, order_pkg_no=1]
package[id=2, order_id=1, order_pkg_no=2]
order_item[id=2, order_id=1, item_no=2, product=Pmce, count=1]
package[id=3, order_id=1, order_pkg_no=1]
order[id=2]
order_item[id=3, order_id=2, item_no=1, product=Entlkzkjv, count=2]
package[id=4, order_id=2, order_pkg_no=1]
order_item[id=4, order_id=2, item_no=2, product=Jgqp, count=3]
package[id=5, order_id=2, order_pkg_no=1]
package[id=6, order_id=2, order_pkg_no=2]

```

OK, we see that this adds `<count>` packages to each `order_item` and the `order_pkg_no` numbering begins with 1 for each `order_item`..

Now suppose that the package number should be a sequence which is unique for a complete order, not just an `order_item` - this is where the package number generation needs a different scope, the one of the order, telling Benerator to apply the same lifecycle to the packages as to the components of the order:

```

<generate name="the_order" type="order" count="2" consumer="ConsoleExporter">
  <id name="id" type="int" generator="IncrementalIdGenerator"/>
  <generate name="the_item" type="order_item" minCount="1" maxCount="3" consumer="ConsoleExporter">
    <id name="id" type="int" generator="IncrementalIdGenerator"/>
    <reference name="order_id" script="the_order.id"/>
    <attribute name="item_no" type="int" distribution="increment"/>
    <attribute name="product" type="string" pattern="[A-Z][a-z]{3,8}"/>
    <attribute name="count" type="int" min="1" max="3"/>
    <generate type="package" count="{the_item.count}" consumer="ConsoleExporter">
      <id name="id" type="int" generator="IncrementalIdGenerator"/>
      <reference name="order_id" script="the_order.id"/>
      <attribute name="order_pkg_no" type="int" distribution="increment" scope="the_order"/>
    </generate>
  </generate>
</generate>

```

yielding the desired result:

```

order[id=1]
order_item[id=1, order_id=1, item_no=1, product=Pgbxll, count=2]
package[id=1, order_id=1, order_pkg_no=1]
package[id=2, order_id=1, order_pkg_no=2]
order_item[id=2, order_id=1, item_no=2, product=Pmce, count=1]
package[id=3, order_id=1, order_pkg_no=3]

order[id=2]
order_item[id=3, order_id=2, item_no=1, product=Entlkzkjv, count=2]
package[id=4, order_id=2, order_pkg_no=1]
order_item[id=4, order_id=2, item_no=2, product=Jgqp, count=3]
package[id=5, order_id=2, order_pkg_no=2]
package[id=6, order_id=2, order_pkg_no=3]

```

16.14.3 Global scope

Using `scope="/"`, any component can be configured to have global scope which means that it is never reset. This can also be used as a performance improvement for resource-heavy database queries combined with a caching distribution.

16.15 Composite Data Generation

For databases, flat data generation is used: Each generated entity stands for itself and is persisted without a context. Nesting `<generate>` elements only correspond to a concept of loop and sub-loop, not to a hierarchical nesting of data. First, the top-level

element is created and sent to its consumer(s) then its child elements are generated and sent to their consumer(s). It is up to the consumer(s) to create and track context and interpret the generated data in a hierarchical manner.

In cases of intrinsically hierarchical data (mongoDB or XML), a root data entity needs to be generated in a composite manner and processed differently: All nested elements are generated until the full tree is built and then the root element is sent to the consumer (and only this element). The consumer can then scan through the entity tree and do its job.

This is what the `<part>` element has been introduced for. It takes the same parameters as the `<generate>` element, but does not call a consumer with the generated data, but puts them into a collection that is set as the parent entity's property.

For example,

```
<generate type='user' minCount='100' consumer='db'>
  <attribute name='name' type='string' />
  <attribute name='age' type='int' min='18' max='78' />

  <part name='addresses' container='list' count='2'>
    <attribute name='street' pattern='[A-Z][a-z]{4} Street' />
    <attribute name='houseNo' type='int' min='2' max='9' />
  </part>
</generate>
```

generates 100 user entities, of which each one has an 'address' component, which is a list of 2 address entities.

Supported settings for the part's container attribute are:

- list
- set
- array

The default is `list`.

16.16 Composite Data Iteration

Using a data source which provides hierarchical data, nested collection components (like `user.addresses` in the previous example) can be iterated explicitly, specifying the root component as **source** and providing the component **name**.

17. Generating Unique Data

17.1 ID Generation

For generating unique data for dataset identifiers like primary keys in a database, see [Common ID Generators](#) for a complete ID generator reference and [Using Relational Databases](#) for database-related id generators.

17.2 Unique Number Generation

Most → Sequences are able to generate unique numbers. Just apply a `unique="true"` to the number configuration:

```
<attribute name="n" type="int" min="3" max="99" unique="true" />
```

17.3 Unique String Generation

17.3.1 Uniqueness with Regular Expression

There are ID generators that generate UUID strings, but in most cases, you have constraints on string length and character select which require you to have something more individual and configurable.

One of the best general approaches is to use Benerator's feature to generate unique strings that match a regular expression. For example, for generating unique phone numbers, you could write:

```
<attribute name="phone" type="string" pattern="[1-9][0-9]{2}\-[0-9]{4}\-[0-9]{5}" unique="true" />
```

For an introduction to regular expressions, read about [Regular Expressions](#).

17.3.2 Making Strings unique

Sometimes you have less strict constraints on the strings you want to make unique. A good example is a common derivation of user names from their real names which takes the first letter of the first name and appends the last name.

This might lead to non-unique results, since John Smith and Joshua Smith would get the same user name jsmith.

The usual solution is to append a number to make the string for the second jsmith unique again: jsmith2.

This is exactly, what the `UniqueStringConverter` does:

```
<generate type="user" count="10" consumer="ConsoleExporter">
  <variable name="person" generator="PersonGenerator"/>
  <attribute name="user_name" script="person.givenName.substring(0, 1) + person.lastName"
    converter="ToLowerCaseConverter, UniqueStringConverter"/>
</generate>
```



Warning

The `UniqueStringConverter` keeps all used strings in memory, so when generating some billion strings, you might get memory problems.

17.4 Removing Duplicate Values

If you need a more individual generation algorithm of which you do not know (or care) how to make it unique, you can append a `UniqueValidator` to filter out duplicate values.

```
<attribute name="code" pattern="[A-Z]{6,12}" validator="UniqueValidator"/>
```



The `UniqueValidator` keeps all used strings in memory, so when generating some billion strings, you might get memory problems.

17.5 Unique iteration through a source

When iterating data from a data source and requiring uniqueness, you need to assure for yourself, that the source data is unique:

```
<attribute name="code" type="string" source="codes.csv" />
```

When applying a distribution to the iterated data, configure `unique="true"` for assuring that the distribution does not repeat itself:

```
<attribute name="code" type="string" source="codes.csv" distribution="random" unique="true" />
```

17.6 Unique Generation of Composite Keys

As an example, let's have a look the following code:

```
<generate type="product" count="6" consumer="ConsoleExporter">
  <attribute name="key1" type="int" />
  <attribute name="key2" type="int" />
</generate>
```

If we need to generate unique combinations of `key1` and `key2` we have different alternatives:

17.6.1 Simplistic Approach

If each value is unique, the combination of them is unique too. The following setting:

```
<generate type="product" count="6" consumer="ConsoleExporter">
  <attribute name="key1" type="int" distribution="increment" unique="true" />
  <attribute name="key2" type="int" distribution="increment" unique="true" />
</generate>
```

The generated values are:

```
product[key1=1, key2=1]
product[key1=2, key2=2]
product[key1=3, key2=3]
product[key1=4, key2=4]
product[key1=5, key2=5]
product[key1=6, key2=6]
```

17.6.2 Cartesian Product

For generating unique composite keys the most convenient way is to create a cartesian product of unique components by nesting two creation loops.

For making the combination of `key1` and `key2` in the following descriptor unique:

one would add an outer 'dummy' loop and create helper variables `x` and `y` in a way that they can be combined like in a cartesian product:

```
<generate type="dummy" count="2">
  <!-- no consumer! -->
  <variable name="x" type="int" distribution="increment" unique="true" />
  <generate type="product" count="3" consumer="ConsoleExporter">
    <variable name="y" type="int" distribution="increment" unique="true" />
    <attribute name="key1" type="int" script="x"/>
    <attribute name="key2" type="int" script="y"/>
  </generate>
</generate>
```

The generated values are:

```
product[key1=1, key2=1]
product[key1=1, key2=2]
product[key1=1, key2=3]
product[key1=2, key2=1]
product[key1=2, key2=2]
product[key1=2, key2=3]
```

17.6.3 Prototype Approach

You can use the prototype approach for getting unique composite keys: A variable's generation algorithm needs to assure uniqueness of the combination:

```
<generate type="product" count="6" consumer="ConsoleExporter">
  <variable name="p" generator="my.HelperClass"/>
  <attribute name="key1" type="int" script="p.x" />
  <attribute name="key2" type="int" script="p.y" />
</generate>
```

The most frequent application of this approach is the generation of unique database references using a [Prototype Query](#).

17.7 Achieving local uniqueness

Sometimes values need to have uniqueness of an identity component of a 'child' entity only in the context of another ('parent') item.

One simple solution is of course to have it globally unique.

If there are more constraints involved, you can of course use an appropriate generator but need to nest the generation of parent and child:

```
<generate type="parent" count="5" consumer="ConsoleExporter">

  <generate type="product" count="5" consumer="ConsoleExporter">
    <variable name="y" type="int" distribution="increment" unique="true" />
    <attribute name="key1" type="int" script="x"/>
    <attribute name="key2" type="int" script="y"/>
  </generate>

</generate>
```

18. Scripting

Benerator supports arbitrary scripting languages that are supported by Java and has its own scripting language **rapiddwellerScript** which is designed specifically for the purpose of data generation.

The invocation syntax is as described for SQL invocation and inlining: You can include the script inline like this:

```
<setup>
  <database id="db" url="jdbc:hsqldb:mem:example" driver="org.hsqldb.jdbcDriver" user="sa" schema="PUBLIC"/>

  <execute type="js">
    print('DB-URL' + db.getUrl());

    // create user Alice
    const Entity = Java.type('com.rapiddweller.model.data.Entity');

    let alice = new Entity('db_user', context.getLocalDescriptorProvider());
    alice.set('id', 1);
    alice.set('name', 'Alice');
    db.store(alice);

    // create user Bob
    let bob = new Entity('db_user', context.getLocalDescriptorProvider());
    alice.set('id', 2);
    alice.set('name', 'Bob');
    db.store(bob);

    // persist everything
    db.flush();
  </execute>
</setup>
```

As you see with the db variable, all objects of the benerator context are made provided to the script. In this case, it is a DBSystem bean, which is used to store Entities created by the script. So, you can import objects of arbitrary Java classes and use them in your favorite scripting language.

Alternatively to inlining script text, you can put it in a script file and invoke this:

```
<execute type="js" uri="test.js"/>
```

You can bind a language of choice by using the mechanisms of *GraalVM*: Scripting for the Java Platform.

With rapiddweller Benerator, GraalVM[js] implementation is shipped. For all other platforms and languages you need to configure language support.

The following attributes are available for the `<execute>` element:

name	description
uri	the URI of the script file to execute
encoding	the encoding of the script file
type	Type (language) of the script
target	a target to execute the script on, typically a database for a SQL script
onError	How to handle errors. One of (ignore , trace , debug , info , warn , error , fatal). fatal causes Benerator to cancel execution in case of an error.
optimize	boolean flag that tells Benerator whether it may optimize script execution for the sake of performance. E.g. For an Oracle SQL script, this would leave out comments for faster table creation.

Benerator supports the following script types:

script	description
shell	system shell invocations, e.g. for invoking batch files.
sql	SQL, it requires specification of the database in a target property.
jar	java library files with a configured main-class
ben	rapiddwellerScript, which is the default script language
ftl	FreeMarker
js	JavaScript is shipped with Benerator GraalVM dependencies
py	is not enabled by DEFAULT - it requires a GraalVM as JVM with Python installed (this is not supported on Windows at the moment)

Example:

```
<setup>

<execute type="js">
  let c = 1;
  const d = 6;

  function add(number1){
    c += 1;
    return number1 + c + d;
  }
</execute>

<iterate type="person" source="person.ent.csv" consumer="ConsoleExporter">
  <variable name="age" type="integer" script="this.age"/>
  <attribute name="multiply" type="int" script="{js:add(age)}/>
</iterate>

<iterate type="person2" source="person.ent.csv" consumer="ConsoleExporter">
  <attribute name="multiply" script="{js:add(person2.age)}/>
  <attribute name="multiply2" script="{js:add(person2.age)}/>
  <attribute name="multiply3" script="{js:add(person2.age)}/>
</iterate>

<generate type="script" count="5" consumer="ConsoleExporter">
  <variable name="count" type="integer" generator="IncrementalIdGenerator"/>
  <attribute name="multiply" script="{js:add(count)}/>
</generate>

</setup>
```



Danger

Make sure not to redefine a variable or method. The defined variables and functions are valid for the whole runtime.

18.1 Shell scripting

You can call shell files or issue shell commands. When in-lining shell commands, script expressions will be resolved. So you could, for example, use global properties for setting parameters of a *sqlplus* call:

```
<execute type="shell">{ftl:sqlplus ${dbUser}/${dbPassword}@${database} @create_tables.sql}</execute>
```

19. rapiddwellerScript

19.1 Motivation

[rd-lib-script](#) former DatabeneScript by Volker Bergmann is a script language designed for simplifying test data generation. Text-targeted template languages like FreeMarker and general languages like JavaScript have specific problems, which can be avoided by a language tailored to the task of data generation.

rapiddwellerScript uses many elements familiar from C-style languages like Java and C#, but adds some specials:

- More convenient object construction
- intuitive date arithmetic
- seamless integration and interaction with Benerator

19.2 Examples

19.2.1 Variable definition

```
<execute>x = 3</execute>
```

19.2.2 Variable assignment

```
<execute>x = x + 1</execute>
```

19.2.3 Object construction

Constructor-based:

```
new MyGenerator('alpha.txt', 23)
```

Properties-based:

```
new MyGenerator { filename='alpha.txt', limit=23 }
```

19.2.4 Date arithmetic

```
new Date() + 5000
```

19.2.5 Java integration

```
(com.rapiddweller.common.SystemInfo.isWindows() ? 'win' : 'other')
```

Benerator integration

```
(context.contains('key') ? 'def' : 'none')
```

19.3 Syntax

19.3.1 Comments

Line comments start with `//` and include the rest of the line.

Normal comments begin with `/*` and end with `*/`

19.3.2 White Space

Spaces, tabs, CR, LF, `\u000C`

19.3.3 Data Types

Signed integral numbers: `long`, `int`, `short`, `byte`, `big_integer`

Signed floating point numbers: `double`, `float`, `big_decimal`

Alpha: `string`, `char`

Date/time: `date`, `timestamp`

Other: `boolean`, *object (Java object)*, *binary (byte[])*

19.3.4 Identifiers

The first character must be an ASCII letter or underscore. An arbitrary number of subsequent characters may be ASCII letters, underscores or numbers.

Legal identifiers: `_test`, `A1234`, `_999`

Illegal identifiers: `1ABC`, `X0`, `FSD`, `alpha.beta`

19.3.5 Escape Sequences

<code>\b</code>	Backspace
<code>\t</code>	Tab
<code>\n</code>	New Line
<code>\r</code>	Carriage Return
<code>\f</code>	Form Feed
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\nnn</code>	Octal encoded character

19.3.6 String Literal

Quoted with single quotes, e.g. `Text`

19.3.7 Decimal Literal

Supporting decimal syntax.

Legal decimal values: `1234.2345E+12`, `1234.2345E12`, `1234.2345e-12`

19.3.8 Integral Number Literal

Supporting decimal, hexadecimal and octal syntax.

Legal decimal values: `0`, `123`

Legal octal values: `01`, `00123`

Legal hexadecimal values: `0x0dFa`

19.3.9 Boolean Literal

Legal values: `true`, `false`

19.3.10 null Literal

Legal value: `null`

19.3.11 Qualified name

`identifier(.identifier)*`

Legal values: `com.rapiddweller.benerator.Generator`, `Generator`

19.3.12 Constructor invocation

Works like in Java, e.g.

```
new MyGenerator('alpha.txt', 23)
```

19.3.13 Bean construction

Instantiates an object by the class' default constructor and calling property setters, e.g.

```
new MyGenerator { filename='alpha.txt', limit=23 }
```

is executed like

```
MyGenerator generator = new MyGenerator();
generator.setFilename("alpha.txt");
generator.setLimit(23);
```

19.3.14 Method invocation

Can occur on static Java methods on a class or instance methods on an object.

Static method invocation: `com.my.SpecialClass.getInstance()`

instance method invocation: `generator.generate()`

19.3.15 Attribute access

Can occur on static Java attributes on a class or instance methods on an object.

Static attribute access: `com.my.SpecialClass.instance`

instance attribute access: `user.name`

19.3.16 Casts

Benerator can casts data types. Cast arguments are Benerator's simple types.

Examples: `(date) '2009-11-23'`, `(long) 2.34`

19.3.17 Unary Operators

Arithmetic Negation: `-`

Bitwise complement: `~`

Boolean Negation: `!`

19.3.18 Arithmetic Operators

Multiplication: `*`

Division: `/`

Modulo division: `%`

Addition: `+`, e.g. `new Date() + 5000`

Subtraction: `-`

19.3.19 Shift Operators

Left shift: `<<` (in XML descriptor files use `<<`)

Right shift: `>>` (in XML descriptor files use `>>`)

Right shift: `>>` (in XML descriptor files use `>>>`)

19.3.20 Relation Operators

`<=`, `<`, `>`, `=>` (in XML descriptor files use `<=`, `<`, `>`, `=>`)

19.3.21 Equality Operators

`==`, `!=`

19.3.22 Bitwise Operators

And: `&` (in XML descriptor files use `&`)

Exclusive Or: `^`

Inclusive Or: `|`

19.3.23 Boolean Operators

And: `&&` (in XML descriptor files use `&&`)

Or: `||`

rapiddwellerScript uses shortcut evaluation like C, C++ and Java: First it evaluates the left hand side of an operation. If the result is completely determined by the result, it does not evaluate the right hand side.

19.3.24 Conditional Expression

`... ? ... : ...`, e.g. `a>3 ? 0 : 1`

19.3.25 Assignment

`qualifiedName = expression`

20. Command Line Tools

20.1 Benerator

Benerator can be called with a Benerator file name as command line parameter, like

```
benerator test.ben.xml
```

If no file is specified, Benerator expects a file `benerator.xml` in the current directory.

You can change default behavior by Java VM parameters, e.g.

```
benerator.bat -Dfile.encoding=iso-8859-1 -Djava.io.tmpdir="C:\temp" test.ben.xml
```

Validation can be turned off from the command line alternatively using a VM parameter:

```
mvn benerator:generate -Dbenerator.validate=false
```

or

```
benerator myproject.ben.xml -Dbenerator.validate=false
```

You can specify the following options on the command line:

Option	Description	Remarks
--version,-v	Display system and version information	
--help,-h	Display help information	
--list	List the available environments or systems. <type> may be <code>env</code> , <code>db</code> or <code>kafka</code> .	
--clearCaches	Clear all caches	
--mode	Activate Benerator mode <code>strict</code> , <code>lenient</code> or <code>turbo</code>	default is <code>lenient</code>
--anonReport	Verify 'pct' percent of anonymized data and display an anonymization report. 'pct' is an integer, 100 for complete tracking	Enterprise Edition only

20.2 DB Snapshot Tool

The DbSnapshotTool creates a snapshot of a full database schema and stores it in a DbUnit XML file. It is invoked from the command line in Windows by calling

```
snapshot [VM-params] export-filename.dbunit.xml
```

or, on Unix and Mac OS X systems,

```
sh snapshot [VM-params] export-filename.dbunit.xml
```

If the export filename is left out, the snapshot will be stored in a file called `snapshot.dbunit.xml`.

You need the following VM parameters to configure database access.

Use them like `-Ddb.user=me` :

Parameter	Description
dbUrl	The JDBC URL of the database
dbDriver	The JDBC driver class name
dbUser	user name
dbPassword	user password
dbSchema	Name of the schema to extract (defaults to the user name)

20.3 Benchmark Tool

Benerator provides a Benchmark Tool to assess and compare the performance of a typical generation or anonymization approaches.

It is of special use for you if you want to assess the generation/anonymization performance of different hard- and software settings, like numbers of cores, operating system, Java virtual machine, system software configuration and Benerator Enterprise Edition's multithreading configuration.

The different benchmarks used perform a list of predefined typical generation and anonymization tasks.

To invoke the Benchmark Tool with standard settings, just open a text console or terminal and enter

```
benerator-benchmark
```

Then the benchmark runs for a few minutes and prints a measurement summary. In Benerator Consumer Edition, only single-threaded execution is supported, so the report will look something like this:

```
+-----+
| Benchmark throughput of Benerator Community Edition 2.0.0-jdk-11-SNAPSHOT |
| on a Mac OS X 11.4 x86_64 with 8 cores                                |
| Java version 11.0.11                                                  |
| OpenJDK 64-Bit Server VM 11.0.11+9 (AdoptOpenJDK)                   |
| Date/Time: 2021-09-17T10:41:43.510603+02:00[Europe/Berlin]           |
| |                                                                      |
| Numbers are reported in million entities generated per hour           |
+-----+-----+
| Benchmark                                     | 1 Thread |
+-----+-----+
| gen-string.ben.xml                           |      37 |
+-----+-----+
| gen-person-showcase.ben.xml                  |      26 |
+-----+-----+
| anon-person-showcase.ben.xml                 |      31 |
+-----+-----+
| anon-person-regex.ben.xml                   |     346 |
+-----+-----+
| anon-person-hash.ben.xml                    |     386 |
+-----+-----+
| anon-person-random.ben.xml                  |     576 |
+-----+-----+
| anon-person-constant.ben.xml                |    2,210 |
+-----+-----+
```

In the header the system settings are reported, then each of the following rows displays the benchmark name and its performance, measured in a million entities generated per hour. This means for example that the `anon-person-constant.ben.xml` anonymizes 2,210 million = 2.210 billion data sets per hour running in a single thread.

The performance numbers above have been measured on a plain Macbook Air M1 of 2020.

For a Benerator Enterprise Edition installation running on a machine with several cores, the benchmark is executed for several characteristic executionMode settings in order to find the sweet spot of executionMode settings.

A benchmark runs on the same system with Benerator Enterprise Edition yields the following result:

```
+-----+
| Benchmark throughput of Benerator Enterprise Edition 2.0.0-jdk-11-SNAPSHOT |
| on a Mac OS X 11.4 x86_64 with 8 cores                                |
| Java version 11.0.11                                                  |
+-----+
```

```
| OpenJDK 64-Bit Server VM 11.0.11+9 (AdoptOpenJDK) |
| Date/Time: 2021-09-17T10:09:44.460364+02:00[Europe/Berlin] |
| |
| Numbers are reported in million entities generated per hour |
+-----+-----+-----+-----+
| Benchmark | 1 Thread | 2 Threads | 4 Threads | 6 Threads |
+-----+-----+-----+-----+
| gen-string.ben.xml | 243 | 379 | 809 | 747 |
+-----+-----+-----+-----+
| gen-person-showcase.ben.xml | 88 | 162 | 249 | 193 |
+-----+-----+-----+-----+
| anon-person-showcase.ben.xml | 83 | 165 | 241 | 187 |
+-----+-----+-----+-----+
| anon-person-regex.ben.xml | 684 | 1,008 | 1,145 | 794 |
+-----+-----+-----+-----+
| anon-person-hash.ben.xml | 923 | 1,344 | 1,142 | 1,187 |
+-----+-----+-----+-----+
| anon-person-random.ben.xml | 1,250 | 1,655 | 1,254 | 1,274 |
+-----+-----+-----+-----+
| anon-person-constant.ben.xml | 1,926 | 2,533 | 1,522 | 1,503 |
+-----+-----+-----+-----+
```

Note that we not only have improved the performance of the Community Edition, but optimized the Enterprise Edition to be even several times faster than the Community Edition.

For your performance optimization in Enterprise Edition, note that with additional threads' comes additional performance, but after a certain level of concurrency is reached, performance does not improve or even may deteriorate seriously. This may have one or more out of several reasons:

- Coordination and synchronization overhead
- More congestion of threads waiting at bottlenecks
- Having serious work load on more threads than CPUs are available: The more threads between a CPU has to switch back and forth, the more time is lost on each context switch and you may end up spending more time switching than working.
- With more threads comes higher throughput, but also higher storage needs. When critical buffer size limits are exceeded, a system's processing capacity may go down significantly though the overall CPU load looks relatively low.

The sweet spot where you have optimum performance with low concurrency usually is where the number of threads equals the number of cores, or is only slightly larger. As you might guess from the performance, the test laptop has 4 cores. Actually, it has more, but its 4 high-performance cores are the only ones that matter for generation and anonymization performance.

The Benchmark Tool has some command line parameters to configure its test runs. For a short summary, type `benerator-benchmark --help`

The general invocation format is

```
benerator-benchmark [options] [name]
```

Name is an optional benchmark name. When specified, only this benchmark is executed. When left out, all benchmarks applicable to the given configuration are called.

Example: In order to only call the `gen-string` benchmark, type

```
benerator-benchmark [options] [name]
```

The command line options are as follows:

Option	Meaning	Default Setting
--ce	Run on Benerator Community Edition (CE)	true on CE
--ee	Run on Benerator Enterprise Edition (EE)	true EE and only available there
--minSecs n	Choose a workload to have the benchmark run at least n seconds	10
--maxThreads k	Use only up to k cores for testing (only on EE)	a bit more than the number of reported cores
--env <spec>	Runs the tests applicable to the specified system(s). <spec> may be an environment name, a system (denoted by environment#system) or a comma-separated list of these (without whitespace)	
--mode m	activates Benerator mode <code>strict</code> , <code>lenient</code> or <code>turbo</code>	lenient
--list	lists the names of the predefined benchmarks	
--help	print this help	

A **--minSecs** settings of 30 requires the benchmark to run with a workload that needs at least 30 seconds to process. It is advisable to choose times which are sufficiently large that the fix initialization time of Benerator has less impact on the measurement and that JVM hotspot optimizers get some time to make Benerator run even more efficiently. If --minSecs is not specified, a default of 10 seconds is used, which is too short for optimum measurements, but was chosen as a defensive measure to save your system's hard drive space: The file generation benchmarks could easily fill up a terabyte of hard disk space in a few minutes. Even on standard developer hardware, a file export of one minute can produce a file of a size of 10 Gigabytes. Generated files are deleted automatically after each benchmark run, but take care not to fill up your disk during a benchmark run.

By default, the Benchmark tool tests thread settings from single-threaded to a concurrency slightly larger than the number of cores of the system it is running on. Unfortunately, some systems report a higher number of cores than are available for our tests. For example a Macbook Air M1 of 2020 reports to have 8 cores, but only 4 of them are high performance cores, 4 are efficiency cores which do not contribute performance to Benerator. So a setting of `--maxThreads 6` makes the Benchmark go up only to 6 threads instead of 10 threads it would have taken by default.

The reports above have been created using

```
benerator-benchmark --ce --minDurationSecs 30 --maxThreads 6
```

20.3.1 Assessing Database Performance

In order to assess database processing performance, you need to configure the relevant database(s) in an environment repository, see [Environment Files](#) for details.

For a quick start, the Benchmark Tool comes with two built-in databases `h2` and `hsqlmem` in an environment `builtin`. This will run the database benchmarks on the built-in h2:

```
benerator-benchmark --builtin#h2
```

In order to test all systems defined in the environment `builtin`, specify just the environment name

```
benerator-benchmark --dev builtin
```

You can test your own databases in a similar manner by defining them in an environment file (eg. `local.env.properties`) and providing it in the command line options as above.

You will notice that the database benchmark may run significantly longer than the simple tests. This is caused by measuring two access types, 'read' and 'write' and requiring each of these to run at least 'minSecs' seconds. For some databases, writing is

significantly slower than reading (up to a factor of 20), so that you need to write data for 10 minutes in order to read data for 30 seconds. The database benchmarks alleviate that a bit, by performing two reads for each write (effectively halving execution time), but still will take long time. So please be patient.

20.3.2 Assessing Kafka Performance

Testing Kafka performance is a bit tricky, so the Benchmark tool needs one dedicated topic per test. You can reuse pre-existing topics, but they must be empty when starting the tests. Otherwise, the benchmark may read pre-existing data leading to wrong performance metrics.

Currently, there are two Kafka benchmarks:

Name	required 'system' name	Description
kafka-small-entity	kafka_small_entity	Reads and writes messages with small entities in JSON format
kafka-big-entity	kafka_big_entity	Reads and writes messages with big entities in JSON format (several KBs)

A `dev` environment file might look like this, and you can use it to map the system names to topic names which are actually available on your Kafka cluster (`dev.env.properties`):

```
kafka_small_entity.kafka.bootstrap.servers=localhost:9092
# use the following line to specify a topic for the kafka-small-entity benchmark
kafka_small_entity.kafka.topic=kafkaQueue1
kafka_small_entity.kafka.format=json

kafka_big_entity.kafka.bootstrap.servers=localhost:9092
# use the following line to specify a topic for the kafka-big-entity benchmark
kafka_big_entity.kafka.topic=kafkaQueue2
kafka_big_entity.kafka.format=json
```

20.4 XML Creator

The XMLCreator reads a XML Schema file and creates a number of XML files that comply with the schema. It can read XML annotations which provide generator configuration in the XML schema file. It is invoked from the command line and has the following parameter order:

```
createxml <schemaUri> <root-element> <filename-pattern> <file-count> [<properties file name(s)>]
```

Their meaning is as follows:

- `schemaUri`: the location (typically file name) of the XML schema file
- `root-element`: the XML element of the schema file that should be used as root of the generated XML file(s)
- `filename-pattern`: the naming pattern to use for the generated XML files. It has the form of a `java.text.MessageFormat` pattern and takes the number of the generated file as parameter `{0}`.
- `file-count`: the number of XML files to generate
- `properties file name(s)`: an optional (space-separated) list of properties files to include in the generation process

Under Windows, an example call would be:

```
createxml myschema.xsd product-list products-{0}.xml 10000 perftest.properties
```

or, on Unix and Mac OS X systems,

```
sh myschema.xsd product-list products-{0}.xml 10000 perftest.properties
```

for generation 10,000 XML files that comply with the XML Schema definition in file `myschema.xsd` and have `product-list` as root element. The files will be named `products-1.xml`, `products-2.xml`, `products-3.xml`, ...

21. Domains

Generator domains are a vehicle for defining, bundling and reusing domain specific data generation, e.g. for personal data, addresses, internet, banking, telecom. They may be localized to specific languages and be grouped to hierarchical datasets, e.g. for continents, countries and regions.

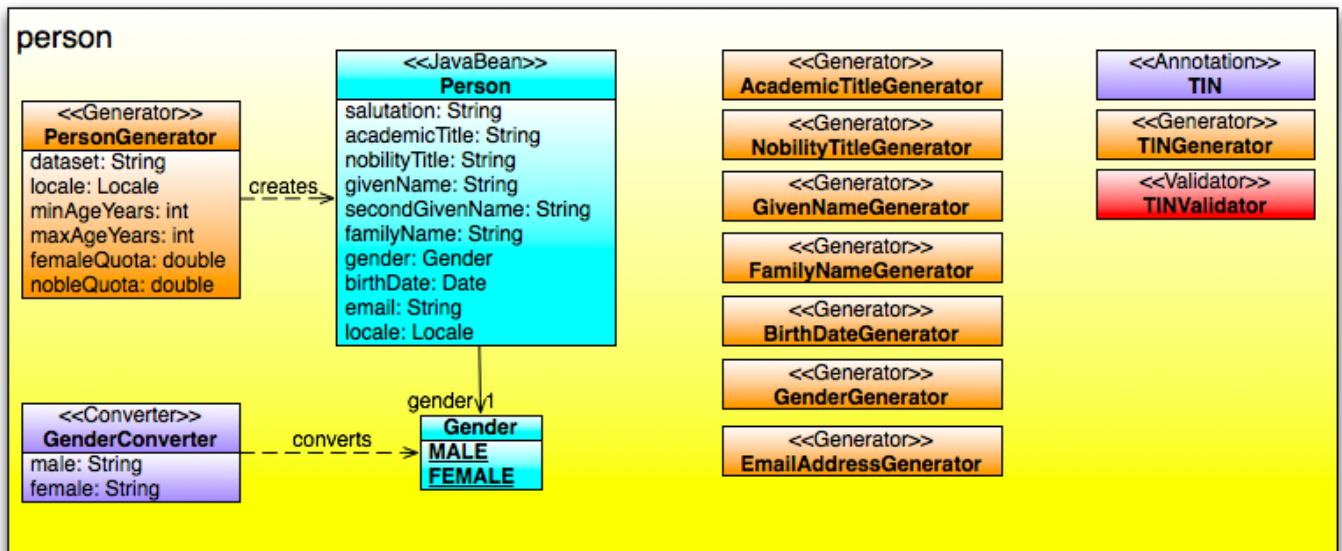
Generator comes packaged with several domains that have simple implementation of specific data generation and may serve as a base for deriving own specific data generation domains:

- **person**: Data related to a person
- **address**: Data related to contacting a person by post
- **net**: Internet and network related data
- **finance**: finance data
- **organization**: Organization data
- **product**: Product-related data

21.1 person domain

The person domain has three major components:

- PersonGenerator: Generates Person beans
- AcademicTitleGenerator: Generates academic titles
- NobilityTitleGenerator: Generates nobility titles
- GivenNameGenerator: Generates given names
- FamilyNameGenerator: Generates family names
- BirthDateGenerator: Generates birth dates
- GenderGenerator: Generates Gender values
- EmailAddressGenerator: Generates Email addresses
- GenderConverter: Converts Gender values to predefined strings
- TIN: Marks a Java attribute or property as a European Tax Identification Number
- TINGenerator: Creates European Tax Identification Numbers
- TINValidator: Validates European Tax Identification Numbers



21.1.1 PersonGenerator

Creates Person beans to be used for prototype-based data generation. It can be configured with dataset and locale property. The generated Person JavaBeans exhibits the properties *salutation*, *title* (both locale-dependent), *givenName*, *familyName* (both dataset-dependent), *gender*, *birthDate*, *age*. If the chosen dataset definition provides name weights, benerator generates person names according to their statistical probability. Of course, gender, salutation and givenName are consistent.

You can use the PersonGenerator like this:

```
<import domains="person"/>

<generate type="user" count="5" consumer="ConsoleExporter">
  <variable name="person" generator="PersonGenerator" dataset="FR" locale="fr" />
  <attribute name="salutation" source="person.salutation" />
  <attribute name="name" script="{person.givenName + ' ' + person.familyName}" />
</generate>
```

to get output similar to this:

```
user[salutation=Mr, name=David Morel]
user[salutation=Mr, name=Robert Robert]
user[salutation=Mr, name=Eric Morel]
user[salutation=Mr, name=Patrick Lefebvre]
user[salutation=Mme, name=Helene Fournier]
```

21.1.2 PersonGenerator Properties

The PersonGenerator can be configured with several properties:

Property	Description	Default Value
dataset	Either a region name or the two-letter-ISO-code of a country, e.g. US for the USA. See ' Advanced Topics > Region nesting '	The user's default country
locale	Two-letter-ISO-code of the language in which to create salutation and titles, e.g. en for English	The user's default language
minAgeYears	The minimum age of generated persons	15
maxAgeYears	The maximum age of generated persons	105
femaleQuota	The quota of generated women (1 → 100%)	0.5
nobleQuota	The quota of generated noble persons (1 → 100%)	0.005

21.1.3 Person Class

The Person class has the following properties:

property name	type	property description
salutation	String	Salutation (e.g. Mr/Mrs)
academicTitle	String	Academic title (e.g. Dr)
nobilityTitle	String	Nobility title (like Duke)
givenName	String	Given name ('first name' in western countries)
secondGivenName	String	An eventual second given name
familyName	String	Family name ('surname' in western countries)
gender	Gender	Gender (MALE or FEMALE)
birthDate	Date	Birth date
age	Integer	actual age
email	String	eMail address
locale	Locale	Language of the person instance (used e.g. for salutation)

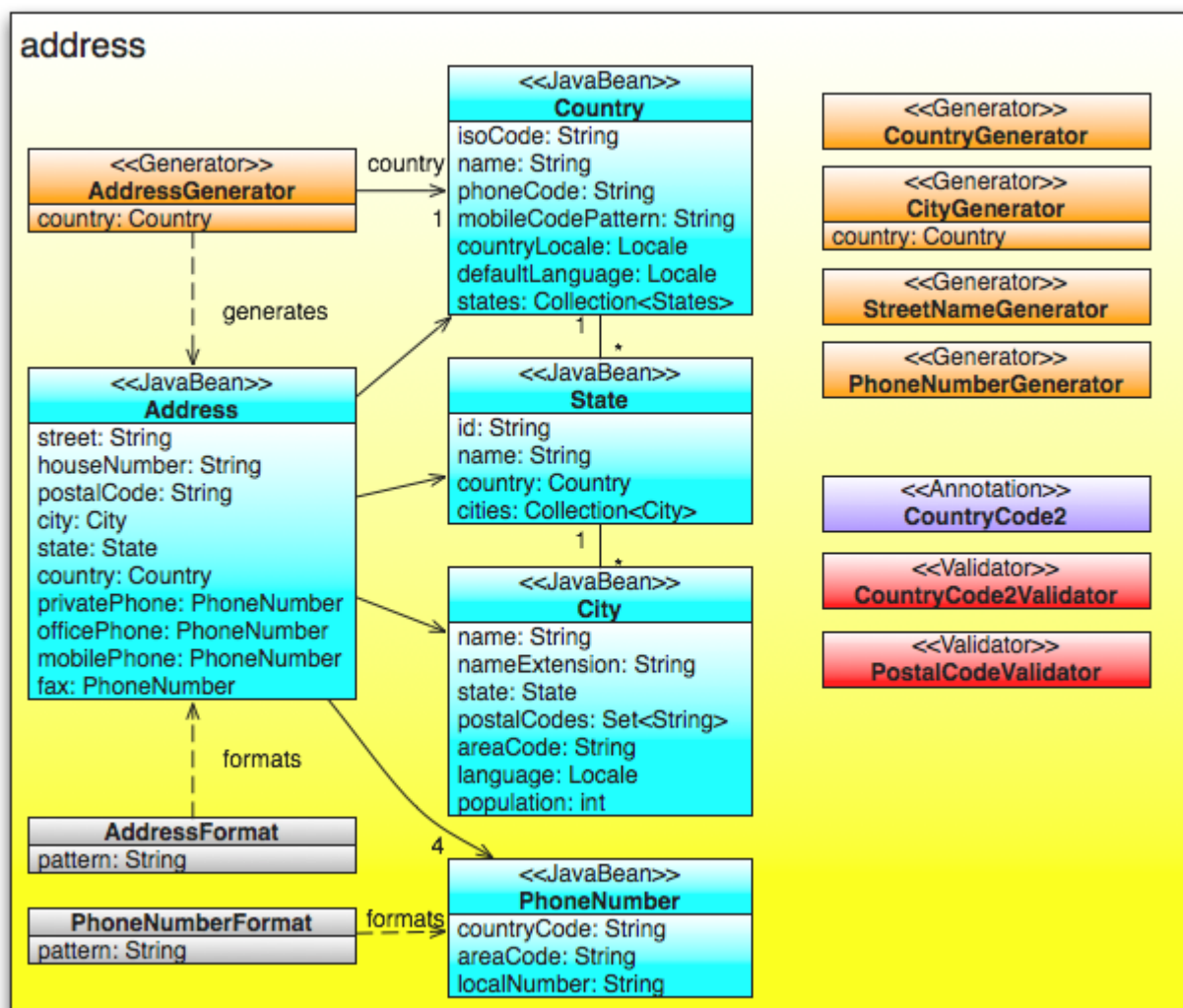
21.1.4 Supported countries

country	code	remarks
Austria	AT	most common 120 given names with absolute weight, most common 40 family names with absolute weight
Australia	AU	most common 40 given names (unweighted), most common 20 family names with absolute weight
Brazil	BR	most common 100 given names (unweighted), most common 29 family names (unweighted)
Canada	CA	most common 80 given names (unweighted), most common 20 family names (unweighted). No coupling between given name locale and family name locale
Switzerland	CH	most common 30 given names with absolute weight, most common 20 family names with absolute weight
Czech Republic	CZ	most common 20 given names with absolute weight, most common 20 family names with absolute weight. Female surnames are supported.
Spain	ES	most common 40 given names (unweighted), most common 40 family names with absolute weight
Finland	FI	most common 785 given names (unweighted), most common 448 family names (unweighted)
France	FR	most common 100 given names (unweighted), most common 25 family names with relative weight
Germany	DE	most common 1998 given names with absolute weight, most common 3421 family names with absolute weight ²
Israel	IL	264 given names (unweighted), most common 30 family names with relative weight
India	IN	most common 155 given names (unweighted), most common 50 family names (unweighted)
Italy	IT	most common 60 given names (unweighted), most common 20 family names (unweighted)
Japan	JP	Kanji letters. Most common 109 given names (unweighted), most common 50 family names with absolute weight
Netherlands	NL	3228 given names (unweighted), most common 10 family names with absolute weight
Norway	NO	most common 300 given names (unweighted), most common 100 family names with absolute weight
Poland	PL	most common 67 given names with absolute weight, most common 20,000 family names with absolute weight. Female surnames are supported.
Russia	RU	Cyrillic letters. Most common 33 given names with relative weight, most common 20 family names with relative weight. Female surnames are supported.
Sweden	SE	779 given names (unweighted), most common 22 family names with relative weight
Turkey	TR	1077 given names (unweighted), 37 family names (unweighted)
United Kingdom	GB	most common 20 given (unweighted), most common 25 family names (unweighted)
USA	US	most common 600 given names and most common 1000 family names both with absolute weight

Your use-cases are more sophisticated and require additional countries, cities or valid addresses down to street level and number, check our [Benerator Editions](#) or feel free to contact us for assistance: team@rapiddweller.com.

21.2 Address domain

- **AddressGenerator**: Generates addresses that match simple validity checks: The City exists, the ZIP code matches and the phone number area codes are right. The street names are random, so most addresses will not stand validation of real existence.
- **PhoneNumberGenerator**: Generates landline telephone numbers for a country
- **MobilePhoneNumberGenerator**: Generates mobile phone numbers for a country
- **CountryGenerator**: Generates countries
- **CountryCode2**: Annotation that marks a Java attribute or property as ISO-3166-1 alpha-2 code
- **CountryCode2Validator**: Java Bean Validation ConstraintValidator for ISO-3166-1 alpha-2 codes
- **CityGenerator**: Generates Cities for a given country
- **StreetNameGenerator**: Generates street names for a given country
- **PostalCodeValidator**: Validates if a given postal code is valid in a given country



The following countries are supported:

country	code	remarks
USA	US	Valid ZIP codes and area codes, no assurance that the street exists in this city.
United Kingdom	GB	Valid area codes, no postcodes, no assurance that the street exists in this city or the local phone number has the appropriate length. Contributions are welcome
Germany	DE	Valid ZIP codes and area codes, no assurance that the street exists in this city or the local phone number has the appropriate length
Switzerland	CH	Valid ZIP codes and area codes, no assurance that the street exists in this city or the local phone number has the appropriate length
Brazil	BR	Valid ZIP codes and area codes, no assurance that the street exists in this city or the local phone number has the appropriate length

21.3 net domain

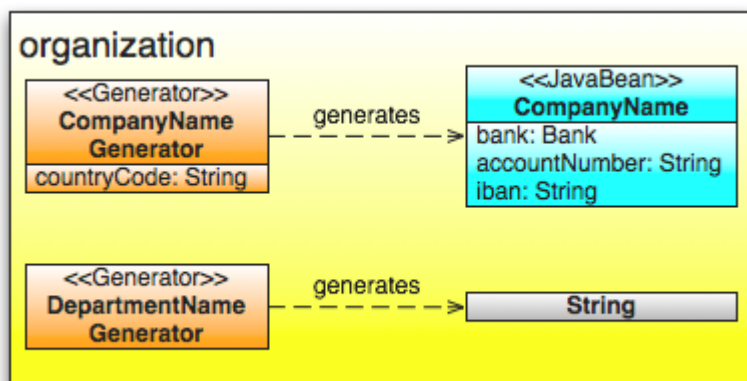
The net domain provides the

- **DomainGenerator**, which generates Internet domain names

21.4 organization domain

Provides the following generators:

- **CompanyNameGenerator**, a generator for company names.
- **DepartmentNameGenerator**, a generator for department names



If you use the CompanyNameGenerator like this:

```
<import domains="organization" />

<generate type="company" count="5" consumer="ConsoleExporter">
  <attribute name="name" generator="CompanyNameGenerator" dataset="DE" locale="de_DE"/>
</generate>
```

you get output like this:

```
company[name=Belanda Aktiengesellschaft & Co. KG]
company[name=MyWare Technologies GmbH]
company[name=WebBox AG]
company[name=Altis AG]
company[name=Ernst Fischer Technik GmbH]
```

Company names can be generated for the following countries:

country	code	remarks
Germany	DE	none
USA	US	none

The Generator creates objects of type `CompanyName`, consisting of **shortName**, **sector**, **location** and **legalForm** information. You can make use of the object as a whole which is converted to a string automatically using the **'fullName'** version as shown in the example above. But you can also make direct use of the basic properties:

```
<import domains="organization" />

<generate type="company" count="5" consumer="ConsoleExporter">
  <variable name="c" generator="CompanyNameGenerator" dataset="DE" locale="de_DE"/>
  <attribute name="name" script="c.shortName + ' ' + c.legalForm" />
</generate>
```

21.5 finance domain

Generates and validates finance related data:

The following classes are provided:

- **BankAccountGenerator**: Generates BankAccount JavaBeans
- **BankAccountValidator**: Validates BankAccount JavaBeans
- **CreditCardNumberGenerator**: Generates strings which represent credit card numbers
- **CreditCardNumberValidator**: Validates strings as credit card numbers
- **IBAN**: Annotation for Java Bean Validation, marking a Java attribute/property as IBAN
- **IBANValidator**: Validates strings with IBANs

21.6 product domain

The product package provides you with Generator classes for EAN codes:

- **EAN8Generator**: Generates 8-digit EAN codes
- **EAN8**: Annotation that marks a Java attribute or property as 8-digit-EAN for bean validation
- **EAN8Validator**: Validates 8-digit EAN codes
- **EAN13Generator**: Generates 13-digit EAN codes
- **EAN13**: Annotation that marks a Java attribute or property as 13-digit-EAN for bean validation
- **EAN13Validator**: Validates 13-digit EAN codes
- **EANGenerator**: Generates both 8-digit and 13-digit EAN codes
- **EAN**: Annotation that marks a Java attribute or property as an EAN for bean validation
- **EANValidator**: Validates 8- and 13-digit-EAN codes

Each generator has a property **'unique'**: If set to true the generator assures that no two identical EAN codes are generated.

21.7 br domain

Provides classes specific to Brazil:

- **CPNJ**: Annotation to mark a Java attribute or property as a CPNJ (Cadastro Nacional da Pessoa Jurídica)
- **CPNJGenerator**: Generates CPNJs
- **CPNJValidator**: Validates CPNJs and can be used as Benerator validator and as ConstraintValidator in Java Bean Validation (JSR 303)
- **CPF**: Annotation to mark a Java attribute or property as a CPF (Cadastro de Pessoa Física)
- **CPFGenerator**: Generates CPFs
- **CPFValidator**: Validates CPFs

21.8 us domain

Provides classes specific for the United States of America:

- **SSN**: Annotation to mark a Java attribute or property as a Social Security Number
- **SSNGenerator**: Generates Social Security Numbers
- **SSNValidator**: Validates Social Security Numbers and can be used as Benerator validator and as ConstraintValidator in Java Bean Validation (JSR 303)

22. Component Reference

Benerator has lots of predefined generators which are available implicitly from the descriptor. Most of them only need to be created explicitly when using the Benerator API programmatically.

22.1 Generators

22.1.1 Domain Generators

For domain-specific generators (e.g. person, address, finance), see [Domains](#).

22.1.2 Common Id Generators

Benerator contains the following common predefined and platform-independent generators:

- **IncrementalIdGenerator**: Creates consecutive id values, starting with 1 by default.
- **UUIDGenerator**: Creates UUIDs by the JDK class `java.util.UUID`
- **HibUUIDGenerator**: Creates UUIDs like the Hibernate UUID key generator
- **LocalSequenceGenerator**: Mimics the behavior of a (named) database sequence on a single client VM. Its property 'cached' (true by default) specifies if sequence value changes shall be persisted immediately or in the end.

22.1.3 Database-related Generators

See [Using Relational Databases](#).

22.1.4 simple type generators

- **CharacterGenerator**: Generates random characters.
- **IncrementGenerator**: Generates numbers starting with one and incrementing the number on each subsequent call
- **StringGenerator**: Generates strings based on character set, prefix, suffix and length characteristics. This is the typical component for generating code numbers. Properties: `charSet` (regular expression for a character class), `locale`, `unique`, `ordered`, `prefix`, `minInitial`, `suffix`, `minLength`, `maxLength`, `lengthGranularity`, `lengthDistribution`
- **RegexStringGenerator**: Generates strings that match a given regular expression. This is the typical component for generating strings that are composed of different sub patterns. Properties: `pattern` (regular expression), `unique`, `ordered`, `locale`, `minLength`, `maxLength`
- **MessageGenerator**: Composes strings using a `MessageFormat`
- **LuhnGenerator**: Generates Luhn-valid strings like credit card numbers

22.1.5 current date / time generators

- **CurrentDateGenerator**: Generates `java.util.Date` objects that represent the current date
- **CurrentDateTimeGenerator**: Generates `java.util.Date` objects that represent the current date and time
- **CurrentZonedDateTimeGenerator**: Generates `java.util.ZonedDateTime` objects that represent the current date and time
- **CurrentMilliTimeGenerator**: Generates long values that denote the number of milliseconds since 1970-01-01 00:00:00
- **CurrentNanoTimeGenerator**: Generates long values that denote a number of milliseconds since an arbitrary point in time (possible even in the future, so values may be negative)
- **CurrentTimeGenerator**: Generates `java.util.Date` objects that represent the current time of the day

22.1.6 arbitrary date / time generators

- **DateGenerator:** Generates date values that represent a certain time at a certain day based on a common Distribution
- **DayGenerator:** Generates date values that represent „day“ dates – dates at midnight
- **DateTimeGenerator:** Generates date values with date and time configurable independently. Its properties are: minDate, maxDate, dateGranularity, dateDistribution, minTime, maxTime, timeGranularity, timeDistribution. For a 9-to-5 datetime on odd days in August 2010, configure
- **ZonedDateTimeGenerator:** TODO

```
<bean id="dtGen" class="DateTimeGenerator">
  <property name='minDate' value='2010-08-01' />
  <property name='maxDate' value='2010-08-31' />
  <property name='dateGranularity' value='00-00-02' />
  <property name='dateDistribution' value='random' />
  <property name='minTime' value='08:00:00' />
  <property name='maxTime' value='17:00:00' />
  <property name='timeGranularity' value='00:00:01' />
  <property name='timeDistribution' value='random' />
</bean>
```

22.1.7 file related generators

- **FileGenerator:** generates java.io.File objects representing files in a given directory structure
- **FileNameGenerator:** generates file names representing files in a given directory structure
- **TextFileContentGenerator:** provides text file contents as String
- **BinaryFileContentGenerator:** provides binary file contents as byte[]

22.1.8 State Generators

- **StateGenerator:** Generates states based on a state machine
- **StateTransitionGenerator:** Like the StateGenerator, but generating Transition objects

22.1.9 Seed Based Generators

- **SeedWordGenerator:** Generates new words based on rules derived from a dictionary.
- **SeedSentenceGenerator:** Generates sentences based on rules derived from a text file.

22.2 Distributions

A Distribution describes stochastic properties for distributing the data that Benerator generates. You can use the predefined distributions or implement and introduce custom implementations. The most important types of distribution are *Sequence*, *WeightFunction* and *CumulativeDistributionFunction*.

A Distribution implements a common concept for generating numbers or taking values from a data source and providing them in a rearranged order or distribution with similar semantics as the number generation feature.

As an example, a 'Skip2' sequence might generate numbers with an increment of 2: 1, 3, 5, 7,... When it is used to redistribute given data item1, item2, item3, item4,..., it would provide the values item1, item3, ...

While most Distribution components implement number generation as well data rearrangement, they are not required to support both concepts.

All Distributions listed below are included in the default imports.

22.2.1 Memory consumption

Distributions that are based on number generation may adopt data redistribution by simply loading all available data into a long list in RAM and then using their number generation feature to determine indices of the data to provide. If the data amount is large, you may get memory problems. In order to provide an easy start, Benerator reduces the default size of these lists to 100,000 elements, prints out an error message if the number is exceeded, but simply continues to work with the reduced amount of data. You can allow Benerator to use a larger cache by adding a `benerator.cacheSize` to your **BENERATOR_OPTS**, e.g.

```
-Dbenerator.cacheSize=2000000
```

If this makes you run into an `OutOfMemoryError`, check the [Troubleshooting](#) section on how to allocate a larger Java heap in Benerator.

22.2.2 Sequences

Sequences reflect the idea of a mathematical sequence. They primarily focus in number generation, but they can be applied for data redistribution as well. Most sequences have a default instance which can be used by their literal, e.g. `distribution="random"` uses the `random` literal for the distribution defined in the class `RandomSequence`.

RandomSequence

Class	RandomSequence
Description	Creates uniformly distributed random values
Default Instance	random

CumulatedSequence

Class	CumulatedSequence
Description	Creates random values with a bell-shape probability distribution
Default Instance	cumulated

StepSequence

Class	StepSequence
Description	Depending on the settings of property 'delta', it starts with the min or max value of the specified range. With each further invocation, the 'increment' value is added. If addition makes the current value exceed the specified number range, the Sequence becomes unavailable. So the numbers provided are unique. Example: increment = -2, range=1..7: 7, 5, 3, 1
Default Instances	increment: Uses delta = 1 to create incremental values, e.g. 1, 2, 3,...

Property	Property Description	Default Value
delta	The difference between the next value and the previous one	1

RandomWalkSequence

Class		RandomWalkSequence
Description	Starting with an $\rightarrow$ initial value, a random value between $\rightarrow$ minStep and $\rightarrow$ **	
	maxStep** is added on each subsequent invocation	
Default Instance	randomWalk	
Property	Property Description	Default Value
minStep	The maximum delta between the next and the previous value	-1
maxStep	The maximum delta between the next and the previous value	1
initial	If no initial value was configured explicitly, number generation starts with the min, max or medium value of the specified range - depending on the settings of minStep and maxStep	null

ShuffleSequence

Class		ShuffleSequence
Description	Can be used to iterate quickly through a large number range with avoiding duplicate values. It starts from an offset of 0 and iterates the number range with a fix increment. After the range is covered, it increases the offset by one and reiterates the range. When the offset reaches the same value as the increment, it is set back to 0 again. For an increment of 3 in a range 1..7, the generated numbers would be 1, 4, 7, 2, 5, 3, 6, 1, 4, ...	
Default Instance	shuffle	
Property	Property Description	Default Value
increment	See the class description	2

WedgeSequence

Class		WedgeSequence
Description	Starting with first the lowest, then the highest available number, this alternatively provides increasing small numbers and decreasing large numbers until they converge in the middle and the Sequence becomes unavailable. So this generation is unique. For a number range 1..7, the generated numbers would be: 1, 7, 2, 6, 3, 5, 4.	
Default Instance	wedge	

BitReverseSequence

Class		BitReverseSequence
Description	Creates numbers by continually increasing an internal counter and providing its value in bit-reversed order. This stops when each available number has been generated once, thus providing unique number generation. This comes close to a unique random distribution.	
Default Instance	bitreverse	

ExpandSequence

Class			ExpandSequence		
Description			Distributes numbers or data of unlimited volume in a unique or non-unique manner, by starting with a limited lower range and continuously expanding data region as data is generated. This comes close to a unique random distribution and can be used to iterate over very huge amounts of data.		
Default Instance			expand		
Property			Property Description		Default Value
cacheSize			The maximum number of elements to keep in RAM at the same time		100
bucketSize			The size of 'buckets' over which to distribute the iterated data. The smaller the number, the more buckets are used and the more erratic the generated data looks.		10
duplicationQuota			The probability by which a data element will be reused in a later call		0

HeadSequence

Class			HeadSequence		
Description			When applied to a data source or generator, only the first few elements are provided. The number of elements is defined by the **		
size** property.					
Default Instance			head		
Property			Property Description		Default Value
size			The size of the buffer		1

LiteralSequence

Class			LiteralSequence		
Description			Defines a number sequence using a comma-separated list literal.		
Default Instance			-		
Property			Property Description		Default Value
spec			A comma-separated list with all values in the order in which they shall be provided ,e.g. '2,3,5,7,11'		-

WeightedNumbers

Class		WeightedNumbers	
Description		Creates numbers based on a weighted-number literal , e.g. '1^70, 3^30' for generating 70% '1' values and 30% '3' values. This is a convenient and simple approach for controlling parent-child cardinalities in nested data generation.	
Default Instance		-	

Property	Property Description	Default Value
spec	A weighted-number literal. It lists weighted values in a comma-separated list. Each weighted value is specified by the numeric value followed by a circumflex (^) and the weight value, for example '1^70,3^30'	-

FibonacciSequence

Class		FibonacciSequence
Description		Generates numbers based on the Fibonacci Sequence
Default Instance		fibonacci

PadovanSequence

Class		PadovanSequence
Description		Generates numbers based on the Padovan Sequence
Default Instance		padovan

22.2.3 Weight Functions

Weight functions are another special case of Distributions. They are based on a function which is supposed to allow continuous value generation, but since Benerator needs to perform a numerical integration for deriving random values, a granularity must be applied. This way, the generated value set is quantized. Another drawback of the approach is that fine-grained generation is memory-consuming and slow.

Thus, it is recommended to avoid weight functions if possible and choose a similar Sequence or CumulativeDistributionFunction instead.

GaussianFunction

Implements the well-known Gaussian Function.

Full class name: `com.rapiddweller.benerator.distribution.function.GaussianFunction`

Parameters: `average` [, `deviation`]

Example:

```
<import class="com.rapiddweller.benerator.distribution.function.*"/>
...
<attribute name="price" type="big_decimal" min="0.1" max="99.99" granularity="0.1"
distribution="new GaussianFunction(50,20)"/>
```

ExponentialFunction

The Exponential Function.

Full class name: `com.rapiddweller.benerator.distribution.function.ExponentialFunction`

Parameters: `[scale,]` frequency

Example:

```
<import class="com.rapiddweller.benerator.distribution.function.*"/>
...
<attribute name="category" type="string" values="A,B,C" distribution="new ExponentialFunction(0.5)"/>
```

DiscreteFunction

Discrete Function that specifies an explicit weight for each possible value

Full class name: `com.rapiddweller.benerator.distribution.function.DiscreteFunction`

Parameters: `weight1 [, weight2 [, weight3 ...]]`

Example:

```
<import class="com.rapiddweller.benerator.distribution.function.*"/>
...
<attribute name="rating" type="int" min="1" max="3" distribution="new DiscreteFunction(1, 2, 1)"/>
```

22.2.4 CumulativeDistributionFunction

The CumulativeDistributionFunction is another special case of a Distribution, which allows for continuous value generation as opposed to Sequence and WeightFunction.

22.2.5 ExponentialDensityIntegral

Inverse of the integral of the probability density $f(x) = a \cdot e^{-ax}$ ($x > 0$), which resolves to $F^{-1}(x) = -\log(1 - x) / a$.

22.3 Converters

Benerator supports two different types of converter interfaces:

- [com.rapiddweller.common.Converter](#)
- [java.text.Format](#)

22.3.1 Converter Setup

Instantiation of converters follows the same principles as other objects handled by benerator. They can be instantiated either as beans (see NoiseInducer example below) or inline.

An inline instantiation of the Mask converter below:

```
<iterate count="5" source="db" consumer="ConsoleExporter">
  <attribute name="column" type="string" converter="Mask"/>
</iterate>
```

Configuration for inline usage of converters (cp. `new Mask{maskChar = '$' }`) to set converter properties with curly brackets notation looks like:

```
<iterate count="5" source="db" consumer="ConsoleExporter">
  <attribute name="column" type="string" converter="new Mask{maskChar='$' }"/>
</iterate>
```

22.3.2 Default Converters

The following converter classes are imported by default:

- **ByteArrayToBase64Converter**: Converts byte arrays to strings which are base-64-encoded
- **ByteArray2StringConverter**: Converts byte arrays to strings
- **ToLowerCaseConverter**: Converts strings to lowercase
- **ToUpperCaseConverter**: Converts strings to uppercase
- **LiteralParser**: Parses strings as numbers, strings, dates and times
- **Mask**: Replaces each character of a string with an asterisk '*' or another configurable character (property : `maskChar`).
- **MiddleMask**: Replaces each character of a string with an asterisk '*' or another configurable character, leaving a configurable number of characters unmasked at the beginning and/or the end of the string.
- **Append**: Appends a configurable suffix.
- **CutLength**: Cuts the length of a string.
- **MD5Hash**, **SHA1Hash**, **SHA256Hash**: Convert any data to a hexadecimal hash code
- **MD5HashBase64**, **SHA1HashBase64**, **SHA256HashBase64**: Convert any data to a hash code in Base64 format
- **JavaHash**: Convert any data to a hexadecimal hash code. This implementation is faster than the hash converters above
- **MessageConverter**: Converts an object, wrapping it with a message string, using a `java.text.MessageFormat`
- **PropertyResourceBundleConverter**: Uses a Java `PropertyResourceBundle` to translate keywords to translations in a given Java Locale
- **ToStringConverter**: Converts arbitrary objects to strings
- **UniqueStringConverter**: Assures uniqueness for all processed Strings by appending unique numbers to recurring instances (attention: limited to a few 100.000 elements)
- **URLEncodeConverter**: Applies a URL encoding to strings
- **URLDecodeConverter**: decodes URL encoded strings
- **PrintfConverter**: formats objects using a pattern in printf format
- **RegexReplacer**: Uses a regular expression to replace parts of the processed strings
- **String2ByteArrayConverter**: Converts Strings to ByteArrays. Might be useful for writing JSON to raw data types (BLOB, BYTEA).
- **SubstringExtractor**: Extracts substrings from strings. It has the properties `from` and `to`. If `to` is not set, it extracts from `from` until the end. If `to` or `from` is negative, it denotes a backwards position count, making e.g. -1 the last character position.
- **EscapingConverter**: Escapes strings in Java style, like `A\tB`
- **Number2CharConverter**: Converts a number to a character of the corresponding ASCII code
- **Char2StringConverter**: Converts a character to a string of length 1
- **EscapingConverter**: Escapes control codes in a string in C and Java style, e.g. with `\r`, `\n`, `\t`
- **Number2CharConverter**: Converts a number to a character with the corresponding ASCII code, e.g. `65` → `A`

The package **com.rapiddweller.text** provides the following converters:

- **DelocalizingConverter**: Converts strings with non-ASCII letters to ASCII strings, e.g. Müller → Mueller, Søer → Soer
- **NameNormalizer**: Normalizes a string by trimming it, normalizing inner white space and formatting each word to start with an uppercase character and continue with lowercase characters
- **NormalizeSpaceConverter**: Trims a string and normalizes inner white space to one space character
- **ToHexConverter**: Renders characters, strings and integral numbers in hexadecimal representation

In the package **com.rapiddweller.benerator.primitive.number** there are two converters that can be used to quantize numerical values:

- **FloatingPointQuantizer, IntegralQuantizer, NumberQuantizer**: Quantize numbers to be a **min** value plus an integral multiple of a **granularity**
- **NoiseInducer**: Adds numerical noise to numbers. The noise characteristics can be configured with the properties `minNoise`, `maxNoise`, `noiseGranularity` and `noiseDistribution`. When setting the boolean property `relative` to `true`, noise is relative, where `maxCount=1` corresponds to 100% noise-to-signal ratio. If `relative=false`, the absolute value of the noise is added or subtracted. Example:

NoiseInducer example:

```
<bean id="inducer" class="com.rapiddweller.benerator.primitive.number.NoiseInducer">
  <property name="minNoise" value="-0.2"/>
  <property name="maxNoise" value="0.2"/>
  <property name="noiseGranularity" value="0.01"/>
  <property name="noiseDistribution" value="cumulated"/>
  <property name="relative" value="true"/>
</bean>

<generate count="5" consumer="ConsoleExporter">
  <attribute name="x" type="int" constant="100" converter="inducer"/>
</generate>
```

produces the result:

```
entity[x=99]
entity[x=105]
entity[x=92]
entity[x=104]
entity[x=99]
```



Find out more about built-in converters in the rapiddweller [rd-lib-common repository](#) and check the [available converter classes](#).

22.3.3 Java Formats

Beware that the [java.text.Format](#) classes are not thread-safe!

- **SimpleDateFormat**: Uses a pattern to format dates as strings
- **DecimalFormat**: Uses a pattern to format numbers as strings

22.4 Validators

22.4.1 Domain Validators

For the validators from the domains see [Domains](#).

22.4.2 Common validators

- **CharacterRangeValidator**: Validates if a character is in a certain range
- **NotNullValidator**: Requires the validated data to be not null
- **StringLengthValidator**: Limits allowed strings to a minimum and/or maximum length
- **StringValidator**: Validates string by min length, max length and a character validator
- **UniqueValidator**: Requires data to be unique (attention: limited to some 100.000 elements)
- **UnluckyNumberValidator**: Checks if a String contains an 'unlucky' number like 13 in western cultures or 4 in east-asian cultures

- **DayOfWeekValidator**: Accepts only Dates of certain (configurable) weekdays
- **RegexValidator**: Validates if a string matches a regular expression
- **LuhnValidator**: Checks if a number string (e.g. credit card number) is Luhn-valid

22.4.3 Tasks

- **FileJoiner**: Joins several files (**sources**) into a **destination** file, optionally **appending** the joint data to an existing destination file, or overwriting it. If **deleteSources** is set to true, the sources are deleted afterwards.
- **FileDeleter**: Deletes a number of **files**.

22.5 Consumers

A Consumer consumes generated data and usually is used for exporting or persisting the data.

22.5.1 LoggingConsumer

Class Name	LoggingConsumer
Import	default
Class Description	Logs all Consumer invocations to a logger

22.5.2 ConsoleExporter

Class Name	ConsoleExporter
Import	default
Class Description	Prints entities in the console
Constructors	Default constructorConstructor with 'limit' argument (see below)

Property	Property Description	Default Value
limit	The maximum number of entries per type to print out	unlimited
nullString	Text to represent <i>null</i> values	""
datePattern	The pattern to render date values	"yyyy-MM-dd"
timePattern	The pattern to render time values	"HH:mm:ss"
timestampPattern	The pattern to render timestamp values	"yyyy-MM-dd'T'HH:mm:ss.SSSSSSSS"
decimalPattern	The pattern to render decimal values	System default
decimalSeparator	The decimal separator to use for decimal values	System default
integralPattern	The pattern to integral number values	System default

22.5.3 JavaInvoker

Class Name	JavaInvoker
Import	<code><import platforms="java"/></code>
Class Description	Maps entity components to method parameters and invokes a method on a Java object with these parameters.

Property	Property Description	Default Value
target	The Java object on which to invoke the method	
methodName	The name of the Java method to invoke	

Usage example:

```
<bean id="service" spec="..." />
<bean id="invoker" spec="new JavaInvoker(ejb, 'enrolCustomer')"/>
```

22.5.4 DbUnitEntityExporter

Class Name	DbUnitEntityExporter
Import	<code><import platforms="dbunit"/></code>
Class Description	Exports entities to a file in DbUnit XML format.

Property	Property Description	Default Value
uri	The URI of the file to create	"data.dbunit.xml"
encoding	The character encoding to use for the file	The system default

22.5.5 XMLEntityExporter

Class Name	XMLEntityExporter
Import	<code><import platforms="xml"/></code>
Class Description	Exports entities to an XML file

Property	Property Description	Default Value
uri	The URI of the file to create	"export.xml"
encoding	The character encoding to use for the file	The system default

22.5.6 NoConsumer

Class Name	NoConsumer
Import	default
Class Description	In some cases a pseudo <code><generate></code> statements acts as a mechanism to perform a loop. In such cases a consumer does not make sense but causes Benerator to emit a warning „No consumers defined for <code><loop name></code> “. In order to avoid this warning, you can use the NoConsumer class, which is an empty implementation of the Consumer interface.

22.5.7 ScriptedEntityExporter

Class Name	ScriptedEntityExporter	
Import	<code><import platforms="script"/></code>	
Class Description	Exports entities to a file in custom format, rendered using a script language, e.g. FreeMarker. Three different script expressions may be applied for header (headerScript property), entity (partScript) and footer (footerScript).	

Property	Property Description	Default Value
uri	The URI of the file to create	
encoding	The character encoding to use for the file	The system default
headerScript	Script to format an eventual header line	
partScript	Script to format an exported entity	
footerScript	Script to format an eventual footer line	
nullString	Text to represent <i>null</i> values	""
datePattern	The pattern to render date values	"yyyy-MM-dd"
dateCapitalization	The capitalization to use when rendering a month name in a date: 'upper', 'lower' or 'mixed'	mixed
timePattern	The pattern to render time values	"HH:mm:ss"
timestampPattern	The pattern to render timestamp values	"yyyy-MM-dd'T'HH:mm:ss.SSSSSSSS"
timestampCapitalization	The capitalization to use when rendering a month name in a timestamp: 'upper', 'lower' or 'mixed'	mixed
decimalPattern	The pattern to render decimal values	System default
decimalSeparator	The decimal separator to use for decimal values	System default
integralPattern	The pattern to integral number values	System default

22.5.8 FixedWidthEntityExporter

Class Name	FixedWidthEntityExporter
Import	<code><import platforms="fixedwidth"/></code>
Class Description	Exports entities to a fixed column width file.

Property	Property Description	Default Value
uri	The URI of the file to create	"export.fcw"
columns	A comma-separated list of column format specifications	
encoding	The character encoding to use for the file	System default
lineSeparator	The line separator to use in the generated file	System default
append	If set to true, data is appended to existing files, otherwise existing files are overwritten	false
nullString	Text to represent <i>null</i> values	""
datePattern	The pattern to render date values	"yyyy-MM-dd"
dateCapitalization	The capitalization to use when rendering a month name in a date: 'upper', 'lower' or 'mixed'	mixed
timePattern	The pattern to render time values	"HH:mm:ss"
timestampPattern	The pattern to render timestamp values	"yyyy-MM-dd'T'HH:mm:ss.SSSSSSSS"
timestampCapitalization	The capitalization to use when rendering a month name in a timestamp: 'upper', 'lower' or 'mixed'	mixed
decimalPattern	The pattern to render decimal values	System default
decimalSeparator	The decimal separator to use for decimal values	System default
integralPattern	The pattern to integral number values	System default

The line format is described as a comma-separated list of property names with format spec, e.g. name[20],age[3r],points[5.2r0]. The format spec consists of

- [] brackets
- the (required) column width
- an optional alignment flag l, r or c (for left, right, center), left by default
- an optional pad character, space by default

So a property configuration of name[20],age[3r],points[5.2r0] would resolve to three columns,

first, a name entry, padded to 20 columns using spaces (default), aligned to the left (default)

second, an age entry, padded to 3 columns using spaces (default), aligned to the right

third, a points column, padded to 5 columns using zeros, having two fraction digits, aligned to the right

and would be rendered like this:

```
Alice Hamilton 2310.05
Bob Durand 4601.23
Helmut Schmidt 10226.14
```

22.5.9 XLSEntityExporter

Class Name	XLSEntityExporter	
Import	<code><import platforms="xls"/></code>	
Class Description	Exports entities to Excel XLS files. For using this exporter you need to add the Apache POI library to the Benerator's lib directory.	
Property	Property Description	Default Value
uri	The URI of the file to create	"export.xls"
nullString	Text to represent <i>null</i> values	""

22.5.10 CSVEntityExporter

Class Name	CSVEntityExporter	
Import	<code><import platforms="csv"/></code>	
Class Description	Exports entities to a CSV file	
Property	Property Description	Default Value
uri	The URI of the file to create	"export.csv"
columns	A comma-separated list of column names	
headless	Flag to leave out column headers	false
separator	The character to use as column separator	","
encoding	The character encoding to use for the file	System default
lineSeparator	The line separator to use in the generated file	System default
endWithNewLine	Specifies if the last row in the file should end with a line break	false
append	If set to true, data is appended to existing files, otherwise existing files are overwritten	false
nullString	Text to represent <i>null</i> values	Empty string
quoteEmpty	When set to 'true', empty strings are formatted with double quotes ("",""), otherwise an empty field (,)	false
datePattern	The pattern to render date values	"yyyy-MM-dd"
dateCapitalization	The capitalization to use when rendering a month name in a date: 'upper', 'lower' or 'mixed'	mixed
timePattern	The pattern to render time values	"HH:mm:ss"
timestampPattern	The pattern to render timestamp values	"yyyy-MM-dd'T'HH:mm:ss.SSSSSSSS"
timestampCapitalization	The capitalization to use when rendering a month name in a timestamp: 'upper', 'lower' or 'mixed'	mixed
decimalPattern	The pattern to render decimal values	System default
decimalSeparator	The decimal separator to use for decimal values	System default
integralPattern	The pattern to render integral number values	System default

22.5.11 SQLEntityExporter

Class Name	SQLEntityExporter	
Import	<code><import platforms="db"/></code>	
Class Description	Exports entities as 'INSERT' commands to a SQL file	
Property	Property Description	Default Value
uri	The URI of the file to create	"export.sql"
encoding	The character encoding to use for the file	System default
lineSeparator	The line separator to use in the generated file	System default
append	If set to true, data is appended to existing files, otherwise existing files are overwritten	false
dialect	The SQL dialect to use in the generated file. Available values: db2, derby, firebird, hsql, h2, oracle, postgres, sql_server	
datePattern	The pattern to render date values	"yyyy-MM-dd"
timePattern	The pattern to render time values	"HH:mm:ss"
timestampPattern	The pattern to render timestamp values	"yyyy-MM-dd'T'HH:mm:ss.SSSSSSSS"
decimalPattern	The pattern to render decimal values	System default
decimalSeparator	The decimal separator to use for decimal values	System default
integralPattern	The pattern to integral number values	System default

22.6 EntitySources (Importers)

Generators provides the following implementations of the EntitySource interface:

22.6.1 DbUnitEntitySource

Class Name	DbUnitEntitySource	
Import	<code><import platforms="dbunit"/></code>	
Class Description	Imports entities from a DbUnit XML file	
Property	Property Description	Default Value
uri	The URI of the file to read	"export.sql"

22.6.2 CSVEntitySource

Class Name	CSVEntitySource	
Import	<code><import platforms="csv"/></code>	
Class Description	Imports entities from a CSV file	

Property	Property Description	Default Value
uri	The URI of the file to read	"export.sql"
encoding	The character encoding used in the file	System default
separator	The line separator used in the file	","
columns	When set, the input file is expected to have no header row	

22.6.3 FixedColumnWidthEntitySource

Class Name	CSVEntitySource	
Import	<code><import platforms="fixedwidth"/></code>	
Class Description	Imports entities from a fixed column width file	

Property	Property Description	Default Value
uri	The URI of the file to read	"export.sql"
encoding	The character encoding of the file	System default
columns	The columns specification (See the FixedWidthEntityExporter for documentation)	

22.6.4 XLSEntitySource

Class Name	XLSEntitySource	
Import	<code><import platforms="xls"/></code>	
Class Description	Imports entities from an Excel(TM) document	

Property	Property Description	Default Value
uri	The URI of the file to read	

22.7 Benerator Utility Classes

22.7.1 RandomUtil

Class Name	RandomUtil
Import	<code><import class="com.rapiddweller.benerator.util.RandomUtil"/></code>
Class Description	Provides basic random functions

Method	Method Description
randomLong(min, max)	Returns a random long between min (inclusively) and max (inclusively)
randomInt(min, max)	Returns a random int between min (inclusively) and max (inclusively)
randomElement(value1, value2, ...)	Returns a random element of the ones listed as parameters
randomElement(List values)	Returns a random element from the 'values' list
randomIndex(List values)	Returns a random index for the 'values' list
randomDigit(min)	Returns a random numerical character with a value of at least min. Example: randomDigit(1) produces characters between '1' and '9'.
randomProbability()	Returns a float between 0 and 1
randomDate(min, max)	Returns a random date between min (inclusively) and max (inclusively)
randomFromWeightLiteral(literal)	Evaluates the weight literal and returns one of the specified values with the specified probability. Example literal: 'A'^3,'B'^1 will produce 75% of 'A' values and 25% of 'B' values.

22.8 rapiddweller Commons Library

The library rd-lib-common derived from Databene Commons from Volker Bergmann is a general-purpose utility collection which also provides some features useful for data generation and manipulation. Its converters and validators are listed above, but there are some general utility classes too. They can be invoked directly using rapiddwellerScript or other supported script languages.

22.8.1 TimeUtil

Class Name	TimeUtil
Import	<code><import class="com.rapiddweller.common.TimeUtil"/></code>
Class Description	Provides time and date related utility methods

Method	Method Description
currentYear()	returns the current year as integer value
today()	returns the current day as date object
yesterday()	returns the previous date as date object
tomorrow()	returns the next day as date object
date(year, month, day)	creates a date object for the specified day in the user's default time zone. 'month' is a zero-based integer, January = 0, February = 1, ...
gmtDate(year, month, day)	creates a date object for the specified day in the time zone GMT. 'month' is a zero-based integer, January = 0, February = 1, ...
date(year, month, day, hours, minutes, seconds, milliseconds)	creates a date object for the specified day and time in the user's default time zone. 'month' is a zero-based integer, January = 0, February = 1, ...
date(millis)	creates a date object in the user's default time zone. The time is specified as milliseconds since 1970-01-01
year(date)	returns the year of the specified date as integer
month(date)	returns the month of the specified date as integer
dayOfMonth(date)	returns the day of month of the specified date as integer
firstDayOfMonth(date)	returns the first day of the specified date's month
lastDayOfMonth(date)	returns the last day of the specified date's month
millis(year, month, day, hour, minute, second)	Calculates the number of milliseconds since 1970-01-01. 'month' is a zero-based integer, January = 0, February = 1, ...
time(hour, minute, second)	Creates a time object for the specified time
time(hour, minute, second, millisecond)	Creates a time object for the specified time
timestamp(year, month, day, hour, minute, second, nanosecond)	Creates a timestamp value for the specified time. 'month' is a zero-based integer, January = 0, February = 1, ...
currentTime()	Creates a time object representing the current time
midnightOf(date)	Rounds down a date value that may include a time to a value that represents midnight (time = 0)
addDays(date, noOfDays)	Calculates a date a given number of days past a given date
addMonths(date, noOfMonths)	Calculates a date a given number of months past a given date
addYears(date, noOfYears)	Calculates a date a given number of years past a given date

22.8.2 Period

Class Name	Period
Import	<code><import class="com.rapiddweller.common.Period"/></code>
Class Description	Provides constants for some time periods

Invocation	Description
Period.SECOND.millis	The number of milliseconds in a second
Period.MINUTE.millis	The number of milliseconds in a minute
Period.HOUR.millis	The number of milliseconds in an hour
Period.DAY.millis	The number of milliseconds in a day
Period.WEEK.millis	The number of milliseconds in a week

22.8.3 IOUtil

Class Name	IOUtil
Import	<code><import class="com.rapiddweller.common.IOUtil"/></code>
Class Description	Provides I/O related utility methods

Method	Method Description
isURIAvailable(uri)	Tells if the file specified by the given URI exists
getContentOfURI(uri)	Provides the content of the specified file as string
getBinaryContentOfUri(uri)	Provides the content of the specified file as byte array
getParentUri(uri)	Determines the parent URI (folder) of the specified URI
getProtocol(uri)	Determines the protocol specified in the URI
download(sourceUrl, targetFile)	Downloads the content of a remote URI to the local file system
copyFile(sourceUri, targetUri)	Copies a file on the local file system

22.8.4 CharUtil

Class Name	CharUtil
Import	<code><import class="com.rapiddweller.common.CharUtil"/></code>
Class Description	Provides character related utility methods

Method	Method Description
ordinal(character)	Returns a character's ordinal as integer
character(ordinal)	Returns the character that corresponds to the given ordinal

23. Using DB Sanity

You can call Databene DB Sanity for checking preconditions before starting data generation and for verifying properness of the generated data in the end. For detailed information about DB Sanity, check its project homepage: <https://sourceforge.net/projects/dbsanity/>. To use its functionality in Benerator, download and install dbsanity4ben from <https://sourceforge.net/projects/dbsanity/files/>.

For calling DB Sanity in a Benerator descriptor file, you define the checks and put them into a single XML file or distribute them over several ones and put them into a subdirectory of your Benerator project, typically called 'dbsanity'.

Import the plugin functionality to your Benerator project using

```
<import platform="dbsanity"/>
```

Then you can call DB Sanity providing a reference to a database you have declared before:

```
<database id="db" ... />
<dbsanity database="db" />
```

Alternatively, you can specify the environment and use a new database connection to perform data verification:

```
<dbsanity environment="mytestdb" />
```

You have the following configuration options:

Option	Description	Default Value
environment	The environment name with the configuration of the database to verify (see 'Database' for environment definition).	-
database	The database to verify	-
in	The directory from which to read the	dbsanity
out	The directory in which to put the report	dbsanity-report
appVersion	An application version for which to perform the checks	*
tables	Comma-separated list of tables on which to restrict the checks	-
tags	Comma-separated list of tags on which to restrict the checks	-
skin	The DB sanity skin to use for reports	online
locale	The locale in which to render values	default locale
mode	DB Sanity's execution mode	default
onError	Configures how to react to a requirements violation	See 'Error Handling'

24. Maven Benerator Plugin

The Benerator plugin enables you to attach Benerator to your build cycle or simply use maven and its configuration capabilities for benerator setup and invocation.

You can use the plugin to

- Invoke Benerator with a descriptor file.
- Create XML files from an XML Schema file, supporting XML Schema annotations for generation setup.

24.1 System Requirements

- Maven 3.x or newer
- JDK 11 or newer

24.2 Getting started

What ever goal you want to accomplish with Benerator Maven Plugin, you need to create a Maven project first. If you are about to create a completely new project, you may want to make use of Benerator's Maven Project Wizard. Otherwise, you need to configure the benerator plugin in your project manually. The minimal configuration in Maven's pom.xml would be:

```
<build>
...

<plugins>
<plugin>
  <groupId>com.rapiddweller</groupId>
  <artifactId>benerator-maven-plugin</artifactId>
  <version>2.0.0</version>
</plugin>
</plugins>

</build>
```

In order to make use of a plugin, it must be listed as dependency, e.g. dbsanity4ben and mongo4ben:

```
<dependencies>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>${database_postgresql.version}</version>
</dependency>
...
</dependencies>
```

When using proprietary database drivers (e.g. Oracle), you need to fetch and store them in your Maven repository manually and stated them as dependency in the pom.xml.

The default descriptor path points to the file benerator.xml in the project's base directory. So put your benerator.xml into this directory and invoke from the command line:

```
mvn benerator:generate
```

Voilà!

24.3 Common configuration elements

You may configure certain aspects of benerator behavior, e.g. file encoding and scope in a `<configuration>` element in your pom.xml's plugin configuration, e.g.:

```

<plugin>
  <groupId>com.rapiddweller</groupId>
  <artifactId>benerator-maven-plugin</artifactId>
  <version>2.0.0</version>

  <configuration>
    <encoding>utf-8</encoding>
    <scope>test</scope>
  </configuration>
</plugin>

```

These options are applicable for all goals of the Maven Benerator plugin. Their meaning is:

- **scope**: the scope of the project dependencies to include in the classpath when running benerator. It can be runtime or test. If left out, it defaults to runtime
- **encoding**: the file encoding to use by default

24.4 Executing descriptor files

The configuration elements available for descriptor file execution are:

- **descriptor**: the path to the benerator descriptor file
- **validate**: turn internal (XML and data model) validations off or on
- **dbDriver**: the JDBC driver class to use, this setting is provided to benerator as variable dbDriver
- **dbUrl**: the JDBC driver class to use, this setting is provided to benerator as variable dbUrl
- **dbSchema**: the database schema to use, this setting is provided to benerator as variable dbSchema
- **dbUser**: the database user name, this setting is provided to benerator as variable dbUser
- **dbPassword**: the database user's password, this setting is provided to benerator as variable dbPassword

You can invoke descriptor file execution by calling the **generate** goal from the command line or your IDE:

```
mvn benerator:generate
```

The db* configuration is available to scripts in your descriptor file as well, e.g. `<database url="{dbUrl}"... />`

24.5 Creating XML files from XML Schema files

Use these `<configuration>` elements in your pom.xml:

- **xmlSchema**:
- **xmlRoot**:
- **filenamePattern**:
- **fileCount**:

Then invoke XML file generation using the **createxml** goal:

```
mvn benerator:createxml
```

24.6 Creating a project assembly

For being able to port a Maven project to a location in which no Maven installation is available or possible, you can make the plugin collect all dependencies in one directory and create a classpath file. Call

```
mvn benerator:assembly
```

and you will find the project files, all dependent binaries and a `classpath.txt` file in the directory `target/assembly`. The `classpath.txt` helps you to set up a classpath definition for your target execution environment more easily.

24.7 Extending the classpath

If you need to extend the classpath to libraries different to your project dependencies, you can add them as dependencies to your plugin configuration (this requires Maven 2.0.9 or newer):

```
<plugin>

  <groupId>com.rapiddweller</groupId>
  <artifactId>benerator-maven-plugin</artifactId>
  <version>2.0.0</version>

  <configuration>
    ...
  </configuration>

  <dependencies>

    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>${database_postgresql.version}</version>
    </dependency>

  </dependencies>

</plugin>
```

24.8 Profile-based configuration

In cooperative software development, you are supposed to keep your individual configuration private. E.g. you might have individual database configurations on your local development systems. You can then specify them as profile properties in a Maven `settings.xml` file in your user directory.

```
<profiles>

  <profile>
    <id>development</id>

    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>

    <properties>
      <database.driver>oracle.jdbc.driver.OracleDriver</database.driver>
      <database.url>jdbc:oracle:thin:@localhost:1521:XE</database.url>
      <database.user>user</database.user>
      <database.pwd>user</database.pwd>
    </properties>

  </profile>

</profiles>
```

You would then refer them in your `pom.xml`:

```
<plugin>
  <groupId>com.rapiddweller</groupId>
  <artifactId>benerator-maven-plugin</artifactId>
  <version>2.0.0</version>

  <configuration>
    <descriptor>src/test/benerator/myproject.ben.xml</descriptor>
    <encoding>ISO-8859-1</encoding>
    <dbDriver>${database.driver}</dbDriver>
    <dbUrl>${database.url}</dbUrl>
    <dbUser>${database.user}</dbUser>
    <dbPassword>${database.pwd}</dbPassword>
    <dbSchema>${database.user}</dbSchema>
  </configuration>

</plugin>
```

24.9 Attaching the Mojo to the Build Lifecycle

You can also configure the benerator plugin to attach specific goals to a particular phase of the build lifecycle. Here is an example:

```
<build>

  <plugins>

    <plugin>
      <groupId>com.rapiddweller</groupId>
      <artifactId>benerator-maven-plugin</artifactId>
      <version>2.0.0</version>

      <executions>

        <execution>
          <phase>integration-test</phase>
          <goals>
            <goal>generate</goal>
          </goals>
        </execution>

      </executions>

    </plugin>

  </plugins>

</build>
```

This causes the Benerator goal 'generate' to be executed whenever integration tests are run. For more information on binding a plugin to phases in the lifecycle, please refer to the Build Lifecycle documentation.

For more information, see Maven's "Guide to Configuring Plug-ins": <http://maven.apache.org/guides/mini/guide-configuring-plugins.html>

Known Issues : Exception after successful run (#1)

25. Extending Benerator

Benerator can be customized in many ways. It provides Service Provider Interfaces which you can implement in Java for introducing own behavior. These are

- **Generator** generates attribute data or entities with specific characteristics
- **Sequence** lets you define your own random or sequence algorithms
- **WeightFunction** allows you to provide a weight function that determines the probability of a certain value.
- **CumulativeDistributionFunction** allows you to provide a weight function that determines the probability of a certain value.
- **Converter** converts data of one type to another and can be used to support custom data classes, data formats or message formats.
- **Validator** validates previously defined data and tells if it is valid. This is useful for low-knowledge data generation where you have e.g. a validation library or system but little knowledge how to construct valid data. In this case, you can generate random data and let the validator decide, which to accept.
- **Consumer** receives the generated data and is typically used to store it in a file or system.
- **EntitySource** allows the user to import predefined entity data, e.g. from files with custom data formats.
- **DescriptorProvider** reads metadata from systems or files and provides them as Benerator descriptors. You can define an own DescriptorProvider for e.g. importing generation characteristics from annotations in an XMI file.
- **StorageSystem** provides access to a system for storing and querying data. This would be the interface to implement for connecting to your application, e.g. SAP, Siebel or a custom one.
- **Task** is an interface for executing custom Java code.

You can as well plug into the script frameworks that you are binding. So, for FreeMarker, you can implement custom methods in Java and call them from the Benerator generation process.

25.1 Custom Generators

25.1.1 Generator Interface

com.rapiddweller.benerator.Generator is the basic Generator interface. It has the following methods:

- **Class <E> getGeneratedType():** Tells the framework of which Java class the generated values are.
- **void init(GeneratorContext context):** This is called to complete a Generators configuration and initialize it. If the configuration is not alright, the init() method is expected to throw an InvalidGeneratorSetupException. If the method finishes without exception, the generator has to be in state running. The next invocation of generate() is expected to return a valid product.
- **boolean wasInitialized():** Tells, if the init() method has already been called. Since Benerator massively uses generator chaining and nesting, this is useful to avoid double initialization for each generator instance.
- **ProductWrapper <E> generate(ProductWrapper <E> wrapper):** Generates an instance of the generic type E and uses the wrapper provided by the caller to return it to the client. If the method is called in an inappropriate state (created or closed), it throws an IllegalGeneratorStateException. If the generator is not available anymore, the method returns null.
- **void reset():** Resets the generator to the initial state. When called, the Generator is expected to act as if ' restarted'. After invocation the state has to be available.
- **void close():** Closes the generator. After invocation the state is unavailable.
- **boolean isThreadSafe():** Tells if the Generator class can safely be called concurrently by multiple threads.
- **boolean isParallelizable():** Tells if the Generator can be cloned and each instance be executed with a dedicated single thread.

25.1.2 Generator States

Generators have the following life cycle:

- **created:** The generator is under construction. This may take several steps, since generators need to be JavaBeans. When setup is done, a Generator must be initialized by calling its `init()` method.
- **running:** Generator construction is done and the generator is available. The user may use the Generator calling the `generate()` method.
- **closed:** The Generator may become unavailable automatically if its value space is depleted or manually when `close()` has been invoked. The Generator may be reset to the running state again by calling `reset()`. When being closed, the generator must be in a state in which it can be safely garbage collected.

25.1.3 Helper classes for custom Generators

It is recommendable to make your custom generator extend one of the following classes:

- **AbstractGenerator:** Implements state handling
- **UnsafeGenerator:** Implements state handling and declares to be neither thread-safe nor parallelizable
- **ThreadSafeGenerator:** Implements state handling and declare to be thread-safe and parallelizable

When deriving a custom generator, prefer delegation to inheritance. This simplifies code maintenance and life cycle handling. Abstract Generator implementations which already implement delegate handling are provided by the following classes:

- **GeneratorWrapper:** Wraps another generator of different product types
- **GeneratorProxy:** Wraps another generator of the same product type
- **MultiGeneratorWrapper:** Wraps multiple other generators, e.g. for composing their products, or arbitrarily choosing one of them for data generationGeneratorWrapper for wrapping a single delegate generator of different product type.

25.2 Custom FreeMarker methods

You can define custom functions in Java which will be called by FreeMarker, e.g. a `helloWorld` function which could be used like this:

```
<attribute name="greeting" script="{helloWorld(Volker)}"/>
```

See the FreeMarker documentation at http://freemarker.sourceforge.net/docs/pgui_datamodel_method.html for some more details. The Java implementation of the class could be something similar to this:

```
public class HelloWorldMethod implements TemplateMethodModel {
    public TemplateModel exec(List args) throws TemplateModelException {return new SimpleString("Hello " + args[0]);}
}
```

A descriptor file would need to instantiate the class, before it can be called:

```
<bean id="helloWorld" class="HelloWorldMethod" />
```

Unfortunately, anything you provide in the method call will be converted to a List of Strings, so date or number formatting may be necessary on the descriptor side and String parsing on the Java Method side. If the result type of the method is not a `SimpleString` the same conversions will be done. so you might need to use strange expressions, e.g. for a method that sums up dates and returns a date:

```
<attribute name="lastupdatedtime" script="${dateSum(deal.created_time?string('yyyy-MM-dd'), deallog._delay)?string('yyyy-MM-dd')}" />
```

25.3 Custom Sequences

Since a Sequence's primary concern is number generation, a Sequence implementor can focus on number generation. By inheriting from the class Sequence, one inherits the data redistribution feature defined in this class and only needs to implement

```
<T extends Number> Generator<T> createGenerator(Class<T> numberType, T min, T max, T granularity, boolean unique);
```

The method needs to be implemented in a way that it creates and returns a new Generator component, which generates numbers of the given numberType with a numerical value of at least min and at most max and a granularity, such that each generated value x is $x = \text{min} + n * \text{granularity}$, with n being an integral number. If the caller of this method indicated that it requires unique number generation and the Sequence is not capable of generating unique numbers, the method is required to throw a

```
com.rapiddweller.ConfigurationError.
```

25.4 Custom WeightFunctions

A WeightFunction specifies a probability density for a numeric parameter. If applied to a number generator it will generate numbers with the same probability density. When applied to collections or arrays, the function will evaluate to the probability of an element by its index number.

For defining your own WeightFunction, you just need to implement the WeightFunction interface:

```
public interface WeightFunction extends Distribution { double value(double param);}
```

Attention: Take into account, that the parameter may become zero: When using the Function for weighing the entries of an import file or a list, the function will be called with zero-based indexes as argument. So, if you want to use a 10,000-element CSV-file weighted by a custom WeightFunction, it must be able to produce useful values from 0 to 9,999.

Example:

```
public class GaussianFunction extends com.rapiddweller.benerator.distribution.AbstractWeightFunction {

    private final double average;

    private final double deviation;

    private final double scale;

    public GaussianFunction(double average, double deviation) {
        this.average = average;
        this.deviation = deviation;
        this.scale = 1. / deviation / Math.sqrt(2 * Math.PI);
    }

    @Override
    public double value(double param) {
        double x = (param - average) / deviation;
        return scale * Math.exp(-0.5 * x * x);
    }

}
```

25.5 Applying a Weight Function

You can weigh any arbitrary imported or numeric data by a Weight Function. A Weight Function is defined by a class that implements the interface `com.rapiddweller.model.function.WeightFunction`:

```
public interface WeightFunction extends Weight {

    double value(double param);

    ...

}
```

When using a weight function, Benerator will serve data items in random order and as often as implied by the function value. Benerator automatically evaluates the full applicable number range (as defined by numerical min/max or number of objects to choose from) and normalize the weights. There is no need to provide a pre-normalized distribution function. You may define custom Weight Functions by implementing the WeightFunction interface.

25.6 Custom Converters

Custom Converters can be used for supporting custom data types or specific data formats. The Converter interface has the following methods:

- **Class <S> getSourceType():** Returns the Java class whose instances can be converted
- **Class <T> getTargetType():** Returns the Java class to which objects are converted
- **T convert(S sourceValue):** Converts a sourceValue of type S to an object of type T
- **boolean isThreadSafe():** Tells if a converter can be executed with several concurrent threads
- **boolean isParallelizable():** Tells if a converter can be cloned and each clone can run with a single dedicated thread

These classes are useful parent classes for a custom Converter implementation:

- UnsafeConverter: Declares to be neither thread-safe nor parallelizable
- ThreadSafeConverter: Declares being thread-safe and parallelizable

Beyond this, a custom Converter should

- provide a public default (no-arg) constructor
- exhibit each relevant property by a public set-method

25.7 Custom Validators

A validator must implement at least one of these two interfaces:

- `javax.validation.ConstraintValidator` (as of JSR 303 - Bean Validation)
- `com.rapiddweller.common.Validator`

Beyond this, a custom validator should

- provide a public default (no-arg) constructor
- make each relevant property configurable by a set-method

25.7.1 `javax.validation.ConstraintValidator`

For implementing ConstraintValidators, see the Bean Validation documentation.

25.7.2 `com.rapiddweller.common.Validator`

For implementing the `com.rapiddweller.common.Validator` interface, a custom validator must implement the method `boolean valid(E object)` method returning `true` for a valid object, `false` for an invalid one.

It is recommended to inherit a custom Validator from the class `com.rapiddweller.common.validator.AbstractValidator`. If the Validator interface will change in future versions, the AbstractValidator will try to compensate for implementations of the old interface. Thus, a simple validator implementation which checks that an object is not null would be:

```
public class NotNullValidator<E> extends com.rapiddweller.common.validator.AbstractValidator<E> {
    public boolean valid(E object) { return (object != null); }
}
```

25.7.3 Implementing both interfaces

If you inherit a custom `Validator` from `com.rapiddweller.common.validator.bean.AbstractConstraintValidator`, it implements both interfaces.

25.8 Custom Consumers

A Consumer is the final destination that receives the data generated by Benerator. It implements the Consumer interface with the following methods:

- **startConsumption(ProductWrapper <E> object)** starts processing of a data object (usually an Entity)
- **finishConsumption(ProductWrapper <E> object)** finishes processing of a data object.
- **flush()** forces the consumer to forward or persist its data.
- **close()** closes the consumer

Consumers must be thread-safe.

Beyond this, it should

- provide a public default (no-arg) constructor
- make each relevant property configurable by a set-method

If your data format and generation supports nesting, the methods `startConsuming()` and `finishConsuming()` are called in a hierarchical manner. So, if A contains B, the invocation sequence is:

```
startConsuming(A)
```

```
startConsuming(B)
```

```
finishConsuming(B)
```

```
finishConsuming(A)
```

For non-hierarchical processing you only need to implement the `startConsuming()` with data processing logic.

25.9 Custom EntitySources

For defining a custom `EntitySource` you need to implement two methods:

- Class <Entity> `getType()`: Must return `Entity.class`
- `DataIterator <E> iterator()`: Must return a `DataIterator` which iterates over entities. If the iterator requires resource allocation, it should free its resources on `close()` invocation.

Beyond this, a custom `EntitySource` should

- provide a public default (no-arg) constructor
- make each relevant property configurable by a set-method

25.10 Custom Tasks

For executing custom Java code, the most efficient choice is to write a `JavaBean` class that implements the interface `com.rapiddweller.task.Task`.

The Task interface consists of callback interfaces for execution logic and lifecycle management.

- **String getTaskName():** returns a task name for tracking execution. On parallel execution, the framework appends a number to identify each instance.
- **TaskResult execute(Context context, ErrorHandler errorHandler):** implements the core functionality of the task. Tasks may be called several times subsequently and the task uses return value to indicate whether it is running, has just finished or is unavailable.
- **void pageFinished():** is called by the framework to inform the task that a 'page' of user-defined size has finished. This can be used, e.g. for grouping several execute() steps to one transaction.
- **void close():** is called by the framework to make the task close all resources and prepare to be garbage-collected.
- **boolean isParallelizable():** Tells if the task can be executed in several threads concurrently. This is not used in Benerator.
- **boolean isThreadSafe():** Tells if it is possible to create clones of the task and execute each clone in a dedicated single thread. This is not used in Benerator.

Benerator provides several implementations of the Task interface which are useful as parent class for custom implementations:

- AbstractTask is a simple abstract implementation that provides useful default implementations for the lifecycle-related methods.
- TaskProxy can be used to wrap another Task object with a custom proxy and add extra functionality.
- RunnableTask can be used to wrap an object that implements the Runnable interface.

A task has access to all Java objects and entities in the context and may arbitrarily add own objects (e.g. for messaging between different tasks). A common usage pattern is to share e.g. a transactional database by the context, have every thread store objects in it and then use the pager to commit the transaction.

26. Using Benerator as Load Generator

Benerator has done the first implementation steps towards being itself used as load generator, not only data generator for preparing load test data.

The basic idea behind the approach is to use a `<generate>` element for composing invocation parameters, and use special 'Consumer' or 'Task' implementations for doing the actual invocation of the system under test.

Performance of task execution can be tracked, setting a stats attribute to true:

```
<run-task class="com.my.TaskImpl" invocations="10000" threads="100" stats="true" />
```

The performance of consumer invocations can be tracked, using the `PerfTrackingConsumer` class:

```
<setup>
  <import platforms="contiperf" />

  <bean id="myWS" spec="new MyWebServiceConsumer(...)" />

  <generate type="params" count="10000">
    <value type="int" min="100" max="10000" distribution="random"/>
    <consumer spec='new PerfTrackingConsumer(myWS)'/>
  </generate>
</setup>
```

26.1 JavaInvoker

To aid you in calling Java code from Benerator, there is a Helper class, `JavaInvoker`, which implements the `Consumer` interface and calls a Java method, which is declared by its name, on a Java object. The data of the generated entity is automatic:

```
<bean id="service" spec="..." />

<bean id="invoker" spec="new JavaInvoker(ejb, 'enrolCustomer')"/>
```

For tracking invocation performance, you need to add the `PerfTrackingConsumer`:

```
<bean id="enrolCustomer" class="PerfTrackingConsumer" >
  <property name="target" value="{invoker}" />
</bean>
```

26.2 Checking performance requirements

You can as well define performance requirements using properties of the class `PerfTrackingConsumer`:

Property	Description
max	the maximum number of milliseconds an invocation may take
percentiles	a comma-separated list of percentile requirements in the format used in ContiPerf

By default, execution times are printed to the console. You can as well plug in custom `ExecutionLoggers` for saving log data to file or feed it to other applications. This is done using the `PerfTrackingConsumer`'s `executionLogger` property.

An example for the properties described above:

```
<import class="com.rapiddweller.contiperf.report.ConsoleReportModule" />

<bean id="enrolCustomer" class="PerfTrackingConsumer" >
  <property name="target" value="{invoker}" />
  <property name="executionLogger" value="{new ConsoleExecutionLogger()}" />
  <property name="max" value="5000" />
  <property name="percentiles" value="90:5000, 95:7000"/>
</bean>
```

27. Troubleshooting

27.1 (Out of) Memory

If you get an `OutOfMemoryError`, first increase the Java heap size by a `-Xmx` environment setting (e.g. by adding `-Xmx1024m` to the `BENERATOR_OPTS` in the script files).

Another potential cause for `OutOfMemoryErrors` is the application of distributions to very large data sets. Most sequences and all other types of distribution require the source data to fit into RAM. So either use an 'unlimited' sequence like 'expand', 'repeat' or 'randomWalk' or simply repeat data set iteration by adding `cyclic="true"` to the configuration.

27.2 temp Directory

In some environments, the temp directory has a very restrictive disk quota. If you need more space for data generation, you can specify another directory by the `-Djava.io.tmpdir` environment setting (e.g. by adding `-Djava.io.tmpdir=/User/me/mytemp` to the `BENERATOR_OPTS` in the script files)

27.3 File Encoding

If no file encoding was specified, benerator uses the default file encoding of the system it runs on - except if the file itself contains encoding info (like XML).

If all used files have the same encoding and it is different to your system's encoding, you can change set benerator's default encoding by the `-Dfile.encoding` environment setting (e.g. by adding `-Dfile.encoding=iso-8859-1` to the `BENERATOR_OPTS` in the script files)

When generating data in heterogeneous environments, it is good practice setting the `defaultEncoding` property of the benerator descriptor file's root element. If only single files have a different encoding, you can specify an encoding property for all built-in file importers and file-based consumers.

A typical error that may arise from wrong file encoding configuration is that file import (e.g. for a CSV file) stops before the end of file is reached.

27.4 Logging

Benerator logs its event using `slf4j` as a logging facade to integrate with the logging tools used by libraries which are used by Benerator. As logging implementation, `Apache Log4j 2` is used. For troubleshooting, it is useful to check and fine-tune logging configuration for tracking down your issue. This is done by editing the `log4j2.xml` file in your `BENERATOR_HOME/bin` directory. Some interesting logging categories to start with:

Set a category to debug for getting detailed information about its execution. The most important log categories are:

name	description
com.rapiddweller.benerator.main	Events of benerator's main classes, e.g. detailed information about which entities are currently generated
com.rapiddweller.benerator.STATE	generator state handling for information which component generator caused termination of the composite generator
com.rapiddweller.benerator.factory	Creating generators from descriptor information
com.rapiddweller.benerator	Top-level directory for all generators and main classes
com.rapiddweller.SQL	SQL commands, e.g. DDL, queries, inserts, updates
com.rapiddweller.JDBC	JDBC operations, e.g. connection / transaction handling
com.rapiddweller.jdbcapi.model.jdbc	Database meta data import
com.rapiddweller.platform.db	All database related information that does not fit into the SQL or JDBC category
com.rapiddweller.platform.xml	XML-related activities
com.rapiddweller.domain	benerator domain packages
com.rapiddweller.model.descriptor	related information
com.rapiddweller.common	low-level operations like data conversion

27.5 Locating Errors

When configuring data generation you are likely to encounter error messages.

Depending on the settings it may be difficult to find out what caused the problem. For tracking database-related errors, set `batch="false"` in your `<database>` setup and use `pagesize="1"` in the `<generate>`. These are default settings, so you do not need to specify them explicitly if you did not change the default.

If that alone does not help, set the log category `com.rapiddweller.benerator.main` to debug level to find out which element caused the error. If there is a stack trace, check it to get a hint of which part of the element's generation went wrong. If that does not help, remove one attribute/reference/id after the other for finding the actual troublemaker. If you still cannot solve the problem, post a message in the benerator forum. You can check out the benerator sources from the SVN source repository, open it in Eclipse and debug through the code.

27.6 Database Privilege Problems

When importing database metadata, you might encounter exceptions when Benerator tries to get metadata of catalogs or schemas it has no access privileges to.

Usually can fix this by choosing the right schema for your database, e.g.

```
<database id="db" ... schema="PUBLIC" />
```

If you are not sure which schema is applicable in your case, edit the logging configuration in `log4j.xml` (as described above) and set the category `com.rapiddweller.platform.db` to **debug**.

You will then get a list of schemas as Benerator scans the database metadata, e.g. for an Oracle system:

```
06:03:45,203 DEBUG [DBSystem] parsing metadata...
06:03:45,203 DEBUG [JDBC] opening connection to jdbc:oracle:thin:@10.37.129.3:1521:XE
06:03:45,226 DEBUG [JDBC] Created connection #4: oracle.jdbc.driver.T4CConnection@741827d1
06:03:45,385 DEBUG [JDBC] opening connection to jdbc:oracle:thin:@10.37.129.3:1521:XE
06:03:45,417 INFO [JDBCDBImporter] Importing database metadata. Be patient, this may take some time...
06:03:45,417 DEBUG [JDBCDBImporter] Product name: Oracle
06:03:45,419 INFO [JDBCDBImporter] Importing catalogs
06:03:45,430 INFO [JDBCDBImporter] Importing schemas
```

```

06:03:45,438 DEBUG [JDBCDBImporter] found schema ANONYMOUS
06:03:45,438 DEBUG [JDBCDBImporter] found schema DBSNMP
06:03:45,438 DEBUG [JDBCDBImporter] found schema DIP
06:03:45,438 DEBUG [JDBCDBImporter] found schema FLOWS_FILES
06:03:45,439 DEBUG [JDBCDBImporter] found schema FLOWS_020100
06:03:45,439 DEBUG [JDBCDBImporter] found schema HR
06:03:45,439 DEBUG [JDBCDBImporter] found schema MDSYS
06:03:45,440 DEBUG [JDBCDBImporter] found schema OUTLN
06:03:45,440 DEBUG [JDBCDBImporter] found schema SHOP
06:03:45,440 DEBUG [JDBCDBImporter] importing schema SHOP
06:03:45,441 DEBUG [JDBCDBImporter] found schema SYS
06:03:45,441 DEBUG [JDBCDBImporter] found schema SYSTEM
06:03:45,441 DEBUG [JDBCDBImporter] found schema TSMSYS
06:03:45,441 DEBUG [JDBCDBImporter] found schema XDB

```

Cross-checking this with your access information should make it easy to figure out which one is appropriate in your case.

See the [Usual Database Settings](#).

27.7 Constraint Violations

Some constraint violations may arise when using database batch with nested create-entities. Switch batch off. If the problem does not occur any more, stick with non-batch generation. Otherwise, you need further investigation. When using Oracle, a constraint violation typically looks like this:

```
java.sql.SQLException: ORA-00001: Unique Constraint (MYSCHEMA.SYS_C0011664) violated
```

It contains a constraint name you can look up on the database like this:

```
select * from user_constraints where constraint_name like '%SYS_C0011541%'
```

The query result will tell you the table name and the constraint type. The constraint types are encoded as follows:

P: Primary key constraint

U: Unique constraint

R: Foreign key constraint

27.8 'value too large for column' in Oracle

Depending on the character set, oracle may report a multiple of the real column with, e.g. 80 instead of 20. So, automatic generation of varchar2 columns may fail. This typically results in Exceptions like this:

```
java.sql.SQLException: ORA-12899: value too large for column "SCHEM"."TBL"."COL" (actual: 40, maximum: 10)
```

This is Oracle bug #4485954, see http://www.oracle.com/technology/software/tech/java/sqlj_jdbc/htdocs/readme_jdbc_10204.html and

<http://kr.forums.oracle.com/forums/thread.jspa?threadID=554236> .

The solution is using the newest JDBC driver, at least 10.2.0.4 or 11.0 . BTW: It is backward compatible with the Oracle 9 databases.

27.9 Time is cut off in Oracle dates

Oracle changed back and forth the mapping of internal types to JDBC times in Oracle 8 and 11, the mapping in Oracle 9 and 10 is wrong, see http://www.oracle.com/technetwork/database/enterprise-edition/jdbc-faq-090281.html#08_00 . In order to fix the behaviour, use the newest available Oracle 11 JDBC driver you can get, it is backward compatible down to Oracle 9 and provides a proper mapping of the date type for all Oracle database versions.

27.10 Composite Keys

Benerator expects single-valued ids. It does not automatically support composite keys and composite references.

Since composite keys typically have a business meaning, most composite keys cannot be automatically generated. So there is no need to support this.

If you encounter a composite key, manually configure how to create each key component.

27.11 Importing Excel Sheets

For a beginner it is sometimes confusing, how Benerator handles imported Excel sheets. For this task it completely relies on the cell type configured in the original sheet. So if you have a Date cell in the Excel sheet and format it as number or text, benerator will interpret it as double or string.

Another popular error comes from columns that contain long code numbers and have the default format: They are imported as numbers and e.g. leading zeros are lost. In such case explicitly format the column as text in Excel.

Apache POI represents all numbers as variables of type `double`. So there are numbers which are simple in decimal format but not so in binary: when importing the number 1.95 from an Excel sheet, the user gets a value of 1.95000000002. For now, you need to round the values yourself, e.g. by a converter.

27.12 Number Generation on Oracle

When generating very large decimal values ($> 1.000.000.000$) in an Oracle database you may observe that smaller numbers are written to the database, losing some trailing zeros or even cutting the whole number to a decimal with one prefix digit and several fractional digits. This results from a bug in Oracles older JDBC drivers and can be fixed by using the newest driver version (note that you can even use `11.x` JDBC drivers for `10.x` databases).

27.13 Unknown Column Type

If your application makes use of a database's proprietary column types, you may run into an exception when Benerator encounters it. If you know how to create and handle data for this column type, you can do so by configuring the database to accept unknown column types :

```
<database ... acceptUnknownColumnTypes="true">
```

27.14 Table 'X' not found in the expected catalog 'Y' and schema 'Z'

This message tells you, that your database configuration is wrong. Check and fix the 'schema' and 'catalog' settings in your database configuration, e.g.

```
<database ... catalog="main" schema="Z" />
```

or your environment configuration, e.g. `xyz.env.properties` :

```
db_catalog=main
db_schema=Z
```



Note: On most systems (e.g. Oracle, HSQL) no catalog needs to be specified.

27.15 Method isApplicationDetached() not found in class Xyz

See '[Migration from 1.1.x to 2.0.0](#)'.

28. Monitoring Benerator

You can monitor Benerator using a JMX client, for example, JConsole.

The following properties can be monitored:

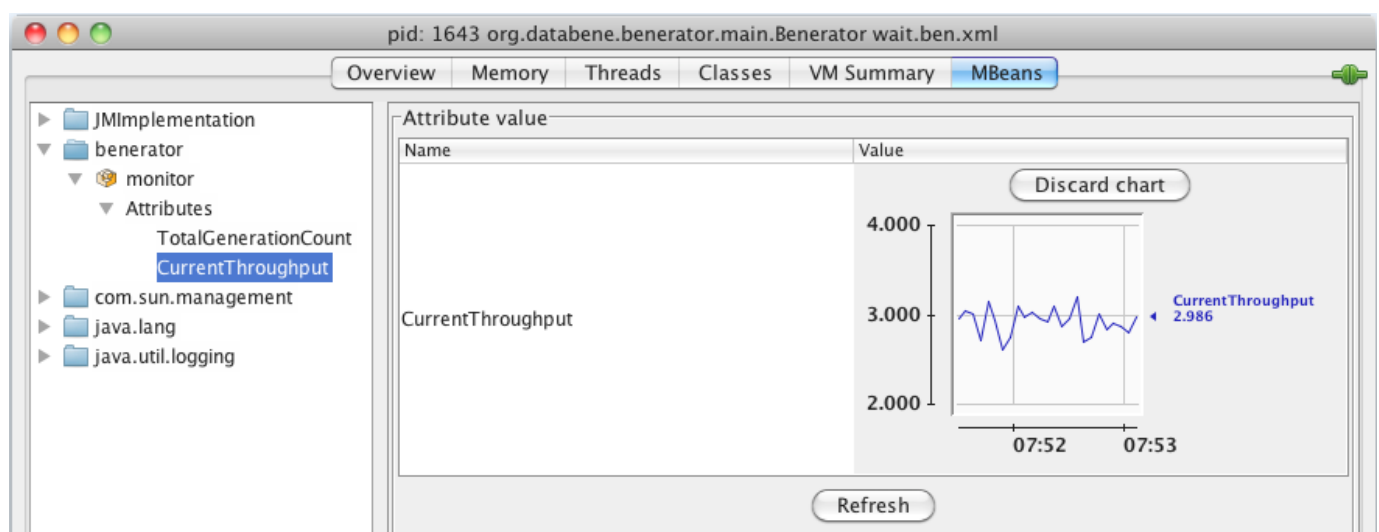
Property	Description
TotalGenerationCount	The total number of generated data sets
CurrentThroughput	The number of data sets generated per second
OpenConnectionCount	The number of currently open database connections
OpenResultSetCount	The number of currently open database query result sets
OpenStatementCount	The number of currently open database statements
OpenPreparedStatementCount	The number of currently open prepared database statements

The first two properties, **TotalGenerationCount** and **CurrentThroughput**, are used for Benerator performance monitoring and optimization. If you suspect Benerator to be 'hanging', first check its **CurrentThroughput**.

The last four properties (**Open...**) for database resource monitoring and database resource leak detection.

28.1 Monitoring with JConsole

1. Start JConsole on the command line
2. Select a process
3. Choose the MBeans tab
4. In the left tree view, select
5. benerator - monitor - Attributes
6. Select the attribute TotalGenerationCount or CurrentThroughput and the value is displayed on the right
7. Double-clicking the number opens a chart that displays the value's evolution over time



28.2 Remote monitoring

For monitoring Benerator execution from a remote machine, you need to set some BENERATOR_OPTS. Here are only the simplest and basic settings.

Warning: The settings described here do not provide any security and thus are recommended only for evaluation! If you do want to monitor a sensitive system in a remote manner, you need to apply security settings as described in <https://docs.oracle.com/en/java/javase/11/management/monitoring-and-management-using-jmx-technology.html>!

These are server-side settings and are independent of the client you are using:

Option	Description
-Dcom.sun.management.jmxremote	Enable remote access
-Dcom.sun.management.jmxremote.port=9003	Configures the port for remote access
-Dcom.sun.management.jmxremote.authenticate=false	Enables anonymous and unsecure access (not recommended)

29. Benerator Performance Tuning

29.1 Using Enterprise Edition

Benerator Enterprise Edition enables you to generate and anonymize data concurrently, in multiple threads. More than this, its engine and many specialized data generators had significant performance improvements compared to the Community Edition. See the [Benchmark Tool Documentations](#) to get some performance comparisons.

29.2 pageSize (database and other transactional systems only)

'pageSize' is Benerator's abstraction of a kind of bracket put around a group of data objects, for databases and Kafka usually this bracket is mapped to a transaction. For example a pageSize of 10 for a database means that each group of 10 objects is committed to the database in one transaction. Transactions incur processing overhead, so performance improves if the number of transactions is reduced. This is achieved by using a larger pageSize.

By default, Benerator uses a defensive pageSize of 1, processing each entity in an own transaction, because this simplifies error diagnosis when working out a generation/anonymization process. But this has a tremendous impact on performance. When configuration is complete, and you need larger performance, use a large pagesize for critical `<generate>` and `<iterate>` elements, e.g. `<generate type="db_user" count="1000000" consumer="db" pagesize="1000">`

29.3 JDBC batch (database only)

JDBC batches provide for significantly better database insertion performance than standard operation. In Benerator this is turned off by default, since error messages that arise from bad generation setup are much harder to analyze in batch mode. When you are finished with defining data generation and need performance for mass data, you can activate batch operation by the batch attribute of the database element:

```
<database ... batch="true" />
```

Benerator is optimized for performance. Thus, you may get problems when combining nested `<generate>` elements with batching. It typically results in exceptions that indicate a violation of a foreign-key constraint.

29.4 Query fetch size (database only)

If you are querying data with large result sets from your database, e.g. when anonymizing production data, you should tune the database's fetch size. It determines how many rows are transmitted from the database to Benerator when accessing the first query result. So it reduces the number of network roundtrips. Benerator uses a fetch size of 100 by default which should be useful in most cases.

You can experiment with higher values, e.g. 1000 by configuring the **fetchSize** and **batch** attributes of the database element:

```
<database ... fetchSize="1000" batch="true" />
```

This is especially useful if the database is accessed over a slow network connection and query result sets are at least as large as the fetch size and are iterated to a relevant extent. When setting the fetch size to a value that is too high, performance may actually decrease.

29.5 Restrict logging

Logging data generation/anonymization details may deteriorate performance tremendously. Benerator is shipped with efficient logging settings, but if you changed the settings in the process of debugging your data generation/anonymization, make sure to increase log filters to `info` level. A high level like 'error' usually do not have a performance impact, but may hide issues which are reported using 'warn', so you should not be too restrictive about logging.

29.6 Id Generation

The most efficient id generation strategy is increment since it works without connecting the database. It works fine for multithreaded generation, too. But for concurrent execution of multiple Benerator processes or continuation of a canceled generation process you need an id generation that is unique among several runs. The most efficient id strategies with such behavior are seqhilo (database-based) and uuid (universally unique string id).

29.7 Relational Data Generation (database)

Nesting `<generate>` elements greatly simplifies development and maintenance of a data generation project but can decrease generation speed. At critical generation steps you might want to sacrifice maintainability for speed by replacing nested structures with 'relational' structures, e.g. replacing this code:

```
<generate name="db_customer" count="1000000" consumer="db">
  <generate name="db_order" minCount="0" maxCount="20 " consumer="db">
    <attribute name="customer_fk" script="{db_customer.id}"/>
    ...
  </generate>
</generate>
```

with something like:

```
<generate name="db_customer" count="1000000" consumer="db" />

<generate name="db_order" count="10000000" consumer="db">
  <reference name="customer_fk" source="db" targetType="db_customer" distribution="random" />
  ...
</generate>
```

29.8 Java Virtual Machine (JVM) Products

Different Java Virtual Machines provide different performance. Changing the JVM may easily impact Benerator performance by 20%, up to more than 100% in exceptional cases.

You can choose among a wide range of JVM products and versions. Well known established and reliable products/providers are OpenJDK, OpenJ9 (IBM), GraalVM (Oracle) and Azul. Different versions of a product may also exhibit notably different performance.

Unfortunately, there is not an optimal JVM and version for any task you might want to perform: Some JVMs will perform significantly faster in single threaded execution but others will be better in multithreaded execution.

29.9 Virtualization Costs

Running Benerator in a virtualization system will always have a certain performance impact. For example, in 2020 Apple discontinued the use of Intel processors and introduced a new architecture using M1 chips in its Mac products. Programs written for the Intel architecture are still supported, but run with in a virtualization system, Rosetta. Though Rosetta's performance is impressive, there is a performance cost which gives JVMs compiled for M1 an advantage over JVMs for the Intel architecture on a Mac. In a simple non-representative test, Azul's M1 JVM was at least 20% faster than OpenJDK for Intel (on average).

The performance cost of a container like Docker is small and usually outweighed by the other advantages this software offers in enterprise computing.

29.10 Scripts

When using other script languages than rapiddwellerScript, the scripts expressions are parsed at runtime, thus are significantly slower than compiled code. For performing CPU-intensive operations or excessive looping, use rapiddwellerScript or program a Java task (See '[Custom Tasks](#)').

29.11 Parsing (Oracle) metadata

On databases with many tables, scanning metadata can take several minutes. One source of superfluous tables is the Oracle recyclebin. You can speed up parsing by including a 'purge recyclebin' in your SQL scripts.

Using regular expressions in the `<database>`'s `excludeTables` and `includeTables` settings, you can restrict the amount of metadata to be parsed.

If metadata retrieval still takes too long, you can use `<database... metaCache="true" />` for storing metadata in a cache file on your local hard drive.

When Benerator executes SQL with an `<execute>` statement, it analyzes if the database structure is modified. In that case, the cached database metadata is invalidated and parsed again when they are needed the next time. If you are certain that the change is irrelevant to subsequent generations steps, you can suppress the invalidation by

```
<execute invalidate="false"> ALTER TABLE... </execute>.
```

29.12 Anonymization Algorithms

Different anonymization algorithms have different performance: An anonymization that takes care of consistency and 'pretty' output useful for showcases must be slower than a stupid one which overwrites the imported data with purely random data.

Advice: Do not exaggerate on anonymization performance optimization! There are always independent data fields in a software, which will cause consistency errors and program failure in the target system if they have not been assessed properly. This is what prototype-based generation was invented for!

Only **after** you have set up anonymization that fits consistency requirements, checked for other JVMs, have applied other performance improvement strategies, maybe tried scaling over multiple threads in Enterprise Edition, you may fiddle around with anonymization of single fields according to the following information.

Here some anonymization approaches in the order of increasing performance:

Approach	Showcase Applicability	Perftest Applicability	Speed Factor
prototype	great	good	1
regex	often passable	passable	6
hash	poor	passable	7
random	poor	passable	9
constant	poor	poor	14

For showcase generation, in which a user is faced with anonymization results, only a prototype or regex approach makes sense.

When anonymizing data for performance tests, one concern is how good the results of the chosen anonymization match the real world's data characteristics, especially for fields used in caches and indexes. A constant approach will make indexes and caches useless as well as the performance test results.

Using Regex, Hash or Random will provide you with a bigger diversity of data than the real world, making the data slightly faster to look up in an index.

You see that the generation of constants is super-fast but useless in most cases.

In many cases, a regular expression is a good compromise between anonymization performance and applicability for showcases and performance tests.

For example, a setup like this

```
<iterate type='Person' source="..." ...>
  <attribute name='streetAndNumber' pattern='[A-Z][a-z]{4,10} Street [1-9][0-9]?'/>
  <attribute name='zip' pattern='[0-9]{5}'/>
  <attribute name='city' pattern='[A-Z][a-z]{4,15}'/>
```

```

<attribute name='email'           pattern='[a-z]{4,10}@[a-z]{4,10}\.(com|org|net)'/>
<attribute name='workPhone'       pattern='[0-9]{8,12}'/>

<attribute name='firstName'       pattern='[A-Z][a-z]{1,8}'/>
<attribute name='midName'         pattern='[A-Z][a-z]{1,8}'/>
<attribute name='lastName'        pattern='[A-Z][a-z]{1,12}'/>

<attribute name='CardExp'         pattern='([1-9])|(1[0-2])/2[1-9]'/>
<attribute name='Cardno'          pattern='[0-9]{16}'/>
</iterate>

```

produces results like this

```

Person[
  streetAndNumber Hxpji Street 3
  zip             27200
  city            Ccxowbyozqti
  email           likoakqcy@nyelekuki.org
  workPhone       599209827
  firstName       Ogszcy
  midName         Jacxmni
  lastName        Pnn
  CardExp         12/24
  Cardno          2390195664099922
]

```

You see that usually regex expressions are a viable approach for generating codes like zip codes and phone numbers, but relatively poor for name-like data, but often acceptable.

In order to get an idea of the costs of different anonymization approaches, check the performance numbers in the [Benchmark Tool Documentation](#) or run it on your system to get an individual insight into your machine's capabilities.

29.13 Benerator Modes

Benerator can run in different modes, which have different performance and strictness characteristics.

A mode is activated by the command line argument `--mode`, eg.

```
benerator --mode turbo
```

29.13.1 lenient

The default mode, Benerator is running in. The full feature set is supported and some configuration issues are handled automatically. This is best for users at the beginning of the learning curve.

29.13.2 strict

Benerator stops on configurations which may have performance implications or may possibly be unintended, assuring the full feature set can run with optimum performance. In many cases this will be faster than lenient mode, but requires more knowledge about Benerator.

29.13.3 turbo

Experimental feature: The fastest mode which in which Benerator can run, but this comes at a price: It uses a restricted feature set: capitalization of variables is ignored and simple-type sub elements may not occur multiple times. This will work fine for most projects in most homogeneous environments, involving only databases and plain file formats. For strongly heterogeneous projects, especially involving XML this may produce unintended results. Use this only when you are desperate for performance and have made sure everything works fine in `strict` mode.

29.14 Distributed Generation

You can distribute generation over several machines. In order to do so you will need two types of descriptor files:

First an initialization descriptor that initializes the systems and generates the deterministic core data,

Second a mass data description file. The initialization file will need to run first on a single machine, then the mass data file can be executed on multiple processes or machines concurrently. For mass data generation you will need to take special care:

Choose an id generation strategy that is able to create unique ids under these circumstances (see '[Generating IDs](#)').

29.15 Distributed Anonymization

You can distribute anonymization over several machines. A simple and scalable approach is the use of a Kafka cluster: Using a Kafka connector, you can export the data you need to anonymize(eg. from a database table) to a Kafka queue and run several machines with Benerator instances that listen to the Kafka queue, import data, anonymize it and export it to the destination system directly or indirectly via another Kafka queue and Kafka connector.

29.16 Ask the experts

Feel free to contact us for assistance on performance improvement: team@rapiddweller.com.

30. Migrating Benerator Projects

30.1 Guidelines

We try to keep execution behaviour and programming interfaces as stable as possible between releases. However, if we consider a behavior as wrong, we do change it.

If you experience generation behaviour changes between Benerator releases, please check this migration guide section in its newest version. Often, we will provide a flag in the `<setup>` attributes which make Benerator keep previous behavior.

30.2 General Advice For Developers of Custom Components

If we need to change an interface in order to improve it or to provide new features, we always try to adapt its direct child class(es) in a way that hides the change from existing implementations.

So, if you are programming custom implementations of Benerator's service provider interfaces (SPIs), it makes your code more stable against changes, if you inherit from a child class of this interface instead of implementing it directly.

30.3 Migration from 1.1.x to 2.0.0

If you did not program custom implementations of Benerator's interfaces or classes, existing projects are supposed to work like before (but faster).

If you programmed a custom distribution by implementing the Distribution **directly** (and did not inherit from a child class), the bad news is that your code will not be compatible with Benerator 2.0.0.

The good news is that this is easily fixed: Edit the Java code of your custom distribution class and replace the `implements Distribution` directive with `extends AbstractDistribution`.

In case you programmed a custom Sequence, you have two options:

1. If the sequence fetches and caches all generator data in its `applyTo()` method, then have `extends DetachedSequence`
2. else implement the `applyTo()` method appropriately.